Fundación Universitaria Internacional de La Rioja (UNIR)

Facultad de Ingeniería

Ingeniería Informática

Planificación de una arquitectura agéntica verificable para facturación electrónica multipaís en Latinoamérica

Trabajo de Fin de Grado

Presentado por: De Mendoza Tovar, Alejandro

Director: Giraldo Hurtado, Johan Sebastián

Ciudad: Bogotá D. C.
Fecha: octubre de 2026

Resumen

La factura electrónica con validación previa es hoy la base de la fiscalización digital en Latinoamérica. Cada país define su propio formato, sus catálogos de códigos y sus reglas de validación, y los modifica por versiones. Para una organización multinacional que factura en varios países, esto significa que cada país o proveedor tecnológico nuevo obliga a construir de nuevo el mapeo entre su modelo de datos canónico y el formato nacional, una tarea manual de aproximadamente cien campos que produce defectos recurrentes y que solo se detectan cuando el proveedor o la autoridad tributaria rechazan el documento.

El propósito de este trabajo es planificar la implementación de una arquitectura agéntica verificable que reduzca el esfuerzo de incorporar nuevos países y proveedores sin comprometer la validez normativa de los comprobantes. La arquitectura combina dos planos: un plano de integración orientado a eventos y un plano agéntico en el que modelos de lenguaje de gran tamaño proponen mapeos, plantillas y pruebas. Entre ambos se ubica un kit de verificación determinista que decide qué propuesta puede llegar a revisión humana, porque valida el esquema vigente, los catálogos oficiales versionados y las reglas derivadas de una taxonomía propia de seis tipos de defecto de mapeo, y relaciona cada regla con la sección del documento normativo que la origina.

La metodología sigue la investigación en ciencia del diseño, con el proceso DSRM como marco principal, y un ciclo de vida por fases con entregables, y se apoya en el análisis de causas, la decisión multicriterio, las pruebas de mutación y la estimación de tres puntos con simulación de Monte Carlo. Un prototipo funcional del kit, evaluado con un banco de 22 defectos inyectados, detectó el 86 % de los defectos en ejecución (intervalo de confianza del 95 %: 67 % a 95 %) y el 100 % con ejemplos validados en integración continua (85 % a 100 %), sin falsos positivos sobre 600 salidas correctas, frente al 23 % de la validación de esquema. Con un banco ampliado de 129 mutantes, ejecutado por el sistema real, la detección en ejecución fue del 93,8 % (88,2 % a 96,8 %), de modo que la meta del 80 % se cumple también con una confianza del 95 % sobre datos sintéticos, sin falsos positivos en 11.402 comprobantes correctos. El plan de ejecución, con diez personas en ocho roles, estima un plazo de 40,2 semanas en el percentil 50 y de 42,2 en el percentil 80, con un costo de COP 1.013 millones y COP 1.059 millones, y un punto de equilibrio estimado de 2,9 países para los agentes y de 5,4 para la inversión completa, bajo la hipótesis, aún no comprobada, de que el esfuerzo asistido es menor que el manual.

Palabras clave: facturación electrónica, agentes de inteligencia artificial, verificación determinista, arquitectura de software, planificación de proyectos.

Abstract

Clearance-based electronic invoicing is now the foundation of digital tax control in Latin America. Each country defines its own format, code lists and validation rules, and changes them through successive versions. For a multinational organization that issues invoices in several countries, every new country or e-invoicing provider means rebuilding the mapping between its canonical data model and the national format. This manual task covers about one hundred fields and produces recurring defects that are only detected when the provider or the tax authority rejects the document.

The purpose of this work is to plan the implementation of a verifiable agentic architecture that reduces the effort of onboarding new countries and providers without compromising the regulatory validity of the documents issued. The architecture combines two planes: an event-driven integration plane and an agentic plane in which large language models propose mappings, templates and tests. Between them sits a deterministic verification kit that decides which proposal may reach human review. It checks the current schema, the versioned official code lists and the rules derived from an original taxonomy of six mapping defect types, and it links every rule to the section of the regulatory document it comes from.

The methodology follows design science research, with DSRM as its main framework, and a phased life cycle with deliverables, supported by root cause analysis, multi-criteria decision analysis, mutation testing and three-point estimation with Monte Carlo simulation. A functional prototype of the kit, evaluated with a bank of 22 injected defects, detected 86% of the defects at run time (95% confidence interval: 67% to 95%) and 100% with validated examples in continuous integration (85% to 100%), with no false positives on 600 correct outputs, against 23% for schema validation alone. With an extended bank of 129 mutants run through the real system, run-time detection was 93.8% (88.2% to 96.8%), so the 80% target is also met at 95% confidence on synthetic data, with no false positives on 11,402 correct documents. The execution plan, with ten people in eight roles, estimates a duration of 40.2 weeks at the 50th percentile and 42.2 at the 80th, at a cost of COP 1,013 and 1,059 million, with an estimated break-even point of 2.9 countries for the agents and 5.4 for the full investment, under the still untested hypothesis that assisted effort is lower than manual effort.

Keywords: electronic invoicing, artificial intelligence agents, deterministic verification, software architecture, project planning.

Índice de contenidos

1. Introducción 13
1.1. Planteamiento general: descripción y justificación del proyecto 14
1.1.1. Planteamiento del problema 14
1.1.2. Descripción y justificación 16
1.2. Objetivos del TFG 17
1.3. Elementos innovadores del proyecto 19
2. Descripción de la situación de partida y planteamiento del problema 21
2.1. Una aproximación a la empresa u organización 21
2.2. Identificación del problema 22
2.3. Análisis del problema 25
2.3.1. Contextualización 25
2.3.2. Antecedentes 26
2.3.3. Trabajos relacionados 27
2.3.4. Revisión sistemática: ejecución y resultados 28
2.3.5. Conclusiones del análisis 33
3. Metodología 34
3.1. Enfoque metodológico y fases del proyecto 34
3.2. Revisión de la literatura 35
3.3. Métodos de diagnóstico 37
3.3.1. Diagrama de Ishikawa 37
3.3.2. Análisis de Pareto 38
3.4. Método de decisión para seleccionar la solución 39
3.5. Métodos de diseño arquitectónico 39
3.6. Método de validación técnica 40
3.7. Métodos de planificación del proyecto 41
3.8. Trazabilidad entre objetivos, métodos y entregables 41
3.9. Protocolo de evaluación experimental 42

3.9.1. Evaluación del verificador 42
3.9.2. Evaluación del agente de mapeo 43
3.9.3. Comparación entre el proceso manual y el asistido 44
3.9.4. Reproducibilidad 44
4. Desarrollo de la solución 46
4.1. Aportación: taxonomía de defectos y kit de verificación 46
4.2. Requisitos y atributos de calidad 49
4.2.1. Requisitos funcionales 49
4.2.2. Escenarios de atributos de calidad 49
4.2.3. Restricciones 49
4.3. Selección de la alternativa de solución 50
4.4. Arquitectura de referencia 50
4.4.1. Contexto del sistema (C4, nivel 1) 50
4.4.2. Contenedores (C4, nivel 2) 51
4.4.3. Stack tecnológico 52
4.4.4. Contratos e interfaces 53
4.4.5. Modelo de datos 54
4.4.6. Seguridad por diseño 54
4.4.7. Vistas de la arquitectura 54
4.4.8. Agentes, herramientas y permisos 56
4.4.9. Revisión humana (human-in-the-loop) 57
4.4.10. Modelo de amenazas y controles 59
4.4.11. Integridad de la normativa y de las reglas 61
4.5. Decisiones arquitectónicas 62
4.6. Perfiles de trabajo y matriz de responsabilidades 64
4.7. Planificación de tareas, esfuerzos y cronograma 64
4.7.1. Fases e hitos de control 65
4.7.2. Estructura de desglose del trabajo y estimaciones 65

4.7.3. Nivelación de recursos 66
4.7.4. Cronograma nivelado 67
4.7.5. Justificación de las estimaciones de entrada 68
4.8. Presupuesto 69
4.8.1. Referentes salariales y sensibilidad del presupuesto 71
4.9. Análisis de riesgos 72
4.10. Plan de calidad, integración y despliegue continuos y gobierno del proyecto 73
4.10.1. Plan de calidad y de pruebas 73
4.10.2. Integración y despliegue continuos con Jenkins 73
4.10.3. Gobierno del proyecto 74
4.11. Evaluación de la solución 74
5. Resultados obtenidos 76
5.1. Prueba de concepto: alcance, implementación y pruebas 76
5.1.1. Implementación 77
5.1.2. Pruebas automatizadas 77
5.2. Banco de defectos por inyección 77
5.2.1. Banco ampliado ejecutado por el sistema real 79
5.3. Agente de mapeo y ciclo gobernado 81
5.4. Consolas de revisión 81
5.5. Pipeline, despliegue y madurez alcanzada 83
5.5.1. Madurez alcanzada 83
5.6. Resultados de la planificación 84
5.6.1. Análisis de incertidumbre 84
5.6.2. Punto de equilibrio 85
5.7. Amenazas a la validez 87
6. Conclusiones 88
Referencias Bibliográficas 89
Anexos 100

Anexo A. Decisiones arquitectónicas completas 100

Anexo B. Supuestos de planificación 106

Anexo C. Resultados detallados del banco de defectos 107

Anexo D. Matriz del estado del arte 108

Anexo E. Evidencia de las búsquedas en Scopus 115

Índice de figuras

Figura 1. Arquitectura propuesta: flujos de integración, kit de verificación y plano agéntico 16

Figura 2. Arquitectura actual de integración: flujos asíncrono y síncrono 22

Figura 3. Flujos actuales de emisión asíncrona y síncrona, estados del comprobante y origen de los defectos 23

Figura 4. Diagrama de flujo de la revisión sistemática asistida y parcial (PRISMA 2020) 30

Figura 5. Fases de la metodología, objetivos y entregables 34

Figura 6. Diagrama Ishikawa rechazo de comprobantes por defectos de mapeo 38

Figura 7. Componentes del kit de verificación (C4, nivel 3) 47

Figura 8. Diagrama de contexto (C4, nivel 1) 51

Figura 9. Diagrama de contenedores (C4, nivel 2) 51

Figura 10. Secuencia del ciclo gobernado de una propuesta de IA 63

Figura 11. Carga semanal por rol tras la nivelación 67

Figura 12. Cronograma nivelado con ruta crítica, P50 y P80 68

Figura 13. Presupuesto por partida 70

Figura 14. Detección de defectos por tipo y mecanismo 78

Figura 15. Consola web de revisión con una propuesta bloqueada por el kit 82

Figura 16. Consola de escritorio JavaFX con una propuesta pendiente de revisión 83

Figura 17. Distribución del plazo total: plan base y escenario con dos personas de QA 84

Figura 18. Índice de criticidad de los paquetes de trabajo 85

Figura 19. Punto de equilibrio en esfuerzo acumulado 86

Figura 20. Búsqueda S1 con los filtros de tipo de documento e idioma: 1007 documentos 116

Figura 21. Búsqueda S2 con los filtros aplicados: 768 documentos 117

Figura 22. Búsqueda S3 con los filtros aplicados: 3945 documentos 118

Figura 23. Búsqueda S4 con los filtros aplicados: 86 documentos 119

Figura 24. Búsquedas guardadas S1 a S4 con su número de resultados 119

Índice de tablas

Tabla 1. Análisis de Pareto de los defectos de mapeo del caso de referencia 15

Tabla 2. Puntos de dolor del proceso actual 24

Tabla 3. Taxonomía de defectos de mapeo 24

Tabla 4. Cobertura de las necesidades del problema por enfoque 28

Tabla 5. Flujo de registros de la revisión sistemática asistida y parcial (PRISMA 2020) 29

Tabla 6. Resultado del cribado por título y resumen por búsqueda y tema 31

Tabla 7. Cruce de brecha sobre los 5659 registros únicos con tres diccionarios 31

Tabla 8. Concordancia entre la regla y la verificación del autor (muestra de 60 registros) 32

Tabla 9. Actividades del DSRM, fases, técnicas y apartados 35

Tabla 10. Estrategia de búsqueda ejecutada en Scopus el 20 de octubre de 2026 36

Tabla 11. Métricas de evaluación derivadas con el enfoque GQM 40

Tabla 12. Trazabilidad entre objetivos, fases, métodos y entregables 41

Tabla 13. Definiciones de la evaluación del verificador 43

Tabla 14. Definición formal de la taxonomía D1 a D6 47

Tabla 15. Requisitos funcionales 49

Tabla 16. Escenarios de atributos de calidad 49

Tabla 17. Matriz de decisión ponderada 50

Tabla 18. Microservicios del plano de integración 51

Tabla 19. Componentes del plano agéntico 52

Tabla 20. Lenguajes y su justificación 52

Tabla 21. Frameworks, librerías e infraestructura 53

Tabla 22. Tablas del núcleo del modelo de datos 54

Tabla 23. Vistas de la arquitectura 55

Tabla 24. Despliegue del prototipo según el chart de Helm (commit 0b6406c) 56

Tabla 25. Agentes y responsabilidades 56

Tabla 26. Permisos mínimos de cada agente sobre las herramientas 57

Tabla 27. Estados de una propuesta y transiciones 57

Tabla 28. Modelo de amenazas y controles 60

Tabla 29. Controles de integridad de la normativa y de las reglas 62

Tabla 30. Resumen de las decisiones arquitectónicas 62

Tabla 31. Equipo del proyecto 64

Tabla 32. Matriz RACI 64

Tabla 33. Fases e hitos de control del proyecto 65

Tabla 34. Estructura de desglose del trabajo con estimaciones de tres puntos 65

Tabla 35. Razones o/m y p/m de las estimaciones por grupo de paquetes 68

Tabla 36. Convergencia de la simulación con tres semillas 69

Tabla 37. Presupuesto por partida (millones de COP) 70

Tabla 38. Consumo estimado de LLM por agente 70

Tabla 39. Salario base implícito por rol y referentes públicos del mercado colombiano 71

Tabla 40. Sensibilidad del costo del proyecto a las tarifas por rol 72

Tabla 41. Registro de riesgos 72

Tabla 42. Etapas del pipeline de Jenkins 73

Tabla 43. Plan de evaluación de la solución 75

Tabla 44. Naturaleza de los resultados 76

Tabla 45. Componentes implementados en la prueba de concepto 77

Tabla 46. Suites de pruebas y resultados 77

Tabla 47. Mutantes detectados por tipo y mecanismo 78

Tabla 48. Intervalos de confianza de la detección y de los falsos positivos 79

Tabla 49. Banco ampliado: detección por tipo con su intervalo de confianza del 95 % (Wilson) 80

Tabla 50. Resultados de la simulación de Monte Carlo 84

Tabla 51. Supuestos del punto de equilibrio 86

Tabla 52. Sensibilidad del punto de equilibrio al esfuerzo asistido 87

Tabla 53. Costo empresa mensual por rol (supuesto) 107

Tabla 54. Otros supuestos del modelo 107

Tabla 55. Resultado por mutante 107

Tabla 56. Matriz del estado del arte: selección dirigida de 40 estudios 108

Tabla 57. Resumen de la verificación de las referencias 114

Tabla 58. Referencias dudosas, pendientes de verificación del autor 115

1. Introducción

Este capítulo presenta el problema que aborda el trabajo, la línea general de la solución que se planifica y los objetivos que guían su desarrollo. El punto de partida es una situación frecuente en las organizaciones que operan en varios países de Latinoamérica: la factura electrónica es obligatoria en casi toda la región, pero cada administración tributaria la regula con un formato, unos catálogos y unas reglas propias. La factura electrónica latinoamericana se considera uno de los aportes de la región a la fiscalidad internacional y el inicio de la digitalización de las administraciones tributarias (Barreix y Zambrano, 2018). Su éxito fiscal tiene, sin embargo, un costo técnico para las empresas, que deben integrar sus sistemas con cada modelo nacional.

En una organización multinacional, la emisión de comprobantes pasa por una plataforma de integración que recibe los documentos de los sistemas de origen en un modelo canónico propio, los transforma al formato que exige cada proveedor tecnológico y los envía para su validación ante la autoridad. La parte más costosa y frágil de esa cadena es el mapeo: la correspondencia campo a campo entre el modelo canónico y el formato del país, incluida la traducción de códigos internos a los códigos de los catálogos oficiales. Hoy ese mapeo se construye a mano para cada país y cada proveedor, se valida con pruebas manuales y depende del conocimiento de pocas personas.

Los modelos de lenguaje de gran tamaño ofrecen una oportunidad para acelerar esa tarea, porque pueden leer una especificación técnica y proponer correspondencias entre esquemas de datos (Parciak et al., 2024). También generan contenido plausible e incorrecto (Huang et al., 2025), algo inadmisible cuando el resultado es un documento tributario. Por eso la solución que se planifica no deja que la inteligencia artificial decida: los agentes proponen, un kit de verificación determinista comprueba cada propuesta contra el esquema, los catálogos y las reglas de la norma, y una persona aprueba antes de que el cambio llegue a producción.

El trabajo corresponde al tipo de planificación de un proyecto de innovación tecnológica en el ámbito profesional. Su alcance llega hasta el diseño de la arquitectura y la planificación de su implementación, y se complementa con un prototipo funcional que valida la viabilidad técnica de las decisiones críticas. El documento se organiza así: el capítulo 1 plantea el problema, los objetivos y los elementos innovadores; el capítulo 2 describe la situación de partida y analiza el problema a partir de sus antecedentes; el capítulo 3 presenta la metodología; el capítulo 4 desarrolla la solución; el capítulo 5 expone los resultados por objetivo; y el capítulo 6 recoge las conclusiones.

1.1. Planteamiento general: descripción y justificación del proyecto

1.1.1. Planteamiento del problema

La factura electrónica es obligatoria en la mayoría de los países de Latinoamérica. Según el Banco Interamericano de Desarrollo, más de quince países de la región la han implementado como instrumento para combatir el fraude y facilitar el cumplimiento tributario (Banco Interamericano de Desarrollo [BID], s. f.). El modelo regional es de control previo: el comprobante debe ser validado por la autoridad tributaria, o por un intermediario autorizado, antes de tener efectos fiscales (Barreix y Zambrano, 2018). Por esta razón, un error en el contenido de una factura no es un problema de calidad de datos, sino un documento rechazado o una contingencia fiscal.

La diversidad normativa es la primera fuente del problema. En Colombia, la Resolución 000165 de 2023 adoptó la versión 1.9 del anexo técnico de la factura electrónica de venta (Dirección de Impuestos y Aduanas Nacionales [DIAN], 2023), y la Resolución 000008 de 2024 modificó los plazos para su implementación (DIAN, 2024). En Perú, la Resolución de Superintendencia 188-2010/SUNAT marcó el inicio formal del sistema de emisión electrónica (Superintendencia Nacional de Aduanas y de Administración Tributaria [SUNAT], 2010; Cabrera et al., 2026). Cada país tiene su propio calendario, su esquema y sus catálogos, y no existe en la región un modelo semántico común como el que define la norma europea EN 16931 (Comité Europeo de Normalización [CEN], 2017).

La segunda fuente es el proceso con el que las empresas responden a esa diversidad. En el caso de referencia de este trabajo, una organización multinacional que emite del orden de cientos de miles de comprobantes al mes en varios países, cada incorporación de un país o de un proveedor tecnológico exige construir a mano el mapeo de aproximadamente cien campos entre el modelo canónico y el formato del proveedor, que puede ser JSON, texto plano por segmentos o XML basado en UBL 2.1 (OASIS, 2013). Estas cifras se presentan como órdenes de magnitud del caso de referencia, anonimizado por confidencialidad. La literatura regional confirma el patrón: las revisiones sobre la implementación de la factura electrónica en Latinoamérica señalan errores en la emisión y validación de comprobantes y la necesidad de soporte técnico permanente para que las empresas se adapten a la norma (Cabrera et al., 2026; Leguizamón y Lugo, 2024).

Para medir y clasificar el problema, los defectos de mapeo registrados en las incorporaciones del caso de referencia se agrupan en seis tipos, que constituyen una taxonomía propia del trabajo y se describen en el apartado 2.2. Como el volumen de defectos en un desarrollo de esta escala hace inviable un conteo exhaustivo, la tabla 1 los ordena según

su frecuencia relativa estimada por juicio experto, siguiendo el principio de Pareto: concentrar el esfuerzo en los pocos tipos que explican la mayor parte de los defectos.

Tabla 1. *Análisis de Pareto de los defectos de mapeo del caso de referencia*

Tipo	Defecto	Posición	Frecuencia relativa estimada (juicio experto)	Frecuencia estimada acumulada
D1	Código de catálogo incorrecto	1	40,8 %	40,8 %
D4	Campo exigido ausente en el modelo canónico	2	24,2 %	65,0 %
D2	Campos intercambiados o importes incoherentes	3	15,8 %	80,8 %
D3	Literal mal formateado	4	10,3 %	91,1 %
D6	Carácter reservado en el contenido	5	6,1 %	97,2 %
D5	Campo obsoleto por versión del estándar	6	2,8 %	100,0 %

Fuente: estimación del autor por juicio experto, a partir de su experiencia en el proyecto de integración del caso de referencia; pesos calculados por el método de centroide de rangos (Barron y Barrett, 1996).

Los tres tipos más frecuentes, D1, D4 y D2, concentran el 80,8 % de los defectos estimados. Son los pocos vitales del análisis: códigos de catálogo equivocados, campos exigidos que el modelo canónico no trae y campos intercambiados. Por eso el kit de verificación prioriza sus reglas, y el banco de defectos incluye al menos dos mutantes de cada tipo, con el mayor número (siete) en D1. Los pesos se derivan del orden de frecuencia mediante el método de centroide de rangos, que convierte una clasificación ordinal en pesos sin exigir mediciones exactas (Barron y Barrett, 1996).

Conviene distinguir dos magnitudes. La frecuencia relativa estimada por juicio experto, la de la tabla 1, expresa el orden en que el autor percibe los tipos de defecto a partir de su experiencia y no procede de un conteo. La frecuencia observada sería la proporción de defectos de cada tipo registrada en las incidencias reales de incorporación, y en este trabajo no se dispone de ella. Por eso el banco de defectos se diseñó con cobertura por tipo, con al menos dos mutantes de cada uno, y no en proporción a esas frecuencias: su reparto no reproduce los pesos de la tabla, de modo que la tasa de detección total del banco no equivale a la proporción de defectos reales que se detectarían. La frecuencia observada se obtendría en el proyecto a partir del registro de incidencias y de la cola de errores no recuperables, clasificando cada rechazo según la taxonomía D1 a D6; con ese registro, la tabla 1 podría contrastarse con datos medidos.

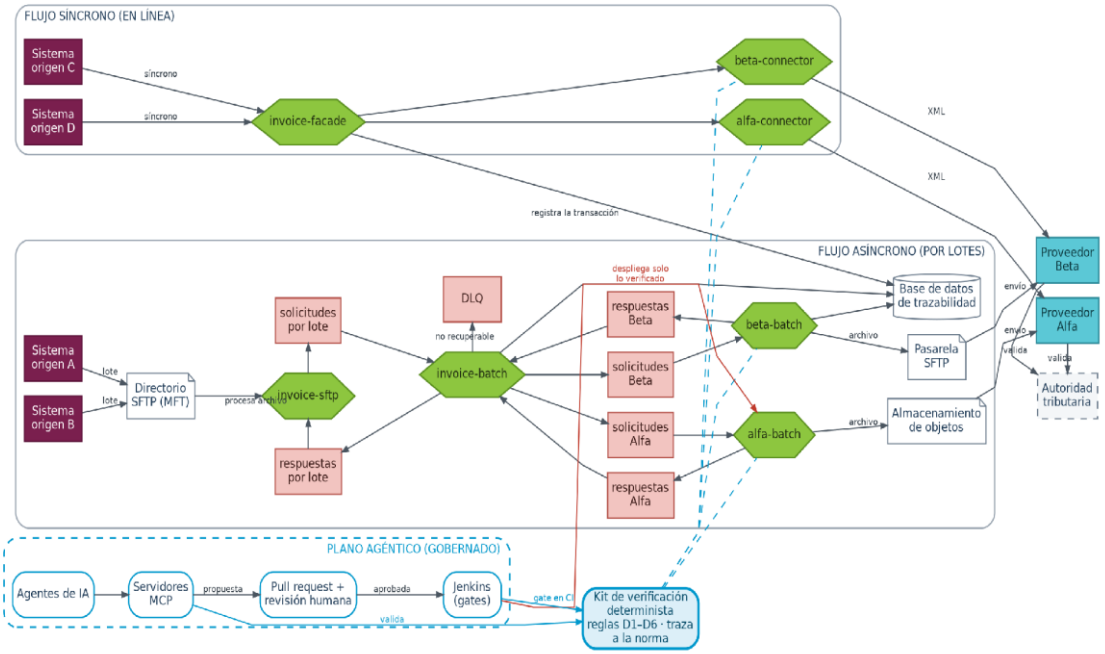
Las áreas afectadas son tres. El área de integración, que asume semanas de trabajo por cada país o proveedor nuevo. El área tributaria, que debe interpretar la norma y validar catálogos sin una herramienta que conecte cada regla con su fuente. Y las áreas de negocio que emiten los documentos, que reciben rechazos tardíos y deben corregir los datos en origen. El problema que aborda este trabajo es, por tanto, cómo reducir el costo de incorporar un país o proveedor nuevo sin comprometer la validez normativa de lo emitido.

1.1.2. Descripción y justificación

La mejora que se propone es un proceso de incorporación de países y proveedores asistido por agentes de inteligencia artificial y gobernado por una verificación determinista. En lugar de que una persona escriba el mapeo desde cero, un agente lee la especificación técnica del proveedor, consulta los catálogos oficiales y propone la tabla de correspondencias, las plantillas y las pruebas. La propuesta no se acepta por sí misma: un kit de verificación la comprueba contra el esquema vigente, los catálogos versionados y las reglas de la taxonomía de defectos, y solo si la supera pasa a revisión humana. Lo aprobado se despliega por el mismo canal de integración y despliegue continuos que el resto del software.

La verificación se aplica en los dos canales de emisión de la plataforma. En el flujo asíncrono, el kit opera dentro de los adaptadores de cada proveedor antes de agrupar y enviar los lotes, de modo que ningún comprobante defectuoso llega a formar parte de un envío masivo. En el flujo síncrono, opera dentro de los conectores en línea antes de invocar al proveedor, con un tiempo de respuesta acotado para no degradar la experiencia de quien espera el comprobante. Una misma regla protege así la emisión masiva y la unitaria, y los agentes proponen mapeos que se verifican contra ambos canales. La figura 1 muestra la arquitectura propuesta, con los dos flujos, el kit de verificación y el plano agéntico.

Figura 1. *Arquitectura propuesta: flujos de integración, kit de verificación y plano agéntico*



Fuente: elaboración propia.

La justificación económica se apoya en el carácter repetitivo del problema. El esfuerzo de incorporación se repite por cada país y por cada cambio de proveedor, de modo que crece

de forma lineal con la expansión de la organización. Una inversión inicial en el kit de verificación y en los agentes solo se justifica si el ahorro por país compensa esa inversión a partir de un número razonable de incorporaciones; por eso el plan del proyecto incluye un modelo de punto de equilibrio cuyos resultados se presentan en el capítulo 5. Además, cada documento rechazado tiene un costo de corrección y un riesgo fiscal, y un control previo objetivo reduce ambos.

En el plano organizacional, la propuesta cambia el papel de los cargos involucrados. El desarrollador de integración deja de transcribir especificaciones y pasa a revisar propuestas con evidencia. El especialista tributario deja de validar documentos uno a uno y pasa a mantener reglas trazables a la norma. El líder técnico asume la aprobación de cada cambio propuesto por la inteligencia artificial, lo que exige definir responsabilidades explícitas mediante una matriz de asignación de responsabilidades.

En el plano de los procesos, el conocimiento que hoy es tácito, como las reglas escondidas en código heredado o los códigos acordados en reuniones, se convierte en reglas escritas, versionadas y verificables. En el plano de la información, cada regla queda ligada a la sección y a la huella digital del documento normativo del que proviene, de manera que un cambio de versión de la norma permite saber qué reglas revisar. Finalmente, la propuesta es coherente con la política pública colombiana: el documento CONPES 4144 de 2025 establece la Política Nacional de Inteligencia Artificial y señala la mitigación de riesgos y el uso ético y responsable de la inteligencia artificial como ejes estratégicos (Departamento Nacional de Planeación [DNP], 2025). Un proceso en el que la inteligencia artificial propone y las personas deciden, con verificación objetiva, es una aplicación concreta de ese principio.

1.2. Objetivos del TFG

Los objetivos se formulan con un solo verbo en infinitivo y de manera que los específicos sean los pasos necesarios para alcanzar el general. Cada objetivo específico tiene un resultado medible y un plazo dentro del calendario del Trabajo de Fin de Grado, y en el capítulo 5 se presentan los resultados obtenidos para cada uno.

Objetivo general

Planificar la implementación de una arquitectura agéntica verificable para la incorporación de nuevos países y proveedores de facturación electrónica en Latinoamérica, validando su viabilidad técnica mediante un prototipo funcional evaluado con un banco de defectos inyectados, en el que el kit de verificación detecte al menos el 80 % de los defectos en ejecución y no produzca falsos positivos sobre el conjunto de comprobantes correctos, informando en ambos casos su intervalo de confianza del 95 %.

Objetivos específicos

OE1. Caracterizar los defectos de mapeo del proceso actual de integración de facturación electrónica mediante una taxonomía de al menos seis tipos, cada uno con su regla de verificación y su operador de mutación, durante las primeras tres semanas del proyecto.

OE2. Diseñar la arquitectura de referencia y el kit de verificación determinista, documentando al menos diez decisiones arquitectónicas, las vistas de la arquitectura, las responsabilidades y permisos de cada agente, el proceso de revisión humana y el modelo de amenazas con sus controles, en un plazo de cuatro semanas.

OE3. Elaborar el plan de ejecución del proyecto, con estructura de desglose del trabajo, cronograma nivelado por recursos, presupuesto, registro de riesgos y estrategia de integración y despliegue continuos, cuantificando plazo y costo con un nivel de confianza del 80 % mediante simulación de Monte Carlo, al cierre del Trabajo de Fin de Grado.

El primer objetivo responde a la necesidad de medir el problema antes de resolverlo: sin una clasificación de los defectos no es posible saber qué debe verificar el kit ni cómo evaluarlo. El segundo objetivo produce el artefacto tecnológico central del trabajo, la arquitectura y su kit de verificación, y lo documenta con sus decisiones, sus vistas, los permisos de cada agente, la revisión humana y el modelo de amenazas; la comprobación de su viabilidad con el prototipo y el banco de defectos queda en el objetivo general, con su propio umbral, de modo que el diseño y la evaluación se miden por separado. El tercer objetivo convierte el diseño en un proyecto ejecutable, con equipo, calendario, costo y riesgos, que es la contribución propia de un trabajo de planificación.

De los objetivos se desprende la pregunta que orienta el trabajo: ¿En qué medida una arquitectura agéntica con verificación determinista detecta, antes de su envío, los defectos de mapeo que la validación de esquema no detecta, y qué esfuerzo, plazo y costo exige planificar su implantación para incorporar nuevos países y proveedores de facturación electrónica? La primera parte de la pregunta se responde con una medición en el prototipo, que corresponde al objetivo general; la segunda, con el plan de ejecución del OE3. La reducción del esfuerzo de incorporación no se mide en este trabajo: se formula como hipótesis y se deja un protocolo para comprobarla en la fase F5 del proyecto (apartado 3.9.3).

Esta pregunta se concreta en tres subpreguntas: (1) ¿qué parte del esfuerzo de incorporar un país o proveedor no es automatizable y cómo debe gestionarse?; (2) ¿qué proporción de los defectos de mapeo detectan la validación de esquema y el kit de verificación, y qué ocurre con los que escapan?; y (3) ¿a partir de cuántos países se recupera

la inversión, bajo la hipótesis explícita de que el esfuerzo asistido es menor que el manual, hipótesis que se deja formulada con un protocolo para comprobarla?

El alcance se delimita así. El trabajo llega hasta el diseño de la arquitectura y la planificación de su implementación, y valida la viabilidad técnica de las decisiones críticas con un prototipo funcional construido sobre una especificación y datos sintéticos. No incluye la operación en producción ni la negociación con proveedores, y no usa datos tributarios reales, por razones de confidencialidad y de protección de datos.

1.3. Elementos innovadores del proyecto

El Manual de Oslo distingue entre la innovación de producto y la innovación de proceso, y exige que el cambio sea significativo frente a lo que la organización hacía antes (OCDE y Eurostat, 2006). Este proyecto es una innovación de proceso: cambia la forma en que la organización incorpora países y proveedores de facturación electrónica. Sus elementos innovadores son cinco, y cada uno representa una contribución concreta frente a lo que existe.

Primero, una taxonomía de defectos de mapeo con reglas y operadores de mutación. La literatura sobre factura electrónica en la región describe beneficios y desafíos de su implementación (Cabrera et al., 2026; Leguizamón y Lugo, 2024), pero en la revisión sistemática asistida del apartado 2.3.4, limitada a Scopus y al cribado por título y resumen, no se identificó una clasificación operativa de los defectos de mapeo hacia formatos de factura electrónica. La taxonomía D1 a D6 asocia a cada tipo de defecto una regla de verificación automática y un operador de mutación que permite inyectarlo de forma controlada. De este modo, la taxonomía sirve a la vez para diagnosticar el problema, para construir el verificador y para evaluarlo.

Segundo, la verificación determinista con procedencia normativa. Las propuestas que usan modelos de lenguaje para emparejar esquemas se evalúan con métricas de coincidencia y reconocen que sus resultados requieren verificación humana (Parciak et al., 2024; Narayan et al., 2022). Combinar un modelo que propone con una verificación determinista ya se ha aplicado en otros dominios regulados (Nguyen et al., 2026; Vergés et al., 2026; Villanueva-Miranda et al., 2026), pero en Scopus no se identificó una propuesta que lo aplique a mapeos hacia formatos nacionales de facturación electrónica con trazabilidad a la norma. Este proyecto agrega ese verificador, que no depende del juicio del modelo: cada regla del kit cita la sección y la huella digital del documento normativo del que proviene. Un auditor puede rastrear cualquier rechazo hasta la norma, y un cambio de versión de la norma indica exactamente qué reglas revisar.

Tercero, un ciclo gobernado para la inteligencia artificial. Los agentes basados en modelos de lenguaje ya razonan y usan herramientas (Yao et al., 2023; Wang et al., 2024), y la revisión de Quevedo-Tumailli et al. (2025) concluye que, aunque mejoran la productividad en la generación de código, requieren una supervisión humana rigurosa. En la arquitectura propuesta, los agentes solo entregan propuestas en forma de solicitudes de cambio; no pueden desplegar ni escribir en producción, y una propuesta que no pasa el kit no puede aprobarse. Cada decisión queda registrada con su verificación, su revisor y su fecha. Ya se han propuesto agentes con verificación para software tributario (Gogani Khiabani et al., 2026), de modo que la contribución no es el ciclo en sí, sino su aplicación a la incorporación de países y proveedores de facturación electrónica.

Cuarto, una evaluación objetiva mediante un banco de defectos por inyección. Las pruebas de mutación miden la capacidad de un conjunto de pruebas para detectar defectos introducidos deliberadamente (Jia y Harman, 2011). El proyecto aplica esa técnica a los mapeos de facturación: a partir de un mapeo correcto se generan versiones defectuosas, una o más por tipo de la taxonomía, y se mide qué porcentaje detecta cada mecanismo de verificación. Así la contribución del kit se demuestra con datos y no con opiniones.

Quinto, una planificación cuantitativa bajo incertidumbre. En lugar de un cronograma con fechas fijas, el plan estima cada paquete de trabajo con tres valores, nivela la carga para que ningún rol supere su capacidad y simula el proyecto con el método de Monte Carlo, siguiendo los enfoques de gestión adaptativa de la guía del PMI (Project Management Institute [PMI], 2021). El resultado es un plazo y un costo acompañados de su probabilidad, un presupuesto que incluye el consumo de los modelos de lenguaje y un punto de equilibrio que indica a partir de cuántos países se recupera la inversión.

El alcance esperado en cada proceso es el siguiente. En la incorporación de países, reducir el esfuerzo manual de mapeo y medir esa reducción frente a una línea base. En la emisión, que el 100 % de los comprobantes pase la verificación antes de salir hacia el proveedor. En la gestión del conocimiento normativo, que cada regla sea trazable a su fuente. Y en la gestión del proyecto, que el plan comunique fechas y costos con su nivel de confianza.

2. Descripción de la situación de partida y planteamiento del problema

2.1. Una aproximación a la empresa u organización

La organización de referencia es una empresa multinacional de servicios con operación en varios países de Latinoamérica. Por razones de confidencialidad no se identifica la organización ni sus proveedores, y las cifras se expresan como órdenes de magnitud. Su facturación es masiva y recurrente: emite del orden de cientos de miles de comprobantes al mes, entre facturas, boletas y notas de crédito, con picos de volumen en los cierres de mes.

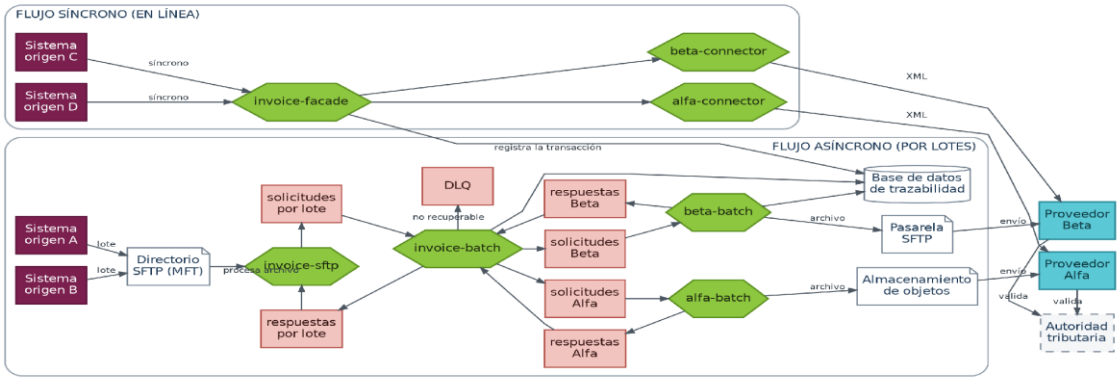
Desde la perspectiva de sistemas, la organización puede describirse en tres niveles. El primero son los sistemas de origen, que gestionan las ventas, los contratos y la contabilidad, y que producen la información de cada comprobante. El segundo es la plataforma de integración, un conjunto de microservicios que recibe los comprobantes, los enruta por país, los transforma al formato de cada proveedor y procesa las respuestas. El tercero son los sistemas externos: los proveedores tecnológicos de facturación autorizados en cada país y las administraciones tributarias que validan los documentos.

La plataforma de integración se organiza en subsistemas. El subsistema de ingesta recibe archivos por transferencia gestionada de archivos y solicitudes por servicios web, valida el modelo canónico y publica cada comprobante en un sistema de mensajería orientado a eventos. El subsistema de enrutamiento decide el proveedor según el país y el tipo de documento. Los adaptadores, uno por proveedor, transforman el comprobante al formato propietario, lo persisten, lo agrupan en lotes y lo envían. El subsistema de respuestas correlaciona el resultado del proveedor con el comprobante original y actualiza su estado, y el subsistema de archivo conserva los documentos durante el plazo legal. Esta forma de organización sigue los patrones clásicos de integración empresarial, como el modelo de datos canónico, el enrutador basado en contenido y el traductor de mensajes (Hohpe y Woolf, 2003).

La plataforma atiende dos modalidades de emisión que comparten la misma infraestructura. El flujo asíncrono, o por lotes, está pensado para la emisión masiva: los sistemas de origen depositan archivos con muchos comprobantes, que se procesan, se agrupan por proveedor y se envían sin que el sistema de origen espere la respuesta. El flujo síncrono, o en línea, está pensado para la emisión unitaria: un sistema de origen solicita un solo comprobante a la fachada de servicios web y espera, en la misma llamada, la respuesta del proveedor, como ocurre en una venta en mostrador o en la expedición inmediata de un documento. Los dos flujos se interconectan en tres puntos. Comparten la base de datos de

trazabilidad, en la que cada comprobante queda registrado con su identificador de correlación sin importar el canal por el que llegó. Comparten los proveedores tecnológicos, que reciben los comprobantes por lotes mediante archivos o de forma unitaria mediante sus conectores en línea. Y comparten las reglas de transformación de cada proveedor, de modo que un defecto de mapeo afecta por igual a la emisión masiva y a la unitaria. La figura 2 representa ambos flujos y sus puntos de interconexión.

Figura 2. *Arquitectura actual de integración: flujos asíncrono y síncrono*



Fuente: elaboración propia.

Los procesos donde se concentran los problemas son la incorporación de un país o proveedor y la corrección de rechazos. En la incorporación intervienen el equipo de integración, el especialista tributario, el proveedor tecnológico, los responsables de los sistemas de origen y los equipos de infraestructura y seguridad. El procedimiento escrito comprende la recepción de la especificación del proveedor, el diseño del mapeo, el desarrollo del adaptador, las pruebas en ambientes previos y la puesta en producción.

Junto al procedimiento escrito existe un procedimiento tácito que explica buena parte de los defectos. Incluye los códigos acordados en reuniones con el proveedor que no quedan en la especificación, las reglas implementadas años atrás en código heredado que nadie documentó y el conocimiento de las pocas personas que conocen cada formato. La gestión de credenciales, llaves y accesos de red también forma parte de este procedimiento: se tramita con varios actores y suele determinar el calendario más que el propio desarrollo. El análisis de la situación de partida debe considerar ambos procedimientos, porque la solución no solo automatiza el mapeo, sino que convierte en reglas explícitas el conocimiento que hoy es tácito.

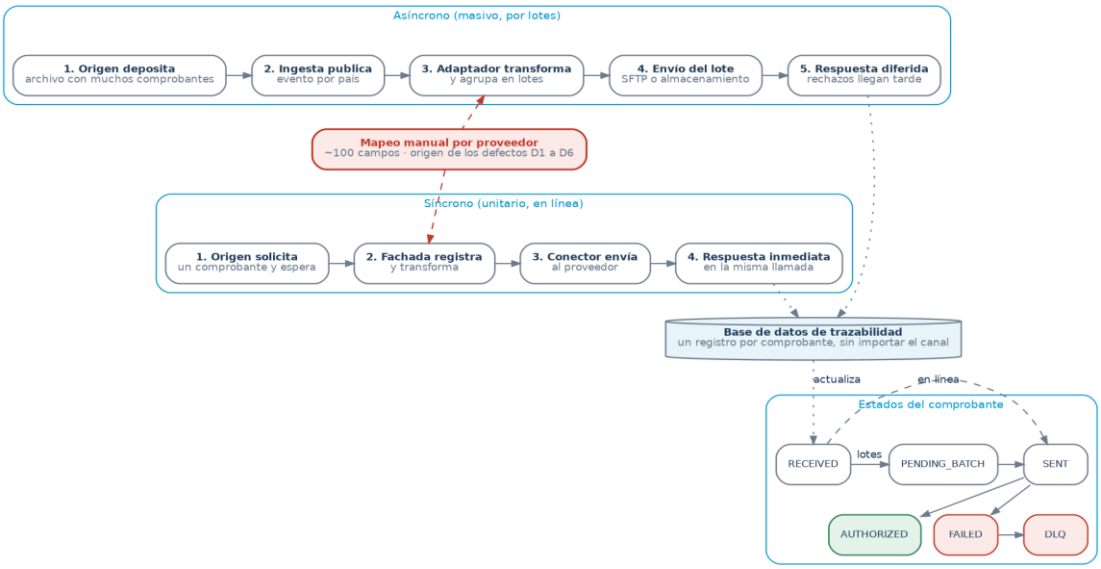
2.2. Identificación del problema

El proceso actual de emisión asíncrona sigue seis pasos: los sistemas de origen generan el comprobante en el modelo canónico; lo depositan mediante transferencia

gestionada de archivos; un servicio de ingesta lo publica en el sistema de mensajería, enrutado por país; un adaptador por proveedor lo transforma, lo persiste, lo agrupa en lotes y lo envía; el proveedor lo presenta ante la autoridad tributaria; y la respuesta regresa a la plataforma, donde el resultado del proveedor es la fuente primaria y la constancia oficial es el respaldo. Cada comprobante pasa por los estados recibido, pendiente de lote, enviado y autorizado o fallido, con reintentos controlados y una cola para los errores que no se pueden recuperar.

El proceso de emisión síncrona es más corto, pero más exigente en tiempo. El sistema de origen envía un comprobante a la fachada en línea; la fachada lo registra en la base de datos de trazabilidad, lo transforma con las reglas del proveedor y lo envía por el conector correspondiente; la respuesta regresa en la misma llamada y el comprobante pasa del estado recibido al de enviado y, de ahí, a autorizado o fallido, sin pasar por la agrupación en lotes. Por eso un mismo defecto de mapeo se manifiesta de forma distinta en cada canal. En el flujo asíncrono se descubre tarde, cuando el lote ya fue enviado y regresa con rechazos masivos que deben corregirse en origen. En el flujo síncrono se descubre de inmediato, pero el error llega al usuario en el momento mismo de la transacción. En ambos casos el origen es el mismo, el mapeo manual, y por eso la verificación que se planifica debe operar en los dos canales. La figura 3 resume los dos procesos, los estados del comprobante y el punto en que se originan los defectos.

Figura 3. *Flujos actuales de emisión asíncrona y síncrona, estados del comprobante y origen de los defectos*



Fuente: elaboración propia.

A partir de este flujo se identifican siete puntos de dolor, presentados en la tabla 2. Algunos son técnicos y otros de gestión, y no todos se resuelven con inteligencia artificial.

Tabla 2. *Puntos de dolor del proceso actual*

#	Punto de dolor	Efecto	Respuesta de la propuesta
P1	Mapeo manual de unos cien campos entre formatos	Semanas de trabajo por país y defectos recurrentes	Agente de mapeo y kit de verificación
P2	Formatos heterogéneos: JSON, texto por segmentos y XML	Un adaptador distinto por proveedor	Arquetipo de adaptador
P3	Datos de origen incompletos que se corrigen en origen	Rechazos que regresan tarde	Verificación en ejecución y clasificación de fallos
P4	Reglas ocultas en código heredado	Conocimiento tácito y riesgo de omisión	Agente de extracción de reglas con evidencia
P5	Credenciales, llaves, puertos y accesos	Del orden de una semana por gestión	Planificación temprana y en paralelo
P6	Coordinación entre muchos actores	Esperas y retrabajo	Matriz de responsabilidades y gestión de interesados
P7	El ciclo se repite por país y por proveedor	Costo que crece de forma lineal	Arquetipo, kit y agentes reutilizables

Fuente: elaboración propia.

El punto de dolor P1 es el núcleo técnico del problema, y sus defectos se clasifican en la taxonomía de la tabla 3. Cada tipo tiene una regla de verificación que el kit implementa y un operador de mutación que permite evaluarlo. Un detalle orienta el diseño: un código de catálogo válido pero equivocado no lo detecta la validación de esquema, porque en UBL los códigos son cadenas de texto libres (OASIS, 2013). Solo lo detecta una regla de negocio que conozca la semántica del catálogo, o la comparación con ejemplos validados por personas.

Tabla 3. *Taxonomía de defectos de mapeo*

Tipo	Defecto	Ejemplo genérico	Regla de verificación	Operador de mutación
D1	Código de catálogo incorrecto	Código tomado de una nota verbal y no de la especificación	Catálogo oficial versionado y coherencia entre campos	Sustituir por otro código, válido o inexistente
D2	Campos intercambiados	Monto, cantidad y precio unitario cruzados	Invariantes aritméticas y ejemplos validados	Permutar dos campos o tomar un total equivocado
D3	Literal mal formateado	Mayúsculas, espacios o nombre de parámetro	Literales y listas cerradas con comparación exacta	Alterar mayúsculas o añadir espacios
D4	Campo exigido ausente en el canónico	Dato obligatorio que el modelo canónico no trae	Cobertura del esquema destino y campos no vacíos	Eliminar el mapeo o mapear a un valor vacío
D5	Campo obsoleto por versión	Campo retirado en la versión vigente del estándar	Esquema vigente y lista de elementos retirados	Añadir un elemento de la versión anterior
D6	Carácter reservado en el contenido	Separador del formato dentro de un texto libre	Conteo de campos y datos adversarios	Desactivar el saneamiento de un texto libre

Fuente: elaboración propia.

El problema queda identificado en tres dimensiones. En la dimensión técnica, el mapeo manual introduce defectos que la validación de esquema no detecta. En la dimensión organizacional, el conocimiento necesario para evitarlos está repartido entre pocas personas

y documentos dispersos. Y en la dimensión de gestión, el calendario de cada incorporación depende de trámites de acceso que no son técnicos. La solución que se planifica ataca las dos primeras con la arquitectura y la tercera con el plan del proyecto.

2.3. Análisis del problema

Este apartado analiza los antecedentes del problema en cinco bloques: la contextualización de los conceptos que usa el trabajo, los antecedentes en los ámbitos internacional, regional, nacional y local, los trabajos relacionados agrupados por tema, los resultados de la revisión sistemática y las conclusiones del análisis. La literatura se buscó mediante una revisión sistemática asistida y parcial que sigue la estructura de la declaración PRISMA 2020 hasta el cribado por título y resumen (Page et al., 2021), cuyo protocolo se describe en el apartado 3.2.

2.3.1. Contextualización

Factura electrónica con validación previa. Es un comprobante fiscal digital que la autoridad tributaria, o un intermediario autorizado, valida antes de que tenga efectos legales. Inicialmente se concibió como un instrumento de control documental para evitar la omisión de ventas y la inclusión de compras falsas, y luego se extendió a otras áreas del control tributario (Barreix y Zambrano, 2018).

Estándar UBL y modelo semántico. UBL (Universal Business Language) es un estándar de OASIS que define documentos comerciales en XML, entre ellos la factura (OASIS, 2013). Varios países latinoamericanos lo usan como base de sus perfiles nacionales, a los que añaden extensiones, catálogos y reglas. En Europa, la norma EN 16931-1 define un modelo semántico común de la factura al que se asocian sintaxis concretas (CEN, 2017).

Integración empresarial orientada a eventos. Es la forma de conectar sistemas mediante mensajes asíncronos y patrones reutilizables, como el modelo de datos canónico, el enrutador basado en contenido y el traductor de mensajes (Hohpe y Woolf, 2003). En este trabajo, el traductor de mensajes es justamente el componente que hoy se escribe a mano.

Requisitos, calidad y verificación del software. La ingeniería de software distingue entre la validación, que comprueba que se construye el producto correcto, y la verificación, que comprueba que el producto cumple su especificación (Sommerville, 2011). Las pruebas y las revisiones técnicas son los mecanismos de aseguramiento de la calidad que hacen visible esa diferencia en la práctica (Pressman, 2010). El kit de verificación de este trabajo es un mecanismo de verificación automático: comprueba que cada comprobante cumple la especificación del país.

Modelos de lenguaje de gran tamaño y agentes. Un modelo de lenguaje de gran tamaño genera texto a partir de instrucciones en lenguaje natural. Un agente basado en estos modelos alterna razonamiento y acciones sobre herramientas para cumplir un objetivo (Yao et al., 2023), y su diseño suele organizarse en perfil, memoria, planificación y acción (Wang et al., 2024). El Model Context Protocol estandariza la forma en que los agentes acceden a herramientas y datos externos (Model Context Protocol, 2026).

Alucinación. Es la tendencia de los modelos de lenguaje a producir contenido plausible pero falso o infiel a la fuente (Huang et al., 2025). La generación aumentada con recuperación reduce el problema al anclar la respuesta en documentos recuperados (Lewis et al., 2020), pero no lo elimina.

Integración y despliegue continuos. Son prácticas de la cultura DevOps que automatizan la construcción, las pruebas y el despliegue del software mediante un canal o pipeline. Giraldo-Hurtado (2023) caracterizó las herramientas que implementan la metodología GitOps y demostró, con un prototipo funcional, el proceso completo de integración y despliegue continuos de una aplicación. En este trabajo, ese canal es el punto donde el kit de verificación actúa como control obligatorio antes de cualquier despliegue.

2.3.2. Antecedentes

Ámbito internacional. La Organización para la Cooperación y el Desarrollo Económicos analiza la factura electrónica como una de las piezas de la administración tributaria digital (OCDE, 2022), y el Banco Interamericano de Desarrollo describe la evolución hacia una administración tributaria basada en datos, en la que la factura electrónica es un componente central (Calijuri, 2024). Europa optó por un modelo semántico común que facilita la interoperabilidad entre países (CEN, 2017). En paralelo, el campo de la inteligencia artificial ha producido marcos de gestión de riesgos, como el del Instituto Nacional de Estándares y Tecnología de los Estados Unidos, que organiza la gestión en las funciones de gobernar, mapear, medir y gestionar (National Institute of Standards and Technology [NIST], 2023).

Ámbito regional. Latinoamérica es pionera en factura electrónica. Barreix y Zambrano (2018) documentan la experiencia regional desde su implementación hasta su impacto recaudatorio. Las revisiones recientes recogen las experiencias de varios países (Leguizamón y Lugo, 2024) y evaluaciones de impacto como la de Ecuador, que analiza el efecto de la factura electrónica sobre el cumplimiento tributario (Ramírez-Álvarez et al., 2022). En Perú, la factura electrónica permitió automatizar controles y mejorar la confiabilidad de la gestión tributaria (De Velazco, 2016), aunque persisten retos técnicos y jurídicos para los contribuyentes (Cabrera et al., 2026). En Chile, la discusión sobre la

equivalencia funcional entre el documento físico y el documento digital muestra que la validez de un comprobante electrónico depende de que cumpla condiciones técnicas precisas (Escobar, 2019).

Ámbito nacional. En Colombia, la DIAN regula la factura electrónica mediante anexos técnicos versionados. La adopción de la versión 1.9 (DIAN, 2023) y la posterior modificación de sus plazos (DIAN, 2024) muestran que cada cambio normativo obliga a las empresas a ajustar sus sistemas. En materia de inteligencia artificial, el documento CONPES 4144 de 2025 fija la política nacional, con ejes de mitigación de riesgos y de uso y adopción responsable (DNP, 2025).

Ámbito local. En el caso de referencia, el antecedente directo es la incorporación sucesiva de países y proveedores con el procedimiento manual descrito en el apartado 2.1, y los defectos de esas incorporaciones, cuya frecuencia relativa se estima por juicio experto en la tabla 1.

2.3.3. Trabajos relacionados

Los trabajos relacionados se agrupan en cuatro temas: a) emparejamiento de esquemas con modelos de lenguaje, b) modelos de lenguaje en la ingeniería de software, c) fiabilidad y verificación de sistemas basados en modelos de lenguaje, y d) tecnología aplicada a la factura electrónica.

a) Emparejamiento de esquemas con modelos de lenguaje. El emparejamiento de esquemas es un problema clásico de la integración de datos (Rahm y Bernstein, 2001). Narayan et al. (2022) mostraron que un modelo fundacional puede realizar tareas de preparación de datos, incluido el emparejamiento, sin entrenamiento específico. Parciak et al. (2024) evaluaron modelos de lenguaje para emparejar esquemas a partir de nombres y descripciones, y encontraron que aceleran el arranque del proceso, pero que el esfuerzo de verificación humana sigue siendo central. Liu et al. (2025) combinaron modelos pequeños y grandes para reducir el costo y mejorar la precisión. Estos trabajos producen correspondencias entre campos, no transformaciones ejecutables con reglas de negocio, y en la revisión ejecutada no se identificaron aplicaciones a formatos tributarios con catálogos versionados por una autoridad (apartado 2.3.4).

b) Modelos de lenguaje en la ingeniería de software. La revisión sistemática de Hou et al. (2024) documenta el uso de modelos de lenguaje en todas las fases del ciclo de vida del software. En el ámbito hispanohablante, Quevedo-Tumailli et al. (2025) revisaron 27 estudios y concluyeron que estos modelos mejoran la calidad del código y la productividad, pero que su uso exige una supervisión humana rigurosa por el riesgo de generar errores o código inseguro. Jimenez et al. (2024) propusieron evaluar a los agentes de código con las

pruebas del propio repositorio, es decir, con un oráculo ejecutable y no con el juicio del modelo.

c) Fiabilidad y verificación. La revisión de Huang et al. (2025) clasifica las causas de la alucinación y concluye que los sistemas con recuperación de documentos la reducen sin eliminarla. Las pruebas de mutación ofrecen una forma objetiva de medir la eficacia de un mecanismo de verificación (Jia y Harman, 2011). Ninguno de estos trabajos aborda la verificación de mapeos tributarios.

d) Tecnología aplicada a la factura electrónica. La literatura regional sobre factura electrónica se concentra en su impacto fiscal, su marco normativo y su adopción por las empresas (Barreix y Zambrano, 2018; Leguizamón y Lugo, 2024; Cabrera et al., 2026). En la revisión ejecutada en Scopus, los trabajos que combinan modelos de lenguaje o agentes con facturas se ocupan sobre todo de extraer datos de documentos o de automatizar las cuentas por pagar; los más cercanos a la integración proponen agentes que componen interfaces de programación de facturación electrónica o que emiten facturas, sin verificación determinista declarada en sus resúmenes (anexo D). La tabla 4 resume cómo responde cada enfoque a las necesidades del problema.

Tabla 4. *Cobertura de las necesidades del problema por enfoque*

Enfoque	Automatiza el mapeo	Garantiza validez normativa	Trazabilidad a la norma	Gobierno de la IA
Plataforma comercial de facturación	Parcial	Sí, en su formato	Opaca para el cliente	No aplica
Desarrollo tradicional	No	Sí, si se programan las reglas	Solo si se documenta	No aplica
Emparejamiento con modelos de lenguaje	Sí, correspondencias	No	No	No tratado
Agentes de código	Sí, código y pruebas	Solo con pruebas existentes	No	Parcial
Arquitectura propuesta	**Sí, gobernado**	**Sí, verificación obligatoria**	**Sí, regla a fuente**	**Sí, NIST y CONPES 4144**

Fuente: elaboración propia a partir de los trabajos revisados.

2.3.4. Revisión sistemática: ejecución y resultados

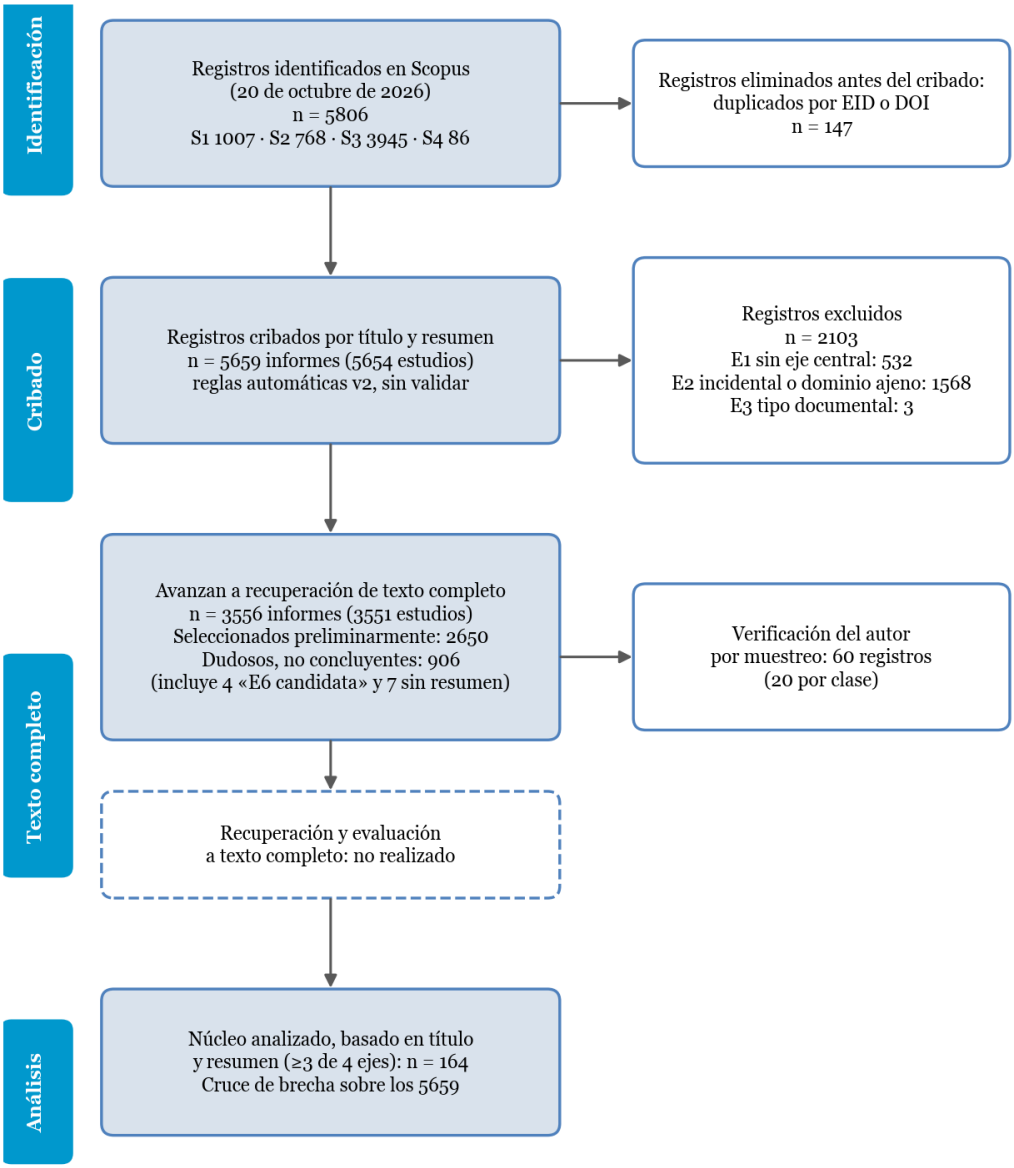
Este apartado presenta los resultados de la revisión sistemática asistida y parcial descrita en el apartado 3.2. La tabla 5 y la figura 4 resumen el flujo de registros según la estructura PRISMA 2020 hasta el cribado por título y resumen; las fases posteriores no se realizaron. Se distinguen informes, que son los registros de Scopus, y estudios, porque cinco pares de registros con el mismo título y distinto año se cuentan como un estudio con dos informes.

Tabla 5. *Flujo de registros de la revisión sistemática asistida y parcial (PRISMA 2020)*

Fase	Informes	Estudios	Observación
Registros identificados en Scopus (S1 a S4)	5806	—	S1 1007, S2 768, S3 3945 y S4 86
Duplicados eliminados antes del cribado	147	—	137 grupos por EID entre búsquedas y 9 por DOI dentro de una misma búsqueda
Cribados por título y resumen	5659	5654	Reglas automáticas, sin validar por el autor
Excluidos	2103	2103	E1 532, E2 1568 y E3 3; E4 y E5: 0
Dudosos, no concluyentes	906	905	Incluye 7 sin resumen y 4 «E6 candidata»
Seleccionados preliminarmente en el cribado por título y resumen	2650	2646	No son estudios incluidos en la revisión
Avanzan a recuperación de texto completo	3556	3551	Seleccionados preliminarmente más dudosos
Recuperación y evaluación a texto completo	No realizado	No realizado	—
Núcleo analizado, basado en título y resumen	164	—	Seleccionados con al menos 3 de los 4 ejes

Fuente: elaboración propia a partir de los archivos de la auditoría (control de recuentos).

Figura 4. *Diagrama de flujo de la revisión sistemática asistida y parcial (PRISMA 2020)*



Fuente: elaboración propia con la estructura de PRISMA 2020 (Page et al., 2021).

Por motivo, los excluidos se reparten así: 532 no tienen el eje central de modelos de lenguaje o agentes (E1); en 1568 los términos del tema son incidentales o el dominio es ajeno, en su mayoría aplicaciones médicas, biológicas o educativas (E2); y 3 son tipos documentales no elegibles (E3). Ninguno quedó fuera por periodo o idioma (E4), y los duplicados se retiraron antes del cribado (E5). Las cifras de cribado son automáticas y no están validadas por el autor: los 2650 registros son seleccionados preliminarmente en el cribado por título y resumen, no estudios incluidos en la revisión, y junto con los 906 dudosos avanzan a recuperación de texto completo, fase que no se realizó. La tabla 6 desglosa el resultado por búsqueda.

Tabla 6. *Resultado del cribado por título y resumen por búsqueda y tema*

Búsqueda y tema	Exportados	Seleccionados preliminarmente	Dudosos	Excluidos
S1 (a) Emparejamiento de esquemas con modelos de lenguaje	1007	414	106	487
S2 (b) Modelos de lenguaje y agentes en ingeniería de software	768	297	258	208
S3 (c) Fiabilidad, verificación y pruebas	3945	2014	557	1370
S4 (d) Tecnología aplicada a la factura electrónica	86	17	9	60

Fuente: elaboración propia. Un registro que aparece en varias búsquedas se cuenta en cada una, y en S2 y S3 las filas suman menos que lo exportado por los duplicados internos.

La búsqueda de facturación electrónica (S4) es la más pequeña y la que más excluye: 60 de sus 86 registros quedan excluidos por las reglas y solo 17 quedan seleccionados preliminarmente. La de verificación y pruebas (S3) aporta la mayor parte de los seleccionados, porque combina modelos de lenguaje con pruebas de software, un campo muy activo.

Para contrastar la afirmación de novedad, sobre los 5659 registros únicos se cruzaron tres ejes, con independencia del cribado: (a) mapeo o integración de datos con modelos de lenguaje; (b) verificación determinista, validación, pruebas de mutación u oráculo; y (c) facturación electrónica, cumplimiento fiscal o dominio regulado. La tabla 7 muestra el número de registros de cada intersección con tres diccionarios: el estricto, limitado al título y las palabras clave; el base, que exige una mención fuerte; y el amplio, en el que basta una mención.

Tabla 7. *Cruce de brecha sobre los 5659 registros únicos con tres diccionarios*

Diccionario	a	b	c	a∧b	a∧c	b∧c	a∧b∧c
Estricto (título y palabras clave)	71	101	57	1	0	0	0
Base (mención fuerte)	602	316	293	5	20	6	0
Amplio (cualquier mención)	2061	2772	1162	762	495	492	201

Fuente: elaboración propia. a: mapeo o integración con modelos de lenguaje; b: verificación determinista, validación, mutación u oráculo; c: facturación, cumplimiento fiscal o dominio regulado.

Con los diccionarios estricto y base ningún registro combina los tres ejes. Con el amplio aparecen 201, pero son coincidencias léxicas: solo 2 contienen términos fiscales y ambos tratan de extraer datos de documentos. Se leyeron los 31 resúmenes de las intersecciones del diccionario base: 10 combinan realmente dos ejes, 1 lo hace parcialmente y 20 solo mencionan los términos, y ninguno de los que combinan pertenece al dominio fiscal. También se leyeron los 32 registros que unen facturación o fiscalidad con modelos de lenguaje o agentes.

El resultado obliga a matizar la novedad. El patrón en el que el modelo propone, una verificación determinista comprueba y una persona aprueba ya aparece en otros dominios regulados: comprobaciones deterministas de códigos y precios contra catálogos oficiales en obra pública (Nguyen et al., 2026), cribado normativo de productos con un agente y reglas parametrizadas (Vergés et al., 2026) y armonización auditable de datos biomédicos con especificaciones de mapeo generadas por agentes y ejecutadas por módulos deterministas (Villanueva-Miranda et al., 2026). También hay correspondencias de esquemas generadas por modelos de lenguaje con explicaciones verificables formalmente (Pinto et al., 2026). En el dominio fiscal, agentes basados en modelos de lenguaje traducen el código tributario a software con pruebas metamórficas (Gogani Khiabani et al., 2026) o generan recomendaciones tributarias con razonamiento por reglas y verificación de consistencia (Gao et al., 2026). En Scopus no se identificó una propuesta que aplique el patrón a mapeos hacia formatos nacionales de facturación electrónica con trazabilidad de cada regla a la norma.

El autor verificó una muestra aleatoria de 60 registros, 20 de cada clase de la regla, sin conocer la decisión automática; tras una segunda lectura cambió 7 decisiones, de Excluir a Incluir. La tabla 8 compara ambas decisiones.

Tabla 8. *Concordancia entre la regla y la verificación del autor (muestra de 60 registros)*

Decisión de la regla	Autor: Incluir	Autor: Dudoso	Autor: Excluir	Concordancia
Incluir	12	6	2	12/20
Dudoso	4	7	9	7/20
Excluir	5	7	8	8/20

Fuente: elaboración propia. Acuerdo total del 45 % (27 de 60); kappa de Cohen de 0,175.

El acuerdo total fue del 45 % (27 de 60), con un kappa de Cohen de 0,175, una concordancia baja. Si solo se distingue entre avanzar a texto completo y excluir, el acuerdo es del 61,7 %, con un kappa de 0,127: la regla excluyó 12 registros que el autor haría avanzar y hizo avanzar 11 que el autor excluiría. La concordancia preliminar con el lector asistido había sido mayor (kappa de 0,31), lo que confirma que esa medida no sustituye la verificación humana. Las cifras de cribado de la tabla 5 son, por tanto, una aproximación automática: la clase Dudoso es no concluyente y las exclusiones automáticas no están validadas.

Los 164 registros del núcleo analizado, basado en título y resumen, son los seleccionados preliminarmente que combinan al menos tres de los cuatro ejes: modelos de lenguaje; mapeo o integración; verificación, pruebas o revisión humana; y facturación o dominio regulado. Su matriz completa se conserva como material de apoyo, y el anexo D presenta una selección dirigida de 40 estudios elegidos por una regla declarada.

2.3.5. Conclusiones del análisis

El análisis permite extraer tres conclusiones. La primera es que el problema es estructural en la región: la factura electrónica con validación previa es obligatoria, sus formatos son nacionales y cambian por versiones, y no existe un modelo semántico común que evite repetir la integración en cada país. La segunda es que los modelos de lenguaje pueden acelerar el mapeo, pero la literatura coincide en que sus resultados requieren verificación y supervisión humana, y en que la alucinación no desaparece con las técnicas actuales. La tercera es que, en la revisión sistemática asistida y parcial ejecutada en Scopus (apartado 2.3.4), el patrón en el que un modelo propone, una verificación determinista comprueba y una persona aprueba aparece en otros dominios regulados, pero no se identificó una propuesta que lo aplique a mapeos hacia formatos nacionales de facturación electrónica con trazabilidad a la norma, ni un plan de proyecto que lo gobierne. La contribución del trabajo es, por tanto, la aplicación de ese patrón a la facturación electrónica latinoamericana, la taxonomía D1 a D6 con sus operadores de mutación y el plan de implantación; el resultado se limita a Scopus, al inglés y al cribado por título y resumen.

3. Metodología

Este capítulo describe cómo se desarrolla el trabajo y justifica cada método seleccionado. Los métodos no se eligen por costumbre, sino porque responden a una necesidad concreta de los objetivos específicos: diagnosticar el problema con evidencia, decidir la solución sin subjetividad, diseñarla de forma verificable y planificar su ejecución con plazos y costos cuantificados. Al final del capítulo, la tabla 12 resume la trazabilidad entre objetivos, fases, métodos y entregables.

3.1. Enfoque metodológico y fases del proyecto

El trabajo sigue el enfoque de investigación en ciencia del diseño (design science research), propio de los sistemas de información, que estudia la creación de artefactos que resuelven problemas relevantes y exige evaluarlos con rigor (Hevner et al., 2004). Este enfoque es adecuado porque la contribución del trabajo es un artefacto tecnológico, una arquitectura con su kit de verificación y su plan de implementación, y no la explicación de un fenómeno. Se adopta el proceso de seis actividades de Peffers et al. (2007): identificar el problema, definir los objetivos de la solución, diseñar y desarrollar, demostrar, evaluar y comunicar. Este proceso, conocido como DSRM (design science research methodology), es el marco metodológico principal del trabajo: las demás técnicas se aplican dentro de alguna de sus actividades.

Para organizar el trabajo en el tiempo, el proceso se concreta en un ciclo de vida por fases con un entregable al final de cada una, similar al modelo híbrido, secuencial por fases e iterativo dentro de ellas, que Giraldo-Hurtado (2023) aplicó para construir y validar un catálogo de herramientas con un prototipo funcional. Cada fase responde a un objetivo específico, de modo que el avance del trabajo se puede verificar objetivo por objetivo. El enfoque combina técnicas cualitativas, como el análisis de causas y el juicio experto, con técnicas cuantitativas, como la simulación y la medición de la detección de defectos, lo que corresponde a un diseño mixto (Hernández Sampieri et al., 2014). La figura 5 presenta las fases.

Figura 5. *Fases de la metodología, objetivos y entregables*



Fuente: elaboración propia.

La fase 0 delimita el problema y construye el estado de la cuestión. La fase 1 diagnostica las causas de los defectos y produce la taxonomía. La fase 2 selecciona la alternativa de solución, diseña la arquitectura y la valida con un prototipo. La fase 3 elabora el plan de ejecución del proyecto. La fase 4 evalúa los resultados frente a las métricas definidas y redacta las conclusiones. Dentro de las fases 2 y 3 el trabajo es iterativo: el diseño y el plan se ajustan a partir de los resultados del prototipo y de la simulación.

La tabla 9 relaciona cada actividad del DSRM con las fases del trabajo, las técnicas específicas que la desarrollan y los apartados en los que se aplican.

Tabla 9. *Actividades del DSRM, fases, técnicas y apartados*

Actividad del DSRM	Fase del trabajo	Técnicas específicas	Apartados
1. Identificar el problema y motivar	Fases 0 y 1	Búsqueda estructurada inspirada en PRISMA 2020; diagrama de Ishikawa; análisis de Pareto por juicio experto	1.1, 2.2, 2.3, 3.2 y 3.3
2. Definir los objetivos de la solución	Fases 0 y 1	Objetivos con métrica y plazo; enfoque GQM; requisitos y escenarios de calidad	1.2, 3.6, 3.9 y 4.2
3. Diseñar y desarrollar	Fases 2 y 3	Decisión multicriterio; C4, vistas y ADR; taxonomía D1 a D6 y kit; modelo de amenazas; EDT, PERT, nivelación de recursos y Monte Carlo para el plan	3.4, 3.5, 3.7, 4.1 y 4.3 a 4.10
4. Demostrar	Fase 2	Prototipo funcional y ciclo gobernado de punta a punta	5.1 y 5.3 a 5.5
5. Evaluar	Fases 2 y 4	Pruebas de mutación con banco de defectos; intervalos de Wilson y regla de tres; simulación de Monte Carlo y punto de equilibrio	3.6, 3.9, 4.11, 5.2, 5.6 y 5.7
6. Comunicar	Fase 4	Memoria del trabajo y repositorio público del prototipo	Documento completo

Fuente: elaboración propia.

3.2. Revisión de la literatura

La revisión de la literatura se ejecutó como una revisión sistemática asistida y parcial que sigue la estructura de la declaración PRISMA 2020 hasta el cribado por título y resumen (Page et al., 2021). No es una revisión sistemática completa: no se recuperaron ni evaluaron textos completos y no se valoró la calidad de los estudios. La búsqueda se realizó en Scopus el 20 de octubre de 2026 con cuatro cadenas, una por cada tema del apartado 2.3.3, sobre el título, el resumen y las palabras clave, limitadas a publicaciones de 2020 a 2026, a artículos, ponencias y revisiones, y al inglés. La tabla 10 recoge las cadenas exactas y los resultados, y el anexo E muestra las capturas de Scopus que las respaldan. Los resultados se presentan en el apartado 2.3.4.

Tabla 10. *Estrategia de búsqueda ejecutada en Scopus el 20 de octubre de 2026*

Búsqueda y tema	Cadena exacta (Scopus, campo TITLE-ABS-KEY, con filtros)	Resultados exportados
S1. Tema a: emparejamiento de esquemas con modelos de lenguaje	TITLE-ABS-KEY("schema matching" OR "schema alignment" OR "data mapping" OR "data integration") AND TITLE-ABS-KEY("large language model" OR LLM OR "foundation model") AND PUBYEAR > 2019 AND PUBYEAR < 2027 AND (LIMIT-TO (DOCTYPE,"ar") OR LIMIT-TO (DOCTYPE,"cp") OR LIMIT-TO (DOCTYPE,"re")) AND (LIMIT-TO (LANGUAGE,"English"))	1007 (1131 antes de los filtros de tipo e idioma)
S2. Tema b: modelos de lenguaje y agentes en ingeniería de software	TITLE-ABS-KEY ("AI agent" OR "AI agents" OR "LLM agent" OR "LLM agents" OR "agentic AI" OR "multi-agent system" OR "multi-agent systems") AND TITLE-ABS-KEY ("software engineering" OR "system integration" OR "enterprise integration" OR "workflow automation" OR "API integration") AND PUBYEAR > 2019 AND PUBYEAR < 2027 AND (LIMIT-TO (DOCTYPE,"ar") OR LIMIT-TO (DOCTYPE,"cp") OR LIMIT-TO (DOCTYPE,"re")) AND (LIMIT-TO (LANGUAGE,"English"))	768
S3. Tema c: fiabilidad, verificación y pruebas	TITLE-ABS-KEY ("software verification" OR "software testing" OR "mutation testing" OR "test generation" OR "human-in-the-loop" OR traceability) AND TITLE-ABS-KEY ("large language model" OR LLM OR "AI agent" OR "AI agents" OR "agentic AI") AND PUBYEAR > 2019 AND PUBYEAR < 2027 AND (LIMIT-TO (DOCTYPE,"ar") OR LIMIT-TO (DOCTYPE,"cp") OR LIMIT-TO (DOCTYPE,"re")) AND (LIMIT-TO (LANGUAGE,"English"))	3945
S4. Tema d: tecnología aplicada a la factura electrónica	TITLE-ABS-KEY ("electronic invoicing" OR "e-invoicing" OR "e-invoice" OR "electronic invoice" OR "electronic invoices" OR "tax compliance" OR "fiscal compliance") AND TITLE-ABS-KEY (interoperability OR "system integration" OR "data mapping" OR "semantic interoperability" OR "artificial intelligence" OR "large language model" OR "AI agent") AND PUBYEAR > 2019 AND PUBYEAR < 2027 AND (LIMIT-TO (DOCTYPE,"ar") OR LIMIT-TO (DOCTYPE,"cp") OR LIMIT-TO (DOCTYPE,"re")) AND (LIMIT-TO (LANGUAGE,"English"))	86

Fuente: elaboración propia a partir de las búsquedas guardadas en Scopus (20 de octubre de 2026). Total: 5806 registros.

Los criterios de elegibilidad son una operacionalización propuesta por el autor a partir de este apartado, del objetivo general y de la pregunta del apartado 1.2. Se incluyen los registros que tratan de forma central uno de los cuatro temas: modelos de lenguaje o agentes aplicados al mapeo, la alineación o la integración de datos (a); a tareas de ingeniería de software (b); la verificación, las pruebas o la revisión humana de sus salidas, o su uso para verificar software (c); y la tecnología aplicada a la facturación electrónica o al cumplimiento fiscal (d). Se excluyen los registros sin el eje central de modelos de lenguaje o agentes (E1), aquellos en los que los términos del tema son incidentales o el dominio es ajeno (E2), los tipos documentales no elegibles (E3), los que están fuera de periodo o idioma (E4) y los duplicados (E5). Los registros que usan modelos de lenguaje solo para extraer datos de facturas escaneadas (E6) no se excluyen de forma automática: se marcan como «E6 candidata» y quedan como dudosos. Un registro sin resumen nunca se excluye por esa causa.

El proceso fue asistido por herramientas automáticas. Los cuatro archivos exportados se consolidaron y deduplicaron con un script en Python que agrupa los registros que comparten identificador de Scopus (EID) o DOI: los 5806 registros dieron 5659 únicos tras eliminar 147 redundantes, y cinco pares con el mismo título y distinto año se conservaron

como un estudio con dos informes. El cribado por título y resumen lo aplicó un clasificador por reglas, con diccionarios de conceptos sobre el título, el resumen y las palabras clave, que asigna a cada registro Incluir, Dudoso o Excluir y anota la regla que lo decidió. Las reglas se ajustaron una sola vez, tras leer una muestra estratificada de 120 registros, y se contrastaron con otra muestra nueva de 120: la concordancia con ese lector asistido fue preliminar y no independiente, porque el mismo modelo de IA escribió las reglas y leyó las muestras, con un kappa de Cohen de 0,31. Después, el autor verificó una muestra aleatoria de 60 registros, 20 por clase, cuyo resultado se presenta en el apartado 2.3.4. Las cifras proceden de los archivos de la auditoría, que se conservan como material de apoyo junto con los scripts que las reproducen.

El proceso tiene limitaciones que condicionan sus conclusiones. Se consultó una sola base de datos, Scopus. IEEE Xplore, ACM Digital Library, ScienceDirect, Redalyc, SciELO, Dialnet, Google Académico, Re-UNIR y los sitios oficiales (DIAN, SUNAT, BID, CIAT y OCDE) no se consultaron en esta revisión, y la normativa y los informes oficiales citados en el capítulo 2 no proceden de esta búsqueda. Solo se recuperaron trabajos en inglés publicados entre 2020 y 2026, lo que puede omitir literatura latinoamericana en español sobre factura electrónica. El cribado lo realizó un solo cribador asistido por reglas, sin duplicación independiente, y su concordancia con la verificación del autor fue baja (apartado 2.3.4). No se recuperaron textos completos ni se evaluó la calidad de los estudios, de modo que los resultados describen lo que dicen los títulos y los resúmenes, no lo que demuestran los trabajos.

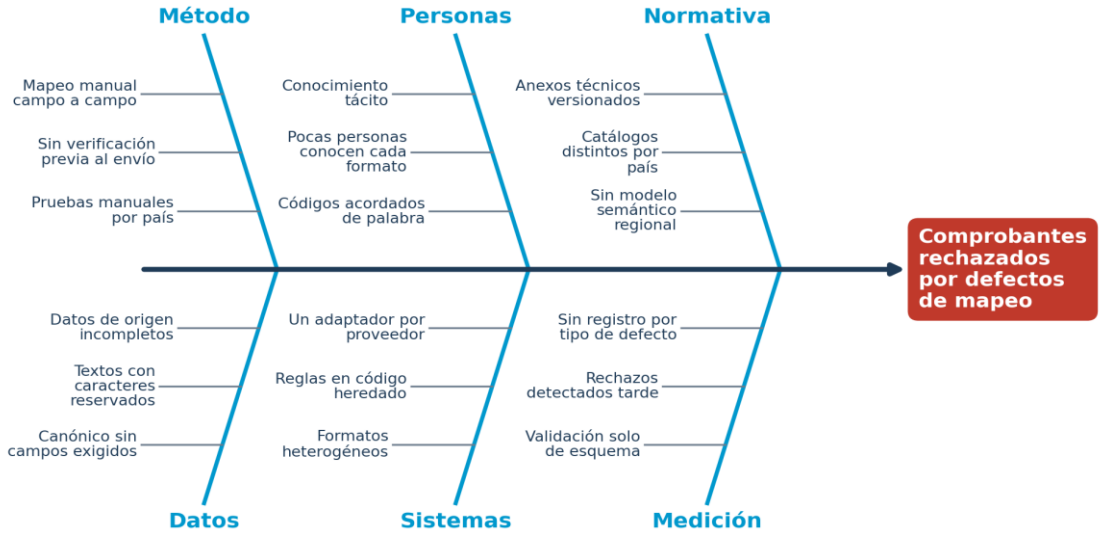
3.3. Métodos de diagnóstico

3.3.1. Diagrama de Ishikawa

El diagrama de causa y efecto, propuesto por Kaoru Ishikawa, organiza las causas posibles de un problema en categorías que convergen en un efecto, y es una de las herramientas básicas del control de calidad (Ishikawa, 1986). Se selecciona porque el rechazo de comprobantes no tiene una causa única: intervienen el método de trabajo, las personas, la normativa, los datos, los sistemas y la forma de medir. El diagrama obliga a considerar todas esas dimensiones antes de proponer una solución y evita atribuir el problema solo a la tecnología.

Las categorías clásicas del método se adaptan al dominio de la integración de facturación electrónica: método, personas, normativa, datos, sistemas y medición. Las causas de cada categoría se identifican a partir de la descripción del proceso actual (apartados 2.1 y 2.2) y de la experiencia del autor en el caso de referencia. La figura 6 presenta el diagrama resultante.

Figura 6. *Diagrama Ishikawa rechazo de comprobantes por defectos de mapeo*



Fuente: elaboración propia.

El diagrama muestra que las causas se refuerzan entre sí. El mapeo manual (método) depende de conocimiento tácito (personas) para interpretar anexos técnicos versionados (normativa), y la única barrera antes del envío es la validación de esquema (medición), que no detecta códigos válidos pero equivocados. Esta lectura justifica que la solución combine un cambio de método, la propuesta asistida por agentes, con un cambio de medición, la verificación determinista, y no solo una de las dos.

3.3.2. Análisis de Pareto

El análisis de Pareto ordena las causas o los tipos de defecto según su frecuencia para concentrar el esfuerzo en los pocos que explican la mayor parte del problema, principio que Juran (1990) denominó la separación de los pocos vitales de los muchos útiles. Se selecciona porque el trabajo necesita priorizar: no todas las reglas del kit de verificación tienen el mismo valor, y el banco de defectos debe cubrir cada tipo con al menos dos mutantes y reforzar el más frecuente.

En un desarrollo de la escala del caso de referencia, el volumen diario de incidencias hace inviable un conteo exhaustivo de defectos por tipo. Por eso el análisis se basa en juicio experto: el autor ordena los seis tipos de la taxonomía según la frecuencia relativa que estima por juicio experto a partir de su experiencia, que no es una frecuencia observada, y el orden se convierte en pesos mediante el método de centroide de rangos, que asigna a cada posición un peso derivado de su lugar en la clasificación y ha mostrado buen desempeño frente a otros métodos de ponderación por rangos (Barron y Barrett, 1996). El resultado se presenta en la tabla 1 y se declara como estimación, no como medición.

3.4. Método de decisión para seleccionar la solución

La selección de la alternativa de solución se realiza con un análisis de decisión multicriterio, en la forma de una matriz de decisión ponderada. El método consiste en definir criterios, asignarles pesos, puntuar cada alternativa en cada criterio y calcular un total ponderado (Romero, 1993). Se selecciona porque elimina la subjetividad de una elección por preferencia: los criterios y los pesos quedan escritos y pueden discutirse, y el resultado se puede reproducir. Es, además, el procedimiento que propone el área de análisis y resolución de decisiones del modelo CMMI para elegir entre alternativas técnicas mediante criterios establecidos (CMMI Product Team, 2010).

Se comparan cinco alternativas: una plataforma comercial de facturación, una plataforma de integración como servicio, el desarrollo tradicional sin inteligencia artificial, el uso de modelos de lenguaje sin verificación y la arquitectura propuesta. Los criterios son la validez normativa de lo emitido, el esfuerzo por país o proveedor nuevo, el control y la trazabilidad, el costo total de propiedad, el riesgo de implantación y la independencia del fabricante. Para comprobar que la decisión es robusta, se aplica un análisis de sensibilidad que varía el peso del criterio más débil de la alternativa ganadora y determina a partir de qué valor cambiaría la decisión.

3.5. Métodos de diseño arquitectónico

El diseño de la arquitectura combina tres métodos complementarios. El primero es el levantamiento de requisitos funcionales a partir del análisis del proceso actual, siguiendo las prácticas del análisis y diseño de sistemas (Kendall y Kendall, 2011). El segundo es la especificación de los atributos de calidad como escenarios medibles, con la forma estímulo, respuesta y medida (Bass et al., 2021), clasificados según el modelo de calidad de producto de ISO/IEC 25010 (ISO/IEC, 2023). Este método se selecciona porque convierte requisitos difusos, como que el sistema sea confiable, en metas verificables, como cero comprobantes perdidos ante un reproceso.

El tercero es la documentación de la arquitectura con el modelo C4, que la describe en niveles de contexto, contenedores y componentes para audiencias distintas (Brown, s. f.), y el registro de cada decisión relevante como un registro de decisión arquitectónica, que documenta el contexto, las opciones, la decisión y sus consecuencias (Nygard, 2011). Estos métodos se seleccionan porque permiten justificar cada elemento de la arquitectura con un escenario de calidad y una decisión documentada, y no solo describirlo.

3.6. Método de validación técnica

La viabilidad técnica del diseño se valida con un prototipo funcional del kit de verificación y del ciclo gobernado de propuestas, construido sobre una especificación y datos sintéticos para no usar información de ninguna organización. El prototipo no sustituye al proyecto planificado: su función es reducir el riesgo técnico de las decisiones críticas antes de comprometer recursos.

Para evaluar el prototipo se aplica la técnica de pruebas de mutación, que mide la capacidad de un mecanismo de verificación para detectar defectos introducidos de forma deliberada (Jia y Harman, 2011). A partir de un mapeo correcto se generan versiones defectuosas, una o más por cada tipo de la taxonomía, y se mide qué porcentaje detecta cada mecanismo: la validación de esquema, el kit en ejecución y el kit con ejemplos validados en el canal de integración continua. Se selecciona porque produce una medida objetiva y reproducible de la eficacia del kit, en lugar de una apreciación.

Las métricas de la evaluación se derivan con el enfoque objetivo, pregunta y métrica (GQM), que exige que cada métrica responda a una pregunta asociada a un objetivo, para no medir por medir (Basili et al., 1994). La tabla 11 presenta las métricas del trabajo.

Tabla 11. *Métricas de evaluación derivadas con el enfoque GQM*

Objetivo	Pregunta	Métrica	Meta
OE1 · Caracterizar	¿Qué tipos de defecto concentran la mayoría de los rechazos?	Frecuencia relativa por tipo de defecto, estimada por juicio experto (análisis de Pareto)	Identificar los tipos que suman al menos el 80 %
OE2 · Diseñar	¿Está la arquitectura documentada de forma completa y verificable?	Decisiones arquitectónicas, vistas, agentes con sus permisos, proceso de revisión humana y amenazas con su control	Al menos diez decisiones; cada amenaza con un control y una decisión o gate asociados
Objetivo general · Validar	¿Qué proporción de los defectos detecta el kit de verificación?	Tasa de detección por tipo: defectos detectados / defectos inyectados, con su intervalo de confianza del 95 %	Al menos 80 % en ejecución y 100 % con ejemplos validados de integración continua
Objetivo general · Validar	¿Rechaza el kit comprobantes correctos?	Falsos positivos sobre salidas correctas, con la cota superior del 95 %	Cero
OE3 · Elaborar	¿Cuál es el plazo y el costo del proyecto con un nivel de confianza conocido?	Duración y costo en el percentil 80 de la simulación	Plazo P80 menor o igual a 12 meses
Objetivo general	¿A partir de cuántos países se recupera la inversión?	Punto de equilibrio n* en número de países o proveedores, bajo la hipótesis de esfuerzo asistido	Calcular n* y su sensibilidad al esfuerzo asistido, y contrastarlo con los países previstos
Hipótesis de esfuerzo (subpregunta 3)	¿Es el esfuerzo asistido menor que el manual?	Horas-persona por incorporación y defectos que llegan al proveedor	Protocolo propuesto (apartado 3.9.3); no se mide en este trabajo

Fuente: elaboración propia.

3.7. Métodos de planificación del proyecto

La planificación del proyecto, contribución central de un trabajo de este tipo, se construye con cinco métodos encadenados. Primero, la estructura de desglose del trabajo descompone el proyecto en paquetes de trabajo con un entregable verificable cada uno (PMI, 2021). Segundo, cada paquete se estima con la técnica PERT, que usa tres valores, optimista, más probable y pesimista, para reflejar la incertidumbre de cada estimación (Malcolm et al., 1959). La duración esperada de cada paquete es $t_e = (o + 4m + p) / 6$.

Tercero, el cronograma se nivela por recursos con un esquema serial de generación de cronogramas, que programa los paquetes en orden de prioridad y los retrasa cuando un rol supera su capacidad (Kolisch, 1996). Este paso se incluye porque un plan que asigna a una persona más trabajo del que puede hacer no es ejecutable. Cuarto, sobre la red nivelada se ejecuta una simulación de Monte Carlo que repite el cálculo del proyecto miles de veces con duraciones aleatorias, y entrega el plazo y el costo como distribuciones de probabilidad (Gray y Larson, 2009). Esto permite comunicar fechas y presupuestos con su nivel de confianza, por ejemplo el percentil 80, y calcular la reserva de contingencia.

Quinto, los riesgos se gestionan con un registro que incluye los riesgos propios de la inteligencia artificial, organizados según las funciones de gobernar, mapear, medir y gestionar del marco del NIST (2023) y alineados con los ejes de mitigación de riesgos de la política nacional de inteligencia artificial (DNP, 2025). Se selecciona este marco porque trata el riesgo de la inteligencia artificial como parte de la gestión del proyecto, con responsables y respuestas, y no como una advertencia técnica.

3.8. Trazabilidad entre objetivos, métodos y entregables

La tabla 12 resume cómo se relacionan los objetivos con las fases, los métodos y los entregables. Cada objetivo específico tiene al menos un método que lo desarrolla y un entregable que lo evidencia, lo que permite verificar en las conclusiones el cumplimiento de cada uno.

Tabla 12. *Trazabilidad entre objetivos, fases, métodos y entregables*

Objetivo	Fase	Métodos	Entregable	Apartado
OE1 · Caracterizar los defectos de mapeo	1. Diagnóstico	Ishikawa, Pareto por juicio experto y centroide de rangos, taxonomía	Causas priorizadas y taxonomía D1 a D6	2.2 y 4
OE2 · Diseñar la arquitectura y el kit	2. Diseño y validación	Decisión multicriterio, C4 y vistas, ADR, escenarios de calidad, modelo de amenazas con STRIDE	Arquitectura de referencia, kit, ADR, permisos de los agentes, revisión humana y modelo de amenazas	4

Objetivo	Fase	Métodos	Entregable	Apartado
OE3 · Elaborar el plan de ejecución	3. Planificación	EDT, PERT, nivelación de recursos, Monte Carlo, gestión de riesgos de IA	Plan con plazo y costo P80, presupuesto y riesgos	4 y 5
Objetivo general	2. Diseño y validación; 4. Evaluación y cierre	Prototipo, banco de defectos con pruebas de mutación, Wilson y regla de tres, GQM y matriz de cumplimiento de objetivos	Resultados del banco, evaluación y conclusiones	3.9, 5 y 6

Fuente: elaboración propia.

3.9. Protocolo de evaluación experimental

Este apartado formaliza cómo se evalúa el artefacto, en coherencia con la actividad de evaluación del DSRM. Separa tres evaluaciones que responden a preguntas distintas: la eficacia del kit de verificación, medida con un banco de defectos inyectados; la calidad de las propuestas del agente de mapeo, medida con independencia del verificador; y la comparación del esfuerzo manual frente al asistido, que se deja como protocolo para la fase F5. La detección de defectos inyectados demuestra la eficacia del verificador, pero no, por sí sola, la reducción del esfuerzo total; por eso cada evaluación tiene sus propias métricas y no se combinan en un único indicador. Cada subapartado precisa además su estado: la evaluación del verificador y la del motor heurístico del agente se ejecutaron en el prototipo, la del motor basado en un modelo de lenguaje se ejecuta en la fase F4 y la comparación de esfuerzo es un protocolo propuesto. El último subapartado indica cómo reproducir lo medido.

3.9.1. Evaluación del verificador

El banco combina casos correctos y defectuosos sobre la especificación sintética del apartado 5.1. Los correctos son 300 comprobantes generados con semilla fija (2026) y 12 ejemplos validados por personas (semilla 7), que cubren cada tipo de documento, de identificación y de tributo, además de datos adversarios. Cada comprobante se transforma con el mapeo de referencia a los dos formatos del prototipo, el XML del Perfil Alfa y el texto por segmentos del proveedor Beta: son 600 salidas correctas para el esquema y el kit en ejecución, y 24 para integración continua. Los defectuosos son 22 mutantes, cada uno una especificación de mapeo obtenida al aplicar un operador de mutación al mapeo de referencia y ejecutada sobre los mismos comprobantes y ejemplos; el reparto es de 7, 4, 4, 3, 2 y 2 mutantes para D1 a D6.

La verdad de referencia se fija por construcción: toda salida del mapeo de referencia es correcta, y toda salida de un mutante que difiere de la correcta del mismo comprobante es defectuosa; las idénticas no cuentan como afectadas. Un mutante equivalente es el que produce salidas idénticas a las correctas en todos los comprobantes y ejemplos; ningún mecanismo puede detectarlo y se excluye (Jia y Harman, 2011). En el banco no hay mutantes

equivalentes: D6-b no afecta a ningún comprobante de la muestra, pero sí a un ejemplo adversario.

La unidad de análisis es el mutante para la detección y la verificación de una salida correcta para los falsos positivos. La tabla 13 define los resultados en cada mecanismo. La métrica principal es la tasa de detección, TP / (TP + FN); se informan también la tasa de falsos positivos, FP / (FP + TN), y la precisión, la proporción de rechazos que corresponden a salidas defectuosas. Cada tasa se acompaña de su intervalo de confianza del 95 % por el método de Wilson (1927), adecuado para proporciones cercanas a 0 o a 1, y, sin falsos positivos, de la cota superior 3/n de la regla de tres (Hanley y Lippman-Hand, 1983). El umbral del 80 % en ejecución es una meta de diseño del autor: retener antes del envío al menos cuatro de cada cinco defectos y dejar el resto a los ejemplos validados, cuya meta es el 100 %. El de cero falsos positivos responde a que un falso positivo bloquea un comprobante válido; como cero eventos no prueban una tasa nula, se informa su cota. Con 22 mutantes, el límite inferior de Wilson solo supera el 80 % si se detectan los 22; con la proporción observada en ejecución harían falta unos 142.

Tabla 13. *Definiciones de la evaluación del verificador*

Concepto	Solo esquema	Kit en ejecución	Kit con ejemplos validados (CI)
Entrada evaluada	Salidas correctas y salidas de cada mutante, en ambos formatos	Las mismas que el esquema	Las del kit en ejecución más los ejemplos validados y sus versiones mutadas
Verdadero positivo (TP)	Mutante rechazado por el esquema en al menos una salida afectada	Mutante rechazado por el esquema o las reglas en al menos una salida afectada	Mutante rechazado en ejecución o que difiere de al menos un ejemplo validado
Falso negativo (FN)	Mutante no rechazado en ninguna salida afectada	Mutante no rechazado por el esquema ni por las reglas	Mutante no rechazado en ejecución ni por los ejemplos
Falso positivo (FP)	Salida correcta rechazada por el esquema	Salida correcta rechazada por el esquema o las reglas	Ejemplo correcto rechazado
Verdadero negativo (TN)	Salida correcta aceptada	Salida correcta aceptada	Ejemplo correcto aceptado
n en el banco actual	22 mutantes; 600 salidas correctas	22 mutantes; 600 salidas correctas	22 mutantes; 24 ejemplos correctos

Fuente: elaboración propia.

3.9.2. Evaluación del agente de mapeo

La evaluación del agente se separa de la del verificador porque responde a otra pregunta: qué calidad tienen las propuestas antes de cualquier control. La entrada es una especificación de proveedor y el modelo canónico; en el prototipo, la especificación del Perfil Alfa v2.1, con 24 campos, y en el proyecto, las especificaciones del conjunto dorado que se construye en la fase F4. La métrica principal es la exactitud por campo: la proporción de campos cuya correspondencia propuesta coincide con la del mapeo de referencia validado por una persona. Se informan además la proporción de propuestas bloqueadas por el kit y, entre

los campos erróneos, cuántos detecta el kit y cuántos llegarían a la revisión humana, de modo que el verificador actúa aquí como instrumento de medida del agente y no como su sustituto. Un bloqueo del kit sobre una propuesta errónea no cuenta como acierto del agente, y un error del agente que el kit no detecta no cuenta como falso negativo del banco, porque el banco mide mutantes diseñados y no propuestas. La línea base es el motor heurístico determinista, que en la prueba de concepto acertó 23 de 24 campos (95,8 %) y cuyo único error bloqueó el kit (apartado 5.3). El motor basado en un modelo de lenguaje se compara con esa línea base en la fase F4, con el arnés de evaluación, la versión del modelo fijada, el costo por propuesta registrado y varias ejecuciones por especificación para estimar su variabilidad; el número de ejecuciones se fija al diseñar el arnés y no se adelanta aquí.

3.9.3. Comparación entre el proceso manual y el asistido

Este subapartado describe un protocolo propuesto, no ejecutado, para comprobar la hipótesis de la subpregunta 3: que el esfuerzo de incorporar un país o proveedor con asistencia de los agentes es menor que el manual. Ningún resultado de este trabajo procede de su aplicación. La tarea comparada es la incorporación completa de un país o proveedor: lectura de la especificación, mapeo, plantillas, pruebas y paso por el kit hasta la aprobación. Se miden dos variables: el esfuerzo en horas-persona, registrado por tarea y por rol en el sistema de seguimiento del proyecto, y el número de defectos que llegan al proveedor, contados como rechazos en su ambiente de pruebas y en las primeras semanas de producción, separados de los que el kit retuvo antes del envío. La línea base manual es la del modelo de planificación, 27,7 personas-semana para el primer país sin agentes (paquete 6.1), y la hipótesis asistida es de 20,8 personas-semana (paquete 6.2).

El procedimiento se aplica en la fase F5. El primer país se incorpora sin agentes y su esfuerzo medido sustituye a la línea base estimada; el segundo se incorpora con agentes, con el mismo equipo, el mismo kit y una especificación de complejidad comparable, medida por número de campos y formato. La hipótesis se considera apoyada si el esfuerzo asistido, normalizado por campo, es menor que el manual sin que aumenten los defectos que llegan al proveedor. Las amenazas principales son el efecto de aprendizaje, porque el segundo país se beneficia de lo aprendido en el primero; la diferencia de complejidad entre países; y el tamaño, porque una sola pareja es un estudio de caso y no permite inferencia estadística. Se mitigan registrando el aprendizaje como categoría propia de esfuerzo, normalizando por campo y repitiendo la medición en las incorporaciones siguientes.

3.9.4. Reproducibilidad

Los resultados medidos del prototipo pueden regenerarse desde su repositorio público (AlejoTechEngineer/veriagent-einvoice-latam, commit 0b6406c de la rama principal). La

generación de datos usa semillas fijas: 2026 para los 300 comprobantes y 7 para los 12 ejemplos validados; la simulación de Monte Carlo usa la semilla 2026 y 10.000 iteraciones. Las versiones de referencia son Java 21 para el kit y Python 3.12 para el plano agéntico, y el archivo prototipo/docs/VERSIONES.md fija las de cada librería. El banco se ejecuta con prototipo/agentic-plane/tfg_agents/evaluation.py, que aplica los operadores de prototipo/agentic-plane/tfg_agents/mutations.py y escribe el resultado en prototipo/results/evaluation.json; el mismo cálculo corre como gate en la etapa gate-bank de prototipo/ci/run-pipeline.sh, que detiene el pipeline si la detección en ejecución queda por debajo del umbral, si integración continua no detecta todos los mutantes o si hay falsos positivos. El modelo de planificación está en prototipo/planning/model.py, con los paquetes en prototipo/planning/wbs.json y los supuestos en prototipo/planning/assumptions.json. Las huellas completan la trazabilidad: cada hallazgo del kit incluye el SHA-256 de la especificación que lo originó, cada propuesta guarda la huella de la especificación con que se generó y la etapa de empaquetado publica el SHA-256 del JAR del kit. Así, un tercero puede comprobar que obtiene los mismos números con el mismo código, los mismos datos y la misma versión de la norma.

4. Desarrollo de la solución

Este capítulo desarrolla la solución que se planifica. Parte de la aportación central, el kit de verificación (apartado 4.1), y la sitúa en su contexto: los requisitos y atributos de calidad que debe cumplir (4.2), la selección de la alternativa (4.3) y la arquitectura de referencia con sus decisiones (4.4 y 4.5). Después desarrolla lo que constituye la contribución propia de un trabajo de planificación: los perfiles del equipo (4.6), la planificación de tareas, esfuerzos y cronograma (4.7), el presupuesto (4.8), el análisis de riesgos (4.9), el plan de calidad, la integración y despliegue continuos y el gobierno del proyecto (4.10) y la evaluación de la solución (4.11).

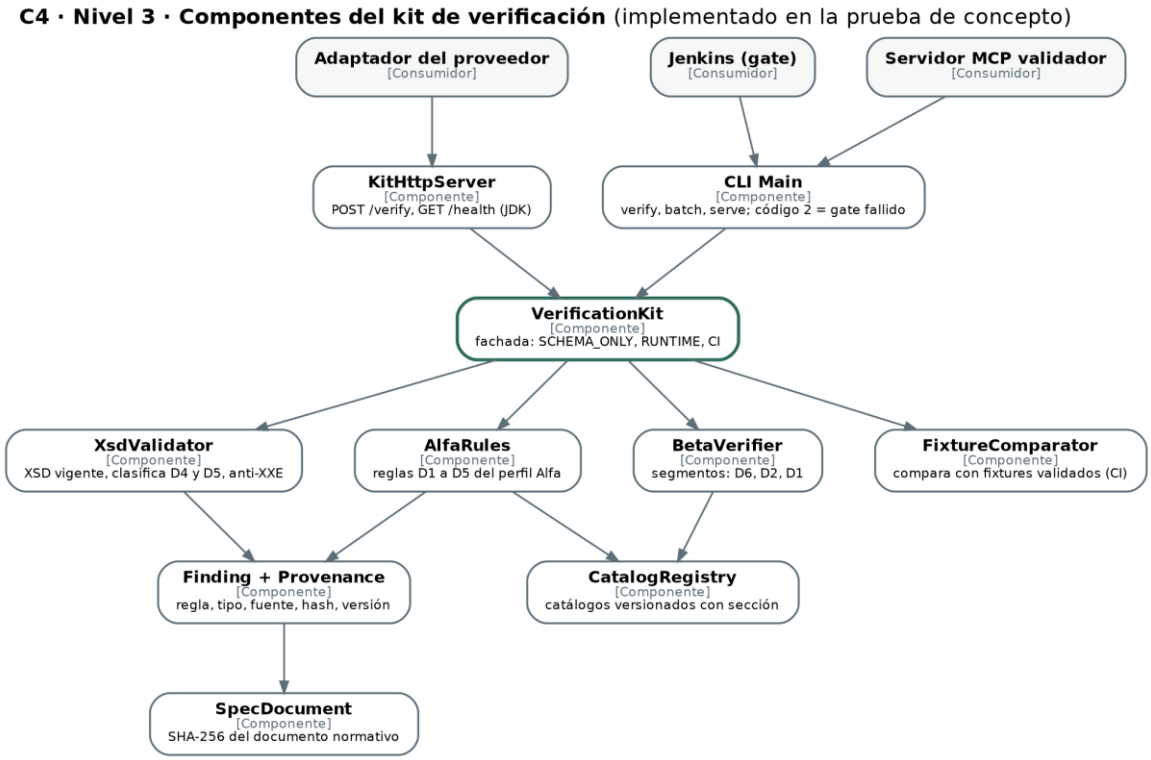
4.1. Aportación: taxonomía de defectos y kit de verificación

La aportación central del trabajo es un **kit de verificación determinista** construido a partir de la taxonomía de defectos de mapeo D1 a D6 caracterizada en el capítulo 2. El kit recibe un comprobante transformado al formato del proveedor y lo somete a tres controles sucesivos: la validación contra el esquema vigente, las reglas de negocio derivadas de cada tipo de defecto y, en el canal de integración continua (CI), la comparación con ejemplos validados por personas (fixtures). Cada hallazgo informa la regla que lo produjo, el tipo de defecto, la sección del documento normativo de la que proviene y la huella digital de ese documento, de modo que cualquier rechazo puede rastrearse hasta la norma.

El kit opera en dos modos. En **modo de ejecución**, se integra en los adaptadores del flujo asíncrono y en los conectores del flujo síncrono, y bloquea cualquier comprobante defectuoso antes de que salga hacia el proveedor. En **modo de integración continua**, actúa como control obligatorio sobre las propuestas de los agentes: una propuesta de mapeo que no supera el kit no puede aprobarse ni desplegarse. Para validar su viabilidad técnica se construyó un prototipo funcional del kit en Java 21, sin dependencias externas, junto con un agente de mapeo que propone correspondencias a partir de la especificación. El prototipo trabaja sobre una especificación y datos sintéticos, para no usar información de ninguna organización.

El kit es el componente más crítico, y por eso es el que implementa la prueba de concepto. La figura 7 muestra su estructura interna. Una fachada ofrece tres modos: solo esquema, ejecución e integración continua. Cada hallazgo lleva la regla que lo produjo, su tipo en la taxonomía, la sección de la especificación, la huella digital (hash SHA-256) del documento normativo y la versión de la regla.

Figura 7. *Componentes del kit de verificación (C4, nivel 3)*



Fuente: elaboración propia. Componentes implementados en la prueba de concepto.

La tabla 14 formaliza la taxonomía D1 a D6 introducida en la tabla 3: para cada tipo precisa su definición, su ámbito, las reglas deterministas que lo detectan con su identificador en el kit, el operador de mutación que lo inyecta en el banco y el mecanismo que se espera que lo detecte. A estas reglas se suma la comparación con ejemplos validados (CI-FX-01), común a todos los tipos, que solo actúa en integración continua.

Tabla 14. *Definición formal de la taxonomía D1 a D6*

Tipo	Definición precisa	Ámbito	Regla determinista (ID)	Operador de mutación	Detección esperada
D1	El campo codificado lleva un código inexistente en el catálogo oficial vigente o un código válido que no corresponde al dato de origen	Tipo de documento, tipo de identificación y tributo, en XML y en segmentos	R-D1-01 código existente en el catálogo; R-D1-02 tipo de identificación y formato del número; R-D1-03 tributo y tarifa; R-D1-04 coherencia del tipo con la referencia de facturación	Sustitución de código: cambia una entrada de la tabla de códigos por otro código, válido o inexistente	Kit en ejecución; si el código es válido y de formato idéntico (D1-e), solo los ejemplos validados

Tipo	Definición precisa	Ámbito	Regla determinista (ID)	Operador de mutación	Detección esperada
D2	Un campo destino toma un valor de origen distinto del especificado, rompa o no las invariantes aritméticas	Cantidades, precios, valores de línea, base, impuesto y total	R-D2-01 cantidad × precio = valor de línea; R-D2-02 base = suma de líneas y total = base + impuesto; R-D2-03 impuesto = suma de base × tarifa; R-D2-04 cantidad positiva y precio no negativo	Permutación de las fuentes de dos campos o reasignación de la fuente de un campo	Kit en ejecución; si la permutación conserva las invariantes (D2-a), solo los ejemplos validados
D3	Un literal o un valor de lista cerrada difiere de la cadena exacta exigida en mayúsculas, espacios o grafía	Identificador de perfil, moneda y nombre del esquema tributario	R-D3-01 identificador de perfil exacto; R-D3-02 nombre del esquema tributario exacto; R-D3-03 moneda de la lista cerrada	Alteración de literal: cambia mayúsculas o añade espacios en un valor fijo o de tabla	Kit en ejecución; el esquema no lo detecta
D4	Un campo obligatorio del formato destino queda sin mapear o con valor vacío	Correo del adquirente, número, fecha de emisión y nombres de las partes	R-D4-01 campo obligatorio presente y no vacío	Eliminación del mapeo de un campo o sustitución por un literal vacío	Esquema si falta el elemento; solo el kit si está vacío
D5	Se envía un elemento retirado en la versión vigente del estándar	Elementos de la lista de retirados del catálogo	R-D5-01 elemento retirado presente	Inserción de un elemento de la versión anterior	Esquema y kit en ejecución
D6	Un carácter reservado del formato aparece sin sanear en un texto libre y desplaza los campos	Textos libres del formato por segmentos	R-D6-01 número de campos por segmento; R-D6-02 campo numérico no numérico	Desactivación del saneamiento de un texto libre	Validación estructural y kit si los datos contienen el separador; si no (D6-b), solo los ejemplos con datos adversarios

Fuente: elaboración propia.

Cada regla del kit lleva un registro de procedencia con cuatro campos: el documento normativo de origen, la sección que la fundamenta, la huella SHA-256 de ese documento y la versión de la regla. La huella se calcula sobre los bytes del documento cada vez que el kit lo carga, de modo que cada hallazgo informa la versión exacta del texto con que se evaluó; en el prototipo, la especificación del Perfil Alfa v2.1 tiene la huella ac6fe10e…61f3a y las reglas, la versión 2.1.0. Las propuestas de los agentes guardan la misma huella en el campo spec_sha256 del registro de propuestas, y la tabla rule_provenance del modelo de datos (tabla 22) la conserva para cada regla con su vigencia. Cuando cambia la versión de la norma cambia la huella: los hallazgos y las propuestas posteriores quedan asociados al nuevo documento, y las reglas cuya procedencia apunta a la huella anterior deben revisarse hasta que una persona confirme o actualice su sección y su versión. En el prototipo esa marca no está automatizada; el control que la impone se describe en el apartado 4.4.11.

4.2. Requisitos y atributos de calidad

4.2.1. Requisitos funcionales

Tabla 15. *Requisitos funcionales*

ID	Requisito	Prioridad
RF-01	Recibir comprobantes por SFTP y REST, validar el canónico con JSON Schema y publicarlos por país	Alta
RF-02	Enrutar cada comprobante al proveedor según país, tipo de documento y una tabla versionada	Alta
RF-03	Transformar al formato del proveedor y verificar con el kit antes de enviar	Alta
RF-04	Procesar respuestas con fuente primaria y respaldo, y actualizar el estado	Alta
RF-05	Archivar PDF, XML y constancia con inmutabilidad y retención por país	Alta
RF-06	Consultar estado y auditoría de cada comprobante por su identificador de correlación	Media
RF-07	Generar desde la especificación del proveedor una propuesta de mapeo, plantillas y pruebas como solicitud de cambio (pull request)	Alta
RF-08	Extraer reglas de código heredado con evidencia de archivo y línea	Media
RF-09	Clasificar los fallos de la DLQ según D1 a D6 y proponer la corrección en origen	Media
RF-10	Registrar cada propuesta de IA con su verificación, revisor y decisión; impedir aprobar lo que no pasó el kit	Alta

Fuente: elaboración propia.

4.2.2. Escenarios de atributos de calidad

Cada escenario sigue la forma estímulo, respuesta y medida (Bass et al., 2021) y se clasifica según ISO/IEC 25010 (ISO/IEC, 2023).

Tabla 16. *Escenarios de atributos de calidad*

Atributo	Estímulo	Medida de respuesta
Eficiencia de desempeño	Día pico de emisión	Al menos 200.000 comprobantes procesados sin degradar el flujo síncrono
Eficiencia de desempeño	Solicitud síncrona	p95 menor a 3 s, sin contar el tiempo del proveedor
Fiabilidad	Operación mensual	99,9 % de disponibilidad en fachada e ingesta
Fiabilidad	Reproceso o mensaje duplicado	Cero pérdidas y cero duplicados gracias a la idempotencia
Adecuación funcional	Comprobante listo para salir	100 % pasa XSD y reglas antes del envío
Mantenibilidad	Nuevo proveedor	Incorporación con arquetipo y agente en una fracción del esfuerzo manual, medida con la línea base
Seguridad	Archivo comprimido malicioso, XXE o secreto expuesto	Rechazo por límites y por configuración segura; cero secretos en código o imágenes; agentes con mínimo privilegio
Seguridad (responsabilidad)	Auditoría de una regla o propuesta de IA	Trazable hasta su fuente normativa, su versión y su aprobador

Fuente: elaboración propia.

4.2.3. Restricciones

- Los agentes no pueden desplegar ni escribir en producción.

- Los datos tributarios reales no salen del perímetro de la organización; el desarrollo y la evaluación usan datos sintéticos.
- La retención del archivo legal cumple el plazo que fije cada país.
- Las credenciales se gestionan en un almacén de secretos con identidad de carga de trabajo.

4.3. Selección de la alternativa de solución

Cada alternativa se puntúa de 1 (peor) a 5 (mejor) en seis criterios ponderados. La arquitectura propuesta obtiene 4,05 sobre 5, frente a 3,35 del desarrollo tradicional, la segunda opción. Las puntuaciones son una valoración del autor y se validarán con un panel de expertos en un trabajo posterior.

Tabla 17. *Matriz de decisión ponderada*

Criterio	Peso	Plataforma comercial	iPaaS	Desarrollo tradicional	LLM sin verificación	Propuesta
Validez normativa de lo emitido	25 %	4	3	4	1	5
Esfuerzo por país o proveedor nuevo	20 %	4	2	1	4	4
Control y trazabilidad	15 %	2	3	4	1	5
Costo total de propiedad	15 %	2	3	3	4	3
Riesgo de implantación	15 %	4	4	4	1	2
Independencia del fabricante	10 %	1	2	5	4	5
Total ponderado	**100 %**	**3,10**	**2,85**	**3,35**	**2,35**	**4,05**

Fuente: elaboración propia.

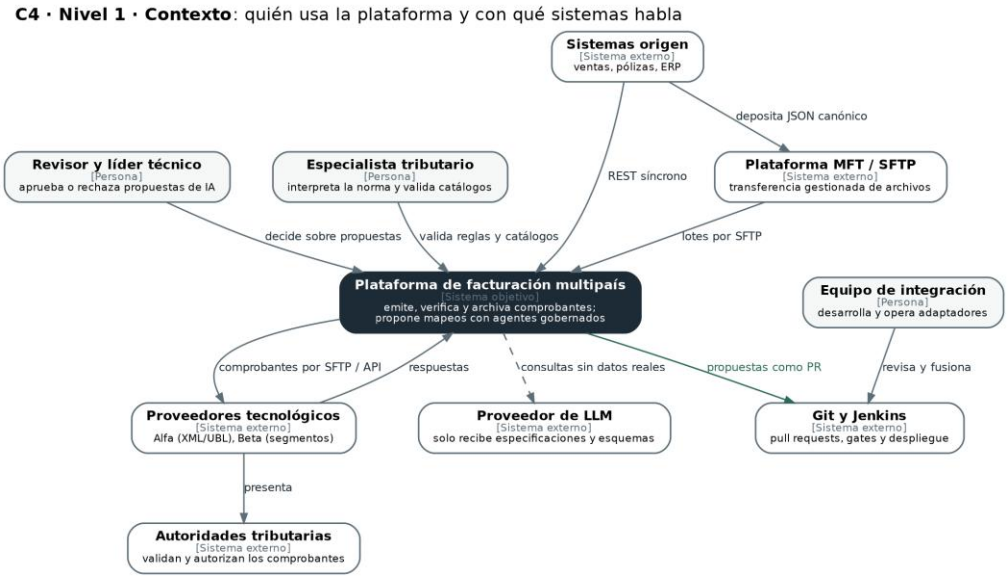
Análisis de sensibilidad. El punto débil de la propuesta es el riesgo de implantación. Si su peso sube y los demás se reducen en proporción, la propuesta sigue ganando mientras ese peso sea menor al 37 %; por encima gana el desarrollo tradicional. La decisión es robusta, pero depende de que el plan reduzca de verdad el riesgo de implantación. Por eso el kit de verificación se construye antes que los agentes: aporta valor por sí solo y, como muestra la prueba de concepto, ya detecta el 86 % de los defectos inyectados.

4.4. Arquitectura de referencia

4.4.1. Contexto del sistema (C4, nivel 1)

La figura 8 muestra quién usa la plataforma y con qué sistemas se comunica. Dos decisiones aparecen ya en este nivel: el proveedor de LLM nunca recibe datos reales, solo especificaciones y esquemas; y todo cambio propuesto por la IA viaja como solicitud de cambio hacia Git y Jenkins.

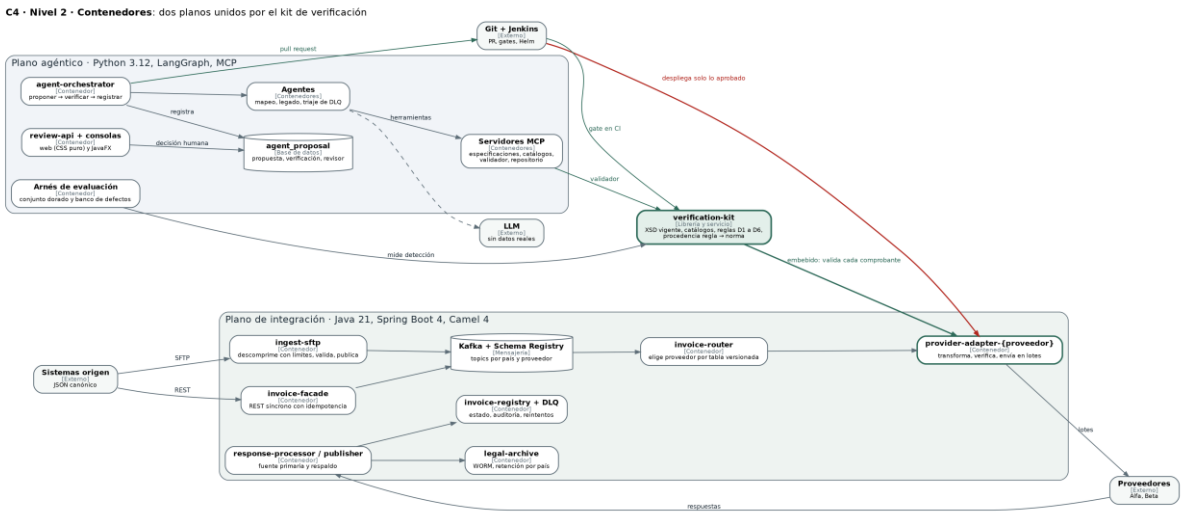
Figura 8. *Diagrama de contexto (C4, nivel 1)*



Fuente: elaboración propia con la notación del modelo C4 (Brown, s. f.).

4.4.2. Contenedores (C4, nivel 2)

Figura 9. *Diagrama de contenedores (C4, nivel 2)*



Fuente: elaboración propia. En verde, la ruta de verificación; en rojo, el único camino hacia producción.

La arquitectura separa dos planos unidos por el kit de verificación. La tabla 18 detalla los diez microservicios del plano de integración, con sus entradas y salidas.

Tabla 18. *Microservicios del plano de integración*

#	Microservicio	Responsabilidad	Entrada	Salida
1	ingest-sftp	Polling del MFT, descompresión acotada, validación del canónico con JSON Schema y publicación por país	Archivos JSON/ZIP por SFTP	{pais}.invoice.request.v1

#	Microservicio	Responsabilidad	Entrada	Salida
2	invoice-facade	API REST síncrona con clave de idempotencia; invoca el conector síncrono	REST (OpenAPI 3.1)	Respuesta HTTP y evento de trazabilidad
3	invoice-router	Elige el proveedor según país, tipo de documento y tabla de enrutamiento versionada	Topic de solicitud por país	{pais}.{proveedor}.request.v1
4	provider-adapter-{proveedor}	Transforma al formato propietario, verifica con el kit, persiste y envía en lotes	Topic del proveedor	ZIP al proveedor y estado en BD
5	provider-connector-{proveedor}	Conector síncrono del proveedor	Llamada de la fachada	Respuesta del proveedor
6	response-processor	Correlaciona respuestas con fuente primaria y respaldo y actualiza el estado	SFTP de retorno	{pais}.invoice.response.v1
7	response-publisher	Devuelve la respuesta al sistema origen en su formato	Topic de respuesta	SFTP o REST hacia el origen
8	legal-archive	Archiva PDF, XML y constancia con inmutabilidad y retención	Evento de autorización	Almacenamiento WORM
9	invoice-registry	Consulta de estado y auditoría de cada comprobante	REST	Vista de trazabilidad
10	dlq-handler	Reintento controlado y clasificación de errores no recuperables	{pais}.invoice.dlq.v1	Reintento, cuarentena o triaje

Fuente: elaboración propia.

Dos piezas transversales sostienen la reutilización. **verification-kit** es una librería Java embebida en cada adaptador y usada como gate en integración continua. **adapter-archetype** es un arquetipo Maven que genera un adaptador nuevo con sus pruebas. El plano agéntico, descrito en la tabla 19, se comunica con el de integración solo a través del kit y del repositorio.

Tabla 19. *Componentes del plano agéntico*

#	Componente	Responsabilidad
11	agent-orchestrator	Coordina los agentes, mantiene el estado del flujo y exige aprobación humana antes de cualquier cambio
12	mapping-agent	Genera desde la especificación la tabla de mapeo, las plantillas y las pruebas; entrega una solicitud de cambio, nunca despliega
13	legacy-extraction-agent	Extrae reglas de código heredado con evidencia de archivo y línea
14	dlq-triage-agent	Clasifica los fallos de la DLQ según la taxonomía y propone la corrección en origen
15	mcp-spec-reader	Servidor MCP: búsqueda sobre las especificaciones con sección y huella digital
16	mcp-catalog	Servidor MCP: catálogos oficiales versionados por país
17	mcp-validator	Servidor MCP: expone el kit para que los agentes validen sus propias propuestas
18	mcp-repo	Servidor MCP: crea ramas y solicitudes de cambio, sin permiso de merge
19	eval-harness	Evaluación continua contra el conjunto dorado y el banco de defectos

Fuente: elaboración propia.

4.4.3. Stack tecnológico

Tabla 20. *Lenguajes y su justificación*

Lenguaje	Uso	Justificación
Java 21 (LTS)	Microservicios del plano de integración y kit	Hilos virtuales para E/S intensiva; ecosistema maduro de integración

Lenguaje	Uso	Justificación
Python 3.12	Agentes, servidores MCP y evaluación	Ecosistema de LLM y MCP más maduro (ADR-03)
HTML, CSS puro y JavaScript sin dependencias (web); JavaFX (escritorio)	Consolas de revisión humana	Sin frameworks en la web; escritorio nativo para revisores
SQL	Persistencia relacional y migraciones	Trazabilidad transaccional
YAML	Rutas Camel, Kubernetes, Helm, JCasC	Configuración declarativa y versionada

Fuente: elaboración propia.

Tabla 21. *Frameworks, librerías e infraestructura*

Capa	Tecnología
Integración (Java)	Spring Boot 4, Apache Camel 4 (kafka, sftp, zipfile, yaml-dsl), Jackson, Resilience4j, MapStruct
Plano agéntico (Python)	SDK de Anthropic con salida estructurada, SDK de MCP, LangGraph, FastAPI, Pydantic, lxml
Pruebas	JUnit 5, Testcontainers, WireMock, Pact, ArchUnit; pytest e Hypothesis
Calidad y seguridad	SonarQube, OWASP Dependency-Check, Trivy, Checkstyle
Cómputo	Kubernetes administrado con Helm; HPA y NetworkPolicy
Mensajería	Kafka con Schema Registry (compatibilidad BACKWARD), SASL/OAUTHBEARER sobre TLS
Datos	SQL Server con Flyway; PostgreSQL con pgvector para RAG con procedencia
Archivo legal	Almacenamiento de objetos con ciclo de vida e inmutabilidad WORM
Secretos e identidad	Key Vault y Workload Identity; cero secretos en código o imágenes
Observabilidad	OpenTelemetry, Prometheus, Grafana y alertas
CI/CD	Jenkins como código (JCasC), Helm, promoción SIT → UAT → PROD con aprobación

Fuente: elaboración propia.

4.4.4. Contratos e interfaces

- **Modelo canónico:** InvoiceRequest e InvoiceResponse, registrados en Schema Registry con versionado semántico. La prueba de concepto implementa InvoiceRequest con Pydantic e invariantes de negocio.
- **Convención de topics:** {pais}.invoice.{request|response|dlq}.v{n} y {pais}.{proveedor}.{request|response}.v{n}, particionados por número de comprobante.
- **APIs REST:** OpenAPI 3.1 con enfoque contract-first y pruebas de contrato con Pact.
- **API del kit:** POST /verify?format=xml|seg&mode=RUNTIME|CI devuelve 200 si el documento pasa y 422 con los hallazgos si no; GET /health expone la huella digital de la especificación cargada.
- **API de revisión:** GET /api/proposals, GET /api/proposals/{id} y POST /api/proposals/{id}/decision, que responde 409 si se intenta aprobar una propuesta que no pasó el kit.

- **Correlación de punta a punta:** un correlationId único viaja por eventos, base de datos y trazas de OpenTelemetry.

4.4.5. Modelo de datos

Tabla 22. *Tablas del núcleo del modelo de datos*

Tabla	Columnas principales	Propósito
authorization_log	id, correlation_id (único), número, tipo, país, proveedor, estado, payload transformado, código y mensaje de error, retry_count, received_at, sent_at, authorized_at, updated_at	Estado y auditoría de cada comprobante; índices por (estado, received_at) y por número
rule_provenance	rule_id, versión, país, proveedor, sección de origen, huella digital del documento fuente, vigencia	Trazabilidad de cada regla hasta la norma
agent_proposal	id, agente, tipo, spec_sha256, mapeo, resultado de verificación, gate_passed, estado, revisor, comentario, decided_at	Auditoría completa de la IA; implementada en la prueba de concepto

Fuente: elaboración propia.

4.4.6. Seguridad por diseño

- **Mínimo privilegio para la IA:** los agentes solo usan herramientas de lectura y crean solicitudes de cambio; una NetworkPolicy les impide alcanzar sistemas de producción.
- **Datos:** el LLM recibe especificaciones y esquemas, nunca comprobantes reales; la evaluación usa datos sintéticos.
- **Entradas hostiles:** el kit rechaza DOCTYPE y entidades externas (XXE) y limita el tamaño de los documentos; la ingesta limita la descompresión.
- **Contenedores:** usuario sin privilegios, sistema de archivos de solo lectura y capacidades eliminadas.
- **Cadena de suministro:** escaneo de dependencias, imágenes y secretos en el pipeline; huella digital del artefacto publicado.

4.4.7. Vistas de la arquitectura

La arquitectura se documenta por vistas, cada una dirigida a una audiencia con preocupaciones propias, siguiendo la práctica de describir un sistema mediante varias estructuras complementarias (Bass et al., 2021) y los niveles del modelo C4 (Brown, s. f.). La tabla 23 identifica siete vistas y el lugar del documento donde se representa cada una. Las de contexto, contenedores y componentes corresponden a los tres primeros niveles del modelo C4; la dinámica describe el comportamiento en el tiempo del ciclo gobernado; la de despliegue, los nodos de ejecución y sus políticas de red; la de datos, las tablas que sostienen la trazabilidad; y la de seguridad reúne las fronteras de confianza, los permisos y los controles.

Tabla 23. *Vistas de la arquitectura*

Vista	Audiencia	Preocupaciones	Dónde se representa
Contexto	Dirección, áreas tributaria y de seguridad, proveedores	Actores, sistemas externos y datos que cruzan la frontera (el LLM no recibe datos reales)	Figura 8 (apartado 4.4.1)
Contenedores	Arquitecto, líder técnico, DevOps	Microservicios de los dos planos, responsabilidades y comunicación	Figura 9; tablas 18 y 19
Componentes	Desarrolladores del kit	Estructura interna del kit y sus modos de operación	Figura 7 (apartado 4.1)
Dinámica	Revisores, líder técnico, auditoría	Secuencia y estados de una propuesta de IA hasta su aprobación	Figura 10; tabla 27
Despliegue	DevOps y SRE, seguridad	Nodos, réplicas, escalado, contexto de seguridad y políticas de red	Tablas 24 y 42
Datos	Equipo de integración, auditoría	Tablas del núcleo, trazabilidad y procedencia de las reglas	Tabla 22; apartado 4.1
Seguridad	Seguridad, auditoría y cumplimiento	Fronteras de confianza, permisos, amenazas y controles	Apartados 4.4.6 y 4.4.8 a 4.4.11

Fuente: elaboración propia.

La vista de despliegue se toma del chart de Helm del prototipo y corresponde al estado del chart en el commit 0b6406c del repositorio (tabla 24). El chart despliega dos cargas de trabajo y una política de red. En ese commit, la política permitía al plano agéntico salir por el puerto 443 hacia cualquier destino. La versión 0.3.0 del chart (commit 51d8bb7) elimina esa regla y limita la salida a cuatro destinos: el kit (8085), la plataforma (8080), un proxy de salida (3128) y el DNS del clúster (53). Esa política está escrita y comprobada con una prueba estática sobre la plantilla, pero no se ha ejecutado en un clúster (apartados 4.4.10 y 4.10.2). El chart sigue sin definir una política de entrada para el kit.

Tabla 24. *Despliegue del prototipo según el chart de Helm (commit 0b6406c)*

Elemento	Componentes	Configuración	Política de red
Despliegue del kit	Kit de verificación, servicio HTTP en el puerto 8085	Dos réplicas y autoescalado de 2 a 6 al 70 % de CPU; en producción, tres réplicas y de 3 a 10 al 65 %; sondas de salud	Sin política de entrada definida en el chart
Despliegue del plano agéntico	API de revisión, agente y orquestador, puerto 8090	Una réplica; volumen persistente de 1 GiB para el registro de propuestas; sin montaje automático del token de la cuenta de servicio; clave del LLM opcional desde un secreto	En el commit 0b6406c: kit (8085), cualquier destino por 443 y DNS (53/UDP). En el chart 0.3.0 (commit 51d8bb7): kit (8085), plataforma (8080), proxy de salida (3128) y DNS (53); escrita y con prueba estática, no ejecutada en un clúster
Contexto de seguridad de ambos contenedores	Kit y plano agéntico	Usuario sin privilegios, sin escalada de privilegios, sistema de archivos raíz de solo lectura y todas las capacidades eliminadas	—
Imagen del kit	JAR del kit, especificaciones, catálogos y esquemas	Imagen mínima con JRE y usuario propio; la normativa viaja dentro de la imagen	—

Fuente: elaboración propia.

4.4.8. Agentes, herramientas y permisos

Cada agente del plano agéntico tiene una responsabilidad acotada y solo las herramientas que necesita para cumplirla, según el principio de mínimo privilegio. La tabla 25 define sus responsabilidades, entradas, salidas y límites, y la tabla 26 cruza cada agente con los cuatro servidores MCP del plano: el lector de especificaciones (spec-reader), los catálogos (catalog), el validador que expone el kit (validator) y el repositorio (repo). El diseño responde al riesgo de agencia excesiva que la guía de OWASP identifica en las aplicaciones con modelos de lenguaje (OWASP, 2024): un agente con más permisos de los necesarios convierte un error o una instrucción maliciosa en una acción con efectos.

Tabla 25. *Agentes y responsabilidades*

Agente	Responsabilidad	Entradas	Salidas	Límites
agent-orchestrator	Coordinar los agentes y el estado del flujo; exigir la revisión humana	Solicitud de incorporación de un país o proveedor	Propuestas registradas con su verificación	No decide, no aprueba y no invoca el repositorio por cuenta propia
mapping-agent	Proponer la tabla de mapeo, las plantillas y las pruebas	Especificación del proveedor, modelo canónico y catálogos	Solicitud de cambio con el mapeo y su procedencia	Sin merge ni despliegue; su propuesta pasa el kit antes de la revisión
legacy-extraction-agent	Extraer reglas de código heredado con evidencia	Código heredado, en modo de lectura	Reglas candidatas con archivo y línea	No modifica el código heredado ni el catálogo de reglas

Agente	Responsabilidad	Entradas	Salidas	Límites
dlq-triage-agent	Clasificar los fallos de la cola de errores según D1 a D6	Mensajes de la cola de errores	Clasificación y corrección en origen propuestas	No reprocesa ni descarta mensajes; esa acción corresponde al rol OPERATOR

Fuente: elaboración propia.

Tabla 26. *Permisos mínimos de cada agente sobre las herramientas*

Agente	spec-reader	catalog	validator	repo
mapping-agent	Lectura	Lectura	Ejecutar verificación	Crear rama y solicitud de cambio, sin merge
legacy-extraction-agent	Lectura	Lectura	Ejecutar verificación	Lectura del código heredado; crear rama y solicitud de cambio, sin merge
dlq-triage-agent	Lectura	Lectura	Ejecutar verificación	Sin acceso
agent-orchestrator	Sin acceso	Sin acceso	Sin acceso	Sin acceso

Fuente: elaboración propia.

Ningún agente puede aprobar ni rechazar propuestas, hacer merge, desplegar, escribir en producción ni modificar las reglas del kit o los documentos normativos. Estas prohibiciones no dependen de la instrucción del modelo, sino de controles externos: el servidor MCP solo expone herramientas de lectura y de validación, el servidor del repositorio no tiene permiso de merge, la política de red impide que el plano agéntico alcance sistemas de producción y el rol AGENT de la plataforma no está autorizado a decidir. En el prototipo ampliado (commit 0b6406c) esto último se comprueba con pruebas automáticas: un token de agente recibe 403 al intentar decidir sobre una propuesta (test_agent_and_operator_cannot_decide) y al intentar aprobar una versión en el registro de mapeos. Los cambios en reglas y normativa siguen el procedimiento del apartado 4.4.11.

4.4.9. Revisión humana (human-in-the-loop)

Ninguna propuesta de un agente produce efectos sin la decisión de una persona. La tabla 27 describe los estados de una propuesta y sus transiciones. En la prueba de concepto inicial, el registro de propuestas usa cuatro estados; en el prototipo ampliado (commit 0b6406c), el registro de versiones de mapeo añade la activación y la sustitución, de modo que aprobar y poner en uso son pasos distintos.

Tabla 27. *Estados de una propuesta y transiciones*

Estado	Significado	Quién lo produce	Transiciones permitidas
BLOCKED_BY_GATE (BLOCKED)	La propuesta no pasó el kit	SYSTEM, al registrar la verificación	Solo a rechazada; aprobarla devuelve 409
PENDING_REVIEW (CANDIDATE)	La propuesta pasó el kit y espera decisión	SYSTEM, al registrar la verificación	A aprobada o rechazada, por un REVIEWER

Estado	Significado	Quién lo produce	Transiciones permitidas
APPROVED	Una persona aceptó la propuesta	REVIEWER distinto de quien propuso	A activa (prototipo ampliado)
REJECTED	Una persona rechazó la propuesta	REVIEWER	Estado final
ACTIVE	La versión aprobada está en uso	REVIEWER	A sustituida cuando se activa otra versión
SUPERSEDED	La versión fue reemplazada por otra	SYSTEM, al activar otra versión	A activa, como reversión, si pasó el kit

Fuente: elaboración propia.

Los roles son cuatro: REVIEWER, la persona que decide; AGENT, la identidad de los agentes, que solo pueden proponer; OPERATOR, quien reprocesa o descarta mensajes de la cola de errores y opera los simuladores; y SYSTEM, los procesos automáticos que registran propuestas y verificaciones. El revisor ve, en la consola web o en la de escritorio, el mapeo propuesto, el resultado del kit con cada hallazgo, su regla, la sección de la norma y la huella del documento. Aprueba solo si la propuesta pasó el kit y si comprueba, contra la especificación, las correspondencias que ninguna regla valida, como dos códigos de formato idéntico o dos campos cuyo producto coincide; rechaza ante cualquier duda, con un comentario que queda registrado. El kit bloquea la aprobación de lo que no lo superó, decida lo que decida el revisor.

Cada decisión se registra con la identidad del revisor, la fecha, el comentario y la verificación, y esa identidad se toma del token autenticado, nunca del cuerpo de la petición ni de cabeceras editables. Una revisión automática de seguridad encontró en la prueba de concepto inicial que la API de revisión tomaba el revisor del cuerpo de la petición (H-01) y que las consolas dependían de ese patrón (H-02), y en el registro de mapeos, que quien proponía una versión podía aprobarla (H-05). Los tres hallazgos se corrigieron y hoy son controles implementados y probados: sin token, toda ruta responde 401 (test_without_token_everything_is_401); los roles AGENT y OPERATOR reciben 403 al decidir (test_agent_and_operator_cannot_decide); un revisor distinto en el cuerpo (test_impersonation_in_body_has_no_effect) o en las cabeceras (test_headers_do_not_change_identity) no cambia la identidad registrada; y el proponente no puede aprobar su propia versión (proposerCannotApprove).

Queda el sesgo de automatización, la tendencia a aceptar la salida de un sistema automático sin examinarla, que aparece en usuarios novatos y expertos y no se elimina con entrenamiento (Parasuraman y Manzey, 2010). Se mitiga con el kit, que filtra antes lo que no puede aprobarse, con la justificación obligatoria de cada decisión y con una auditoría por muestreo de las propuestas aprobadas a cargo de una segunda persona (riesgo R6 de la tabla 41). La auditoría por muestreo está implementada y probada en el prototipo (commits

462b976 y 784e3c2). Un operador o un proceso del sistema sortea, entre las versiones de mapeo ya aprobadas por una persona, las que pasan a una segunda revisión, con una tasa configurable, del 10 % por defecto, y una semilla; la tasa, la semilla, el valor sorteado, quién muestreó y cuándo quedan registrados, de modo que la selección es reproducible. La segunda revisión solo la registra un revisor distinto de quien aprobó, que confirma o disputa la aprobación con la identidad de su token: quien aprobó recibe 409 y los agentes reciben 403 en las tres operaciones. Las decisiones se registran en audit_log y en la tabla review_audit, no en authorization_log, que guarda una fila por comprobante. Lo verifican las pruebas de ReviewAuditTest (3 de 3: selectionIsReproducible, roles y sampleAndSecondReview) y SecurityPolicyTest. Una revisión de seguridad detectó una condición de carrera entre dos segundas revisiones simultáneas, corregida en el commit 784e3c2, para la que no hay una prueba concurrente específica. La cola de segunda revisión se usa a través de la API, sin pantalla propia en las consolas.

4.4.10. Modelo de amenazas y controles

El modelo de amenazas sigue el método de Shostack (2014): identifica los activos, traza las fronteras de confianza, clasifica cada amenaza con las categorías STRIDE (suplantación, S; manipulación, T; repudio, R; revelación de información, I; denegación de servicio, D; y elevación de privilegios, E) y asigna a cada una un control. Para las amenazas propias de la inteligencia artificial se toman como referencia el Top 10 de OWASP para aplicaciones con modelos de lenguaje (OWASP, 2024) y la base de conocimiento MITRE ATLAS (MITRE, s. f.), que recogen, entre otras, la inyección de instrucciones, el envenenamiento de datos, la agencia excesiva y la exfiltración mediante las herramientas de los agentes.

Los activos son siete: los documentos normativos y los catálogos, el catálogo de reglas del kit, los mapeos aprobados, el registro de propuestas y de auditoría, las credenciales y los tokens, los datos tributarios y el pipeline de integración y despliegue. Las fronteras de confianza son cinco: entre las especificaciones de los proveedores, que son entradas externas, y los agentes; entre el plano agéntico y el proveedor del modelo de lenguaje; entre los agentes y el kit o el repositorio, porque la salida de un agente se trata como una entrada no confiable; entre las personas que revisan y la API de revisión; y entre el pipeline y producción. La tabla 28 presenta cada amenaza con su control, la decisión o el gate con que se relaciona y el riesgo residual.

Tabla 28. *Modelo de amenazas y controles*

ID	Amenaza	Activo	STRIDE	Impacto	Control	Decisión o gate	Riesgo residual
A-01	Inyección de instrucciones en una especificación de proveedor que induce al agente a proponer un mapeo dañino	Mapeos	T	Alto	Salida del agente tratada como no confiable; kit obligatorio; herramientas de solo lectura; revisión humana	ADR-04 y ADR-05; gate de mapeo vigente	Medio: un error que ninguna regla ve depende de los ejemplos y del revisor
A-02	Datos envenenados: catálogos o normativa cargada alterados	Normativa y catálogos	T	Alto	Huella SHA-256 en cada hallazgo; catálogo versionado; cambios por solicitud de cambio revisada (apartado 4.4.11); gate de huellas en CI	ADR-10 y ADR-12	Bajo: el gate de huellas en CI compara cada documento con un manifiesto versionado (commit d3076b8); no ejecutado en un servidor Jenkins
A-03	Uso indebido de herramientas por un agente: merge, despliegue o escritura	Repositorio y producción	E	Alto	Mínimo privilegio; repositorio sin merge; política de red de salida; rol AGENT sin permiso de decisión	ADR-04 y ADR-11	Bajo
A-04	Exfiltración de datos tributarios hacia el proveedor del LLM o mediante herramientas	Datos tributarios	I	Alto	El LLM solo recibe la especificación y los esquemas; datos sintéticos; acuerdos de no retención; lista blanca de destinos de salida	ADR-10; riesgo R3	Medio: la lista blanca del cliente HTTP está probada; la política de red del chart 0.3.0 está escrita, no ejecutada en un clúster (commit 51d8bb7)
A-05	Suplantación del revisor: decidir en nombre de otra persona	Registro de decisiones	S	Alto	Identidad del token; 401 sin token; 403 para AGENT y OPERATOR; suplantación sin efecto (H-01 y H-02 corregidos)	ADR-11	Bajo; tokens de prueba, sin proveedor de identidad real
A-06	Aprobación de un mapeo por quien lo propone	Mapeos aprobados	E	Medio	Proponente distinto del aprobador (H-05 corregido)	ADR-11 y ADR-12	Bajo

ID	Amenaza	Activo	STRIDE	Impacto	Control	Decisión o gate	Riesgo residual
A-07	Repudio de una decisión de aprobación	Registro de auditoría	R	Medio	Registro con el revisor tomado del token, la fecha, el comentario y la verificación	ADR-04	Bajo
A-08	Documentos hostiles: entidades externas XML o descompresión desbordada	Kit e ingesta	D	Medio	DOCTYPE rechazado antes del esquema (R-E0-03); límites de tamaño y de descompresión	ADR-05	Bajo
A-09	Dependencias vulnerables o secretos en el código y en las imágenes	Pipeline e imágenes	T	Alto	Escaneo de dependencias, secretos y configuración; escaneo de imágenes	Gate de seguridad (tabla 42)	Medio; la verificación de huellas de la normativa en CI ya está implementada (commit d3076b8)
A-10	Aceptación de un mapeo erróneo por sesgo de automatización	Mapeos	T	Alto	Gate que impide aprobar lo bloqueado; auditoría por muestreo con segunda revisión por otra persona (commits 462b976 y 784e3c2)	ADR-05; riesgo R6	Medio
A-11	Consumo no acotado del LLM	Presupuesto	D	Bajo	Límites por ejecución	Riesgo R7	Bajo

Fuente: elaboración propia.

Las amenazas A-05, A-06 y A-10 tienen hoy controles implementados y probados, descritos en el apartado 4.4.9; las A-02 y A-09, el gate de huellas del apartado 4.4.11; y la A-04, la lista blanca de destinos del plano agéntico, cuya política de red en el clúster está escrita, pero no ejecutada. Los controles marcados como planificados forman parte del alcance del proyecto y se incorporan a los paquetes de fundaciones de plataforma y del gate del kit. El modelo se revisa en cada hito de control y cada vez que se incorpora un proveedor, porque cada especificación nueva es una entrada externa nueva.

4.4.11. Integridad de la normativa y de las reglas

La validez del kit depende de que la normativa, los catálogos y las reglas que usa sean exactamente los aprobados. Su integridad se protege con cinco controles, relacionados con las amenazas A-02, A-06 y A-09 de la tabla 28. Primero, cada documento normativo tiene una huella SHA-256 que el kit calcula al cargarlo en cada ejecución y que acompaña a cada hallazgo. Segundo, el catálogo de reglas está versionado, y cada hallazgo informa la versión de la regla que lo produjo. Tercero, toda modificación de reglas, catálogos o normativa se

hace mediante una solicitud de cambio que revisa y aprueba una persona distinta de quien la propone. Cuarto, el gate de integración continua compara la huella de cada documento normativo con la registrada en un manifiesto versionado y detiene el pipeline si difieren. Quinto, la normativa se almacena en modo de solo lectura en ejecución.

La tabla 29 indica el estado de cada control en el prototipo ampliado: la columna de estado corresponde al commit 0b6406c, salvo el gate de huellas, añadido en el commit d3076b8. Con ese gate, un documento normativo alterado no solo produce hallazgos con una huella distinta: detiene el pipeline.

Tabla 29. *Controles de integridad de la normativa y de las reglas*

Control	Estado en el prototipo ampliado (commit 0b6406c)	Amenazas
Huella SHA-256 por documento normativo, calculada en cada ejecución	Implementado: el kit la calcula al cargar la especificación y la incluye en cada hallazgo y en cada propuesta	A-02
Catálogo de reglas versionado	Implementado en parte: cada hallazgo informa la versión de la regla (2.1.0), definida en el código del kit	A-02
Cambios solo por solicitud de cambio revisada por otra persona	Implementado para los mapeos (el proponente no puede aprobar); planificado para reglas y normativa	A-02 y A-06
Verificación de huellas en el gate de CI	Implementado y probado (commit d3076b8): ci/gate-normative.sh recalcula el SHA-256 de 3 especificaciones, 3 catálogos y 2 esquemas y lo compara con el manifiesto ci/normativa.sha256; falla si una huella difiere, si falta un documento o si aparece uno no declarado. La prueba ci/test-gate-normative.sh acierta en 8 de 8 casos: el estado limpio pasa y el gate falla con cada una de 6 alteraciones. Ejecutado con ci/run-pipeline.sh; en el Jenkinsfile figura en «Gates previos», no ejecutado en un servidor Jenkins	A-02 y A-09
Almacenamiento de solo lectura	Implementado en parte: normativa, catálogos y esquemas viajan en la imagen del kit, que se ejecuta con sistema de archivos raíz de solo lectura; el almacenamiento inmutable se planifica en el proyecto	A-02

Fuente: elaboración propia.

4.5. Decisiones arquitectónicas

Cada decisión se documenta como ADR (Nygard, 2011) con contexto, opciones, decisión, consecuencias y el escenario de calidad que la justifica. La tabla 30 las resume; el anexo A las presenta completas.

Tabla 30. *Resumen de las decisiones arquitectónicas*

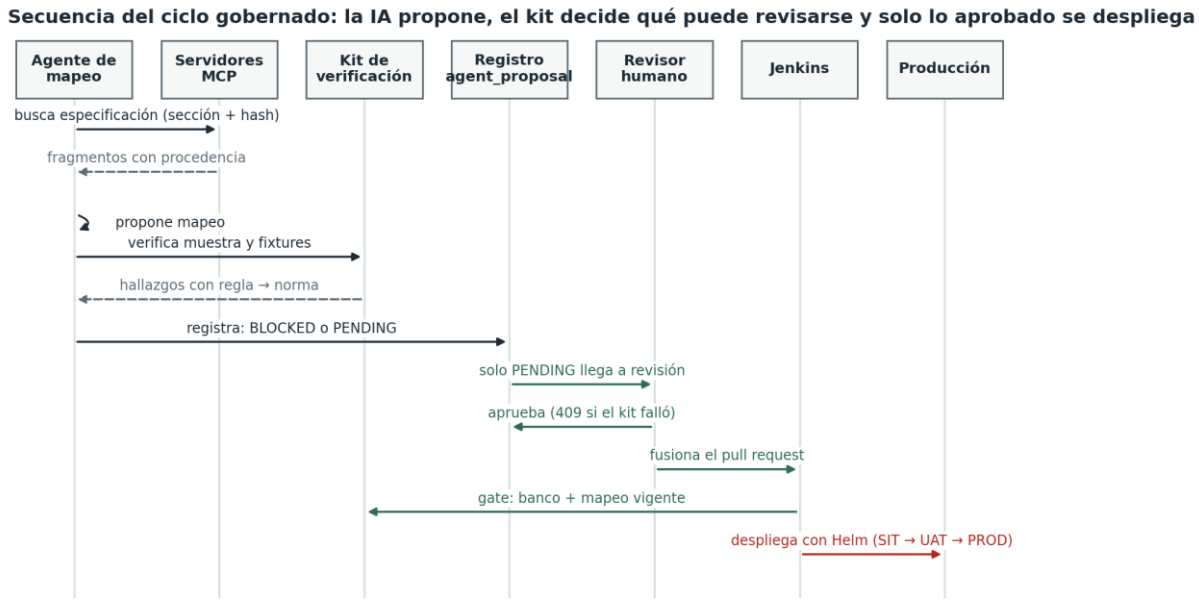
ADR	Decisión	Escenario que la justifica
01	Topics de Kafka por país y proveedor	Fiabilidad: un proveedor caído durante 2 horas no degrada la emisión de los otros países (0 mensajes perdidos, latencia del resto sin cambio).
02	Adaptador por proveedor generado con arquetipo	Mantenibilidad: incorporar un proveedor nuevo parte de un esqueleto que ya compila, pasa el kit y tiene su pipeline.
03	Python para el plano agéntico	Mantenibilidad: el plano agéntico puede cambiar de proveedor de LLM sin tocar el plano de integración.

ADR	Decisión	Escenario que la justifica
04	Los agentes solo proponen mediante solicitud de cambio	Seguridad (responsabilidad): cualquier cambio en producción se puede rastrear hasta su propuesta, su verificación y la persona que la aprobó.
05	Verificación determinista como gate en ejecución y en CI	Adecuación funcional: el 100 % de los comprobantes pasa XSD y reglas antes de salir; ninguna propuesta que falle el kit puede aprobarse.
06	Respuesta con fuente primaria y respaldo	Fiabilidad: ningún comprobante autorizado queda sin estado final tras la reconciliación diaria.
07	Archivo legal asíncrono de mejor esfuerzo con reintento	Disponibilidad: una caída del almacenamiento de 30 minutos no detiene la emisión y se recupera sin pérdida.
08	Schema Registry con compatibilidad BACKWARD	Mantenibilidad: un cambio del canónico no rompe a ningún consumidor desplegado.
09	Migraciones de base de datos con Flyway	Mantenibilidad: cualquier entorno se reconstruye desde cero con el mismo esquema.
10	RAG con procedencia para leer especificaciones	Seguridad (responsabilidad): cada regla y propuesta se rastrea hasta la sección y la versión exacta del documento que la origina.
11	Identidad del token y mínimo privilegio por rol	Seguridad (autenticidad): ninguna decisión se registra con una identidad distinta de la del token autenticado; un agente nunca aprueba.
12	Integridad de la normativa y de las reglas	Seguridad (integridad): ningún documento normativo ni regla cambia sin solicitud de cambio revisada por otra persona y sin huella verificada.

Fuente: elaboración propia.

Las ADR 04 y 05 son el núcleo de la propuesta: convierten un modelo probabilístico en una pieza aceptable para un dominio regulado. La figura 10 muestra cómo se combinan en el ciclo gobernado.

Figura 10. *Secuencia del ciclo gobernado de una propuesta de IA*



Fuente: elaboración propia. Implementada de punta a punta en la prueba de concepto, salvo el despliegue a producción.

4.6. Perfiles de trabajo y matriz de responsabilidades

Tabla 31. *Equipo del proyecto*

Rol	Responsabilidad principal	Capacidad (FTE)
Gerente del proyecto	Plan, presupuesto, riesgos, interesados y accesos	1
Arquitecto de soluciones	Arquitectura, ADR, gobierno de la IA	1
Líder técnico de integración	Diseño detallado, revisión de código y aprobación de propuestas de IA	1
Desarrolladores backend Java (3)	Microservicios, adaptadores y kit	3
Ingeniero de IA/ML	Agentes, servidores MCP y evaluación continua	1
Ingeniero DevOps/SRE	Kubernetes, Jenkins, observabilidad y seguridad	1
QA de automatización	Pruebas de contrato, integración, ejemplos validados y banco de defectos	1
Especialista tributario	Interpretación normativa y catálogos	1

Fuente: elaboración propia.

R = realiza, A = rinde cuentas, C = consultado, I = informado.

Tabla 32. *Matriz RACI*

Actividad	GP	AS	LT	DEV	IA	OPS	QA	TRIB
Arquitectura y ADR	I	A/R	C	C	C	C	I	I
Fundaciones de plataforma	I	C	A	C	I	R	I	I
Plano de integración	I	C	A	R	I	C	C	I
Kit de verificación	I	C	A	R	C	I	R	C
Plano agéntico	I	A	C	I	R	C	C	C
Aprobación de propuestas de IA	I	C	A/R	C	C	I	C	C
Incorporación de un país	A	C	R	R	R	C	R	C
Accesos y credenciales	A/R	I	C	I	I	R	I	I
Plan de pruebas y calidad	I	C	C	R	C	C	A/R	C
Gobierno y riesgos de IA	A	R	C	I	R	C	C	C
Presupuesto y seguimiento	A/R	C	C	I	I	I	I	I

Fuente: elaboración propia.

4.7. Planificación de tareas, esfuerzos y cronograma

El proyecto se planifica en seis fases con seis hitos de control (G0 a G5), un equipo de diez personas en ocho roles y un presupuesto que incluye el consumo de LLM como partida propia. Todas las cifras de este capítulo salen de un modelo reproducible desarrollado para este trabajo (anexo B): estimaciones de tres puntos, nivelación de recursos y 10.000 iteraciones de Monte Carlo. Los supuestos están declarados en el anexo B.

4.7.1. Fases e hitos de control

Tabla 33. *Fases e hitos de control del proyecto*

Fase	Contenido	Hito de salida (criterio verificable)
F1 Fundaciones	Planificación, accesos, Kubernetes, CI/CD, Kafka, observabilidad	G0: entornos SIT y UAT operativos con el pipeline en verde
F2 Kit de verificación	XSD por país, catálogos versionados, reglas D1 a D6, gate en CI	G1: kit de verificación con reglas D1 a D6 y gate de integración continua operativo con ejemplos validados
F3 Primer país	Plano de integración y primer país sin agentes (línea base)	G2: primer país autorizado en producción
F4 Plano agéntico	Servidores MCP, agentes, arnés y conjunto dorado	G3 (decisión de continuar): los agentes superan el umbral del conjunto dorado
F5 Segundo país asistido	Incorporación con agentes en condiciones comparables y banco de defectos por inyección	G4: segundo país autorizado en producción y ≥ 80 % de detección en ejecución medido con el banco de defectos
F6 Medición y cierre	Línea base, informe y transferencia	G5: informe de resultados aprobado

Fuente: elaboración propia.

El orden de las fases no es arbitrario. El kit llega antes que los agentes porque aporta valor por sí solo y reduce el riesgo de implantación, el punto débil señalado en la matriz de decisión. El primer país se incorpora sin agentes para medir la línea base; el segundo, con agentes, para medir la diferencia en condiciones comparables. **G3 es un punto de decisión real:** si los agentes no alcanzan el umbral, la fase 5 continúa sin ellos y el proyecto conserva el valor del kit y del arquetipo.

4.7.2. Estructura de desglose del trabajo y estimaciones

Cada paquete de trabajo se estima con tres valores en semanas: optimista (o), más probable (m) y pesimista (p). La duración esperada sigue la aproximación PERT (Malcolm et al., 1959):

$$t_e = (o + 4m + p) / 6$$

El esfuerzo de cada paquete es su duración esperada por la suma de las dedicaciones de los roles asignados. A estas se suma una dedicación continua de gerencia (0,5), arquitectura (0,5), liderazgo técnico (1,0) y especialista tributario (0,25) durante todo el proyecto.

Tabla 34. *Estructura de desglose del trabajo con estimaciones de tres puntos*

ID	Paquete de trabajo	Fase	o	m	p	t_e	Esfuerzo (p-s)	Depende de
1.1	Planificación y línea base	F1	2	3	4	3,0	3,0	
1.3	Accesos, credenciales y llaves	F1	4	6	12	6,7	2,0	1.1 (recurso: 2.1)
2.1	Kubernetes, Helm y CI/CD	F1	3	4	6	4,2	4,2	1.1
2.2	Kafka y Schema Registry	F1	2	3	5	3,2	4,8	2.1
2.3	Observabilidad, secretos e identidad	F1	2	3	4	3,0	1,5	2.1 (recurso: 1.3)

ID	Paquete de trabajo	Fase	o	m	p	t_e	Esfuerzo (p-s)	Depende de
3.1	Ingesta y fachada síncrona	F3	3	4	6	4,2	5,4	2.2
3.2	Enrutador y tabla de enrutamiento	F3	1	2	3	2,0	2,0	2.2
3.3	Arquetipo de adaptador	F3	2	3	5	3,2	3,2	3.2, 4.4
3.4	Respuestas y archivo legal	F3	3	4	6	4,2	5,4	3.1
3.5	Registro, DLQ y reintentos	F3	2	3	4	3,0	1,5	3.1
4.1	Validación XSD/UBL por país	F2	1	2	3	2,0	2,0	1.1
4.2	Catálogos oficiales versionados	F2	2	3	4	3,0	2,2	4.1
4.3	Reglas D1 a D6 con procedencia	F2	3	4	7	4,3	7,6	4.2
4.4	Gate del kit en CI	F2	1	2	3	2,0	2,0	4.3, 2.1
5.1	Servidores MCP	F4	2	3	5	3,2	3,2	4.4
5.2	Agente de mapeo	F4	4	6	10	6,3	6,3	5.1, 5.5
5.3	Agente de extracción de legado	F4	3	4	7	4,3	4,3	5.1 (recurso: 5.2)
5.4	Agente de triaje de DLQ	F4	2	3	5	3,2	1,6	5.1, 3.5
5.5	Arnés de evaluación y conjunto dorado	F4	2	3	5	3,2	3,2	4.3 (recurso: 3.3)
6.1	Primer país sin agentes	F3	6	8	14	8,7	27,7	3.3, 3.4, 1.3 (recurso: 5.5)
6.2	Segundo país asistido por agentes	F5	4	6	11	6,5	20,8	6.1, 5.2, 7.2
7.1	Línea base del proceso manual	F6	1	2	3	2,0	1,0	6.1
7.2	Banco de defectos por inyección	F5	1	2	3	2,0	1,6	5.5 (recurso: 6.1)
7.3	Informe de resultados y cierre	F6	2	2	4	2,3	2,3	6.2, 7.1

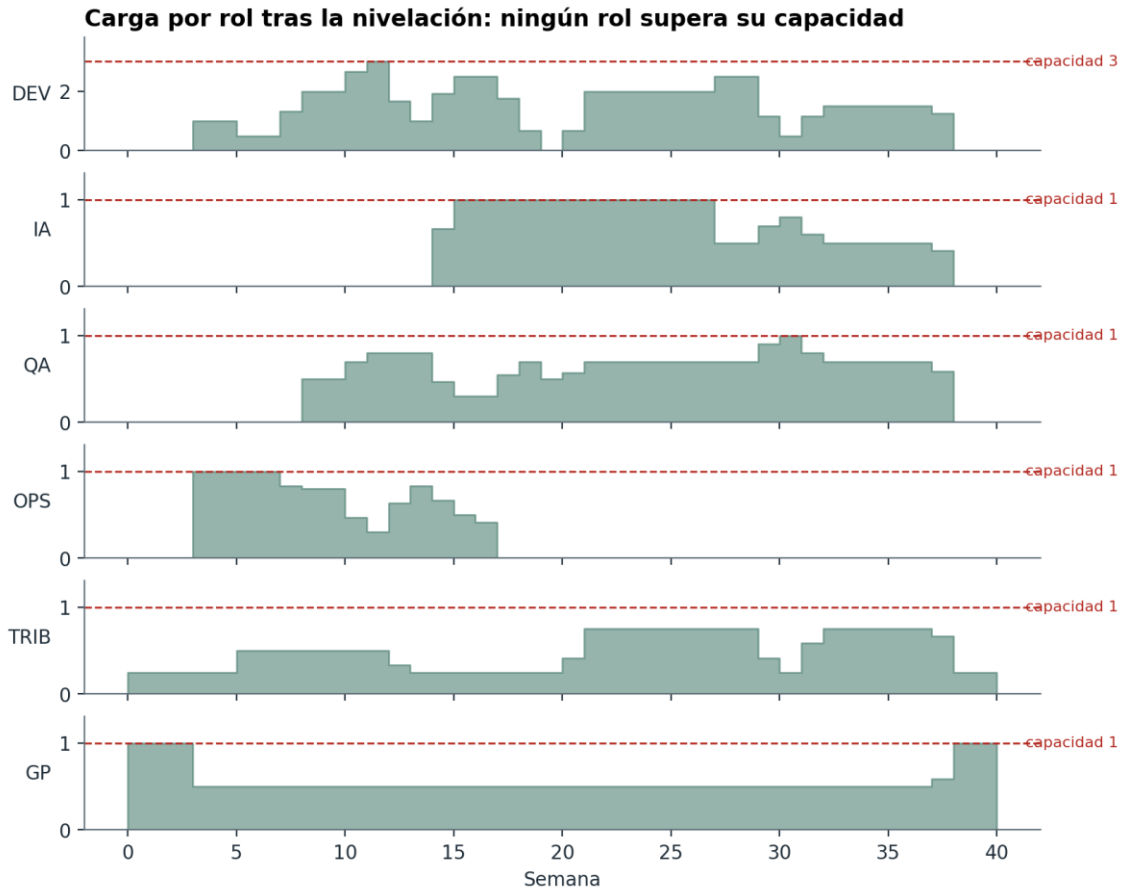
Fuente: elaboración propia. O, m, p y t_e en semanas; esfuerzo en personas-semana (p-s). Las dependencias de recurso las añade la nivelación (apartado 4.7.3).

4.7.3. Nivelación de recursos

Un cronograma calculado solo con dependencias técnicas sobreasignaba al ingeniero de IA (2,0 FTE en su pico, con capacidad de 1), a DevOps y a QA. Un plan que ignora la capacidad del equipo no es ejecutable. El modelo aplica un esquema serial de generación de cronogramas (Kolisch, 1996): programa los paquetes en orden de prioridad, según su inicio tardío, y retrasa cada uno hasta que ningún rol supere su capacidad en ninguna semana. Cada retraso por recurso se registra como una dependencia fin-inicio explícita, de modo que la simulación posterior respeta la capacidad del equipo.

La nivelación añadió 6 dependencias de recurso: 2.1 → 1.3; 3.3 → 5.5; 5.5 → 6.1; 6.1 → 7.2; 5.2 → 5.3; 1.3 → 2.3. La duración esperada pasó de unas 36 a 40,2 semanas. Ese es el costo real de un equipo de diez personas, y la figura 11 confirma que, tras nivelar, ningún rol supera su capacidad.

Figura 11. *Carga semanal por rol tras la nivelación*

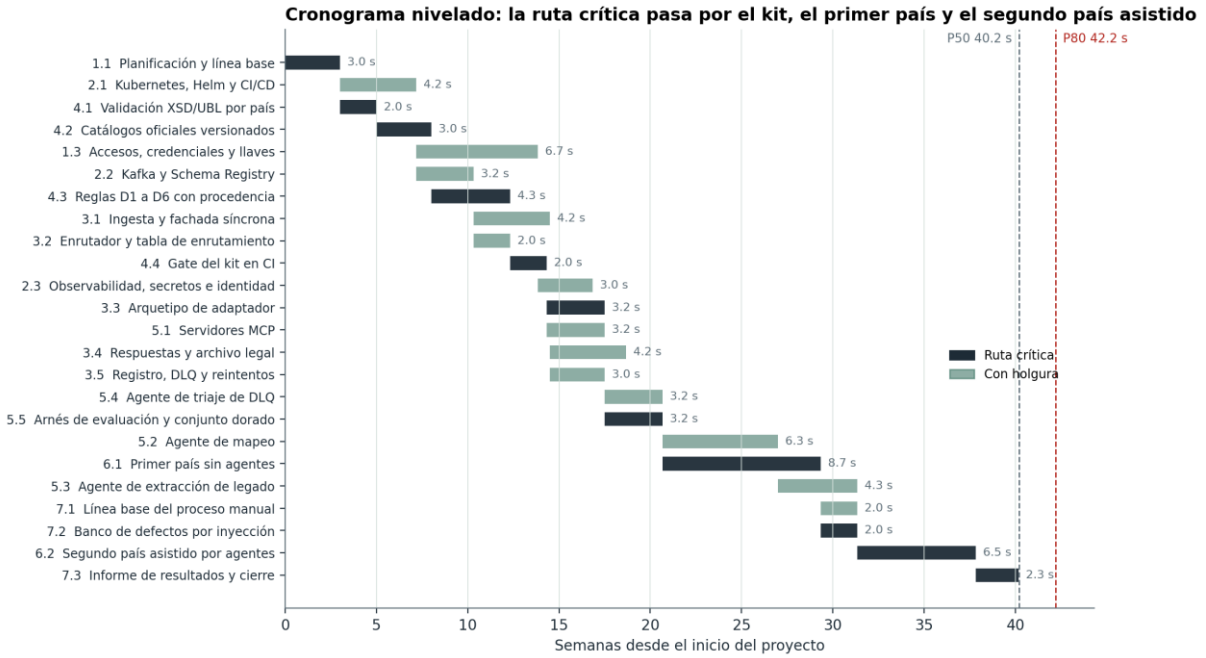


Fuente: elaboración propia. Salida del modelo de planificación. La línea discontinua es la capacidad del rol.

4.7.4. Cronograma nivelado

La ruta crítica determinística es 1.1 → 4.1 → 4.2 → 4.3 → 4.4 → 3.3 → 5.5 → 6.1 → 7.2 → 6.2 → 7.3. Pasa por el kit de verificación (4.1 a 4.4), el arquetipo (3.3), el primer país (6.1) y el segundo país asistido (6.2). Por eso el plan concentra en esos paquetes a las personas más experimentadas y los vigila semanalmente.

Figura 12. *Cronograma nivelado con ruta crítica, P50 y P80*



Fuente: elaboración propia. Salida del modelo de planificación con duraciones esperadas. En oscuro, la ruta crítica.

4.7.5. Justificación de las estimaciones de entrada

Las estimaciones de tres puntos de la tabla 34 son supuestos del autor, formulados por juicio a partir de su experiencia en proyectos de integración, y no proceden de datos históricos de la organización. Para hacer visible su forma, la tabla 35 resume, por grupo de paquetes, la razón entre el valor optimista y el más probable (o/m) y entre el pesimista y el más probable (p/m), calculadas sobre los valores de la EDT tal como están. En conjunto, o/m vale 0,67 de media y p/m, 1,60: las estimaciones son asimétricas hacia la derecha, porque el autor considera más probable un retraso que un adelanto. La asimetría mayor está en la incorporación de países (p/m de 1,79) y en los accesos y credenciales (p/m de 2,00), los paquetes que más dependen de terceros.

Tabla 35. *Razones o/m y p/m de las estimaciones por grupo de paquetes*

Grupo de paquetes	Paquetes	o/m: media (rango)	p/m: media (rango)
Fundaciones de plataforma	1.1, 1.3 y 2.1 a 2.3	0,68 (0,67–0,75)	1,57 (1,33–2,00)
Plano de integración	3.1 a 3.5	0,67 (0,50–0,75)	1,50 (1,33–1,67)
Kit de verificación	4.1 a 4.4	0,60 (0,50–0,75)	1,52 (1,33–1,75)
Plano agéntico	5.1 a 5.5	0,68 (0,67–0,75)	1,68 (1,67–1,75)
Incorporación de países	6.1 y 6.2	0,71 (0,67–0,75)	1,79 (1,75–1,83)
Medición y cierre	7.1 a 7.3	0,67 (0,50–1,00)	1,67 (1,50–2,00)
Total	24 paquetes	0,67 (0,50–1,00)	1,60 (1,33–2,00)

Fuente: elaboración propia.

Cada duración se simula con una distribución beta-PERT de parámetro $\lambda = 4$ sobre (o, m, p), cuya media es la misma t_e de la técnica PERT, y el proyecto se recalcula en 10.000 iteraciones sobre la red nivelada. Para comprobar que ese número basta, el modelo se ejecutó con tres semillas distintas (tabla 36): el P50 del plazo varía entre 40,18 y 40,21 semanas y el P80, entre 42,18 y 42,21, menos de 0,05 semanas; el costo P50 varía entre COP 1.012,4 y 1.013,0 millones y el P80, entre 1.058,4 y 1.059,9 millones, menos del 0,2 %. Con la semilla de referencia, los percentiles ya se estabilizan en torno a las 5.000 iteraciones (P50 de 40,20 y P80 de 42,16 semanas, frente a 40,21 y 42,20 con 10.000). La convergencia indica que la incertidumbre de los resultados procede de los supuestos de entrada y no del muestreo; por eso la validación de las estimaciones con un panel de expertos sigue siendo necesaria.

Tabla 36. *Convergencia de la simulación con tres semillas*

Semilla	Plazo P50 (semanas)	Plazo P80 (semanas)	Costo P50 (millones de COP)	Costo P80 (millones de COP)
2026 (referencia)	40,21	42,20	1.012,95	1.059,22
7	40,18	42,18	1.012,43	1.058,44
12345	40,19	42,21	1.012,50	1.059,87

Fuente: elaboración propia.

4.8. Presupuesto

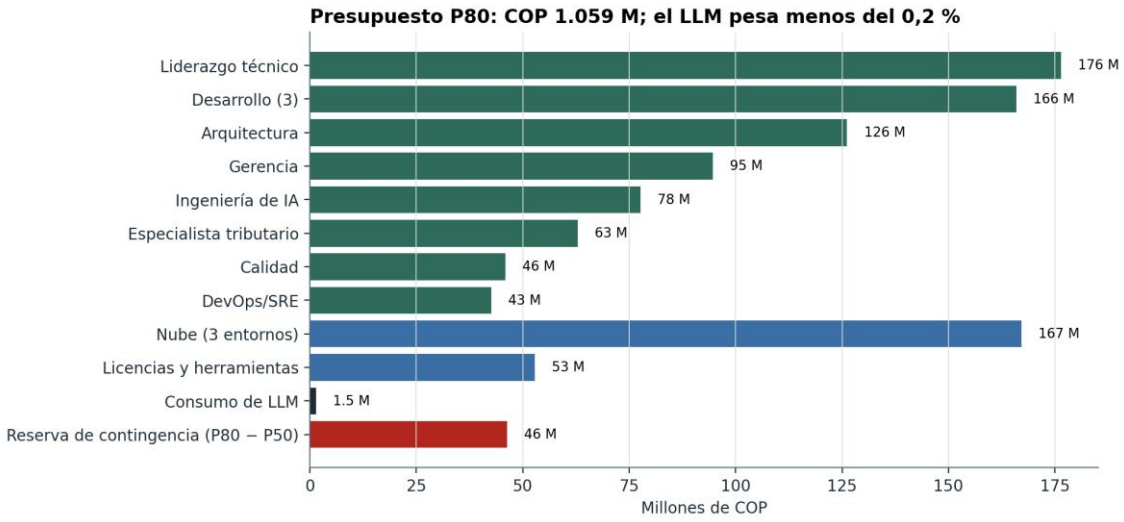
El presupuesto tiene cuatro partidas: personas (dedicación por rol × semanas × costo empresa), nube, licencias y herramientas, y consumo de LLM. La tabla 37 muestra el presupuesto con duraciones esperadas; la reserva de contingencia lo lleva al P80.

Tabla 37. *Presupuesto por partida (millones de COP)*

Partida	Millones de COP	Participación
Líder técnico de integración	176,4	17,4 %
Desarrolladores backend Java (3)	165,9	16,4 %
Arquitecto de soluciones	126,2	12,4 %
Gerente del proyecto	94,6	9,3 %
Ingeniero de IA/ML	77,7	7,7 %
Especialista tributario	62,9	6,2 %
QA de automatización	45,9	4,5 %
Ingeniero DevOps/SRE	42,6	4,2 %
Subtotal personas	792,3	78,2 %
Nube (tres entornos)	167,1	16,5 %
Licencias y herramientas	52,9	5,2 %
Consumo de LLM	1,5	0,15 %
Total con duraciones esperadas	**1.013,7**	**100 %**
Reserva de contingencia (P80 − P50)	46,3	—
Presupuesto P80 (simulación)	**1.059,2**	—

Fuente: elaboración propia. Costos de personal como costo empresa mensual supuesto (anexo B); TRM supuesta de 4.000 COP/USD. La reserva de contingencia es la diferencia entre el P80 y el P50 de la simulación, por lo que no coincide exactamente con la diferencia entre el presupuesto P80 y el total con duraciones esperadas.

Figura 13. *Presupuesto por partida*



Fuente: elaboración propia. Salida del modelo de planificación.

El consumo de LLM se estima por agente con la expresión siguiente, donde N es el número de ejecuciones, T los tokens de entrada y salida y p el precio por token:

$$C_LLM = \Sigma_a N_a \cdot (T_a,entrada \cdot p_entrada + T_a,salida \cdot p_salida)$$

Tabla 38. *Consumo estimado de LLM por agente*

Agente	Ejecuciones	Tokens de entrada	Tokens de salida	USD	COP
Mapeo	60	60.000	8.000	18,00	72.000

Agente	Ejecuciones	Tokens de entrada	Tokens de salida	USD	COP
Legado	250	30.000	4.000	37,50	150.000
Triaje	6.000	6.000	1.000	198,00	792.000
Arnés	400	60.000	8.000	120,00	480.000
Total				**373,50**	**1.494.000**

Fuente: elaboración propia.

El hallazgo es contraintuitivo: el LLM representa menos del 0,2 % del presupuesto. El costo de la IA no está en los tokens, sino en las personas que diseñan la verificación, revisan las propuestas y mantienen el conjunto dorado. Por eso el plan invierte en el kit y en la revisión, no en ahorrar tokens.

4.8.1. Referentes salariales y sensibilidad del presupuesto

Las tarifas por rol son supuestos del autor, calibrados contra referentes públicos del mercado colombiano. El costo empresa mensual de cada rol resulta de multiplicar un salario base por un factor de 1,5, que cubre prestaciones sociales, seguridad social y aportes parafiscales; las fuentes del mercado sitúan ese costo adicional entre el 38 % y el 58 % sobre el salario (Buk, 2026). La tabla 39 compara el salario base implícito de cada rol con los rangos publicados. Cinco roles tienen un referente público y quedan dentro de su rango; los de IA, QA y especialista tributario no tienen una cifra pública verificada, por lo que se declaran como supuestos.

Tabla 39. *Salario base implícito por rol y referentes públicos del mercado colombiano*

Rol	Salario base implícito (COP al mes)	Referente público	Estado
Gerente del proyecto	12,0 M	Gerente de proyectos de TI: 7 a 12 M (Colombia Move, 2026)	Dentro del rango, en el extremo superior
Arquitecto de soluciones	16,0 M	Desarrollador senior o arquitecto: 8 a 15 M o más (Colombia Move, 2026); perfiles senior de software hasta 23 M (Michael Page, 2026, como se citó en Portafolio, 2026, y en Valora Analitik, 2026)	Dentro del rango de perfiles senior
Líder técnico de integración	12,7 M	Mismo referente de desarrollador senior o arquitecto	Dentro del rango
Desarrolladores backend Java (3)	8,7 M	Desarrollador senior: 8 a 15 M o más (Colombia Move, 2026)	Dentro del rango, en el extremo inferior
Ingeniero DevOps/SRE	12,0 M	Ingenieros DevOps: 12 a 25 M (Robert Half Technology, 2024, como se citó en La FM, 2024)	Dentro del rango, en el extremo inferior
Ingeniero de IA/ML	12,0 M	Perfiles de aprendizaje automático entre los mejor pagados, sin cifra pública verificable (Michael Page, 2026, como se citó en Portafolio, 2026)	Supuesto del autor
QA de automatización	6,7 M	Sin referente público verificado	Supuesto del autor
Especialista tributario	9,3 M	Sin referente público verificado	Supuesto del autor

Fuente: elaboración propia. M = millones de pesos colombianos. El salario base implícito es el costo empresa mensual dividido por el factor de 1,5.

Para que la conclusión no dependa de un único conjunto de tarifas, el modelo se ejecutó también con tarifas un 20 % menores y un 25 % mayores. El calendario no cambia, porque no depende de las tarifas; solo se mueve el costo, y el proyecto sigue siendo viable en los tres escenarios. El punto de equilibrio, expresado en esfuerzo, tampoco depende de ellas.

Tabla 40. *Sensibilidad del costo del proyecto a las tarifas por rol*

Escenario	Tarifas por rol	Costo P50 (millones de COP)	Costo P80 (millones de COP)
Bajo	20 % menos	855	894
Base	**Calibradas (tabla 39)**	**1.013**	**1.059**
Alto	25 % más	1.211	1.266

Fuente: elaboración propia. Salida del modelo de planificación con 10.000 iteraciones por escenario.

4.9. Análisis de riesgos

Los riesgos de IA se asignan a las funciones del marco del NIST: gobernar (G), mapear (Ma), medir (Me) y gestionar (Ge) (NIST, 2023). Probabilidad (P) e impacto (I): A = alto, M = medio, B = bajo.

Tabla 41. *Registro de riesgos*

ID	Riesgo	P	I	Respuesta	NIST	Dueño
R1	El agente propone mapeos incorrectos pero plausibles	A	A	Kit obligatorio, revisión humana y banco de defectos (comprobado en la prueba de concepto)	Me, Ge	LT
R2	Deriva del modelo o cambio de versión del proveedor de LLM	M	A	Versión fijada y evaluación continua contra el conjunto dorado	Me	IA
R3	Exposición de datos tributarios en las consultas al LLM	M	A	Solo especificaciones y esquemas; datos sintéticos; acuerdos de no retención	G, Ge	AS
R4	Cambio normativo durante el proyecto	A	M	Catálogos versionados y reglas con procedencia y huella digital	Ma	TRIB
R5	Retraso en credenciales, llaves y accesos	A	A	Gestión desde la fase 1 y trabajo en paralelo	—	GP
R6	Confianza excesiva de los revisores en la IA	M	A	Gate que impide aprobar lo bloqueado; auditoría por muestreo	G, Ge	QA
R7	Consumo de LLM por encima de lo presupuestado	B	B	Límites por ejecución; el LLM pesa menos del 0,2 %	Ge	IA
R8	Dependencia de un único experto tributario	M	M	Reglas como código con procedencia; transferencia de conocimiento	G	GP
R9	QA como cuello de botella del calendario	A	M	Segunda persona de QA en F2 a F4 (escenario del apartado 5.6)	—	GP

Fuente: elaboración propia.

4.10. Plan de calidad, integración y despliegue continuos y gobierno del proyecto

4.10.1. Plan de calidad y de pruebas

La calidad se asegura con controles automáticos en el pipeline: análisis estático con quality gate, escaneo de dependencias, imágenes y secretos, pruebas de contrato, reglas de arquitectura y, como control distintivo, el kit de verificación con el banco de defectos. Las pruebas se organizan en cinco niveles:

1. Unitarias, incluidas pruebas basadas en propiedades para las invariantes de D2.
2. De integración con Kafka, base de datos y SFTP reales en contenedores.
3. De contrato entre servicios y con proveedores simulados.
4. De carga para el escenario de 200.000 comprobantes en un día pico.
5. De evaluación de agentes con el conjunto dorado y el banco de defectos por inyección.

4.10.2. Integración y despliegue continuos con Jenkins

Jenkins se instala como código: una imagen con las herramientas del pipeline, los plugins declarados y la configuración JCasC con usuarios, permisos, credenciales y el job del pipeline. Cada etapa del Jenkinsfile invoca el mismo script que se ejecuta en local, de modo que lo que pasa en el portátil del desarrollador pasa igual en el servidor.

Tabla 42. *Etapas del pipeline de Jenkins*

Etapa	Qué hace	Detiene el pipeline si…
Compilar	Compila el kit con -Xlint:all -Werror y la consola JavaFX	Hay errores o advertencias de compilación
Gate · huellas de la normativa	Recalcula el SHA-256 de especificaciones, catálogos y esquemas y lo compara con el manifiesto versionado (commit d3076b8)	Una huella difiere, falta un documento o aparece uno no declarado
Pruebas (en paralelo)	JUnit 5 del kit y pytest con Hypothesis del plano agéntico; publica reportes JUnit	Falla una prueba
Gate · banco de defectos	Ejecuta los 22 mutantes sobre 300 comprobantes y 12 ejemplos validados	CI no detecta el 100 %, ejecución < 80 % o hay falsos positivos
Gate · mapeo vigente	Verifica el mapeo aprobado en modo CI sobre ejemplos validados y una muestra	Algún documento tiene hallazgos
Seguridad	Trivy sobre el repositorio: vulnerabilidades, secretos y configuración	Hallazgos altos o críticos
Empaquetar	JAR del kit con su huella digital y prueba de humo del servicio HTTP	/verify no responde 200 y 422 como se espera
Imágenes	Construye, escanea y publica las imágenes	Vulnerabilidades críticas
Desplegar SIT	Helm upgrade --install y prueba de humo en el clúster	El despliegue o la prueba fallan
Aprobación y UAT/PROD	input con aprobadores definidos; Helm con --atomic	Nadie aprueba o el despliegue falla (rollback automático)

Fuente: elaboración propia.

El chart de Helm despliega el kit con autoescalado horizontal y el plano agéntico con un volumen persistente para el registro de propuestas. La salida del plano agéntico se controla en dos niveles (commit 51d8bb7). En el cliente HTTP, una lista blanca solo permite conectar con el kit, la plataforma y, si hay un modelo configurado, su proveedor, y revisa cada petición y cada redirección antes de conectar; la prueba test_egress.py pasa 13 de 13 casos, entre ellos el rechazo de otro host, de otro puerto, de otro esquema, de localhost, de los metadatos de la nube y de file://, y permitió encontrar y corregir un defecto de compatibilidad con el SDK del modelo. En el clúster, la versión 0.3.0 del chart elimina la salida abierta por el puerto 443 y la limita al kit, la plataforma, un proxy de salida y el DNS; esta política está escrita y comprobada con una prueba estática sobre la plantilla (test_helm_egress.py, 2 de 2), pero no se ha ejecutado en un clúster, ni con helm lint ni con helm template, de modo que no se ha comprobado que el plugin de red la aplique. Los contenedores se ejecutan sin privilegios y con sistema de archivos de solo lectura.

Los gates de seguridad del pipeline son de dos clases. Implementados en el Jenkinsfile del prototipo: la etapa de seguridad ejecuta Trivy sobre el repositorio con los analizadores de vulnerabilidades de dependencias, de secretos y de configuración, y detiene el pipeline ante hallazgos altos o críticos; y la etapa de imágenes escanea la imagen del kit y la detiene ante vulnerabilidades críticas. Implementado y probado (commit d3076b8): el gate de huellas de la normativa descrito en el apartado 4.4.11, que se ejecutó con el script de etapas del pipeline y figura en el Jenkinsfile en «Gates previos», aunque no se ha ejecutado en un servidor Jenkins.

4.10.3. Gobierno del proyecto

El seguimiento semanal revisa el avance sobre la ruta crítica y el índice de criticidad; el quincenal, los riesgos y el consumo de la reserva. Cada hito G0 a G5 es una revisión formal con criterio verificable, y G3 incluye la decisión explícita de continuar o no con los agentes. Las métricas del capítulo 3 se publican en un tablero desde la fase F2.

4.11. Evaluación de la solución

La evaluación de la solución responde a cuatro preguntas: qué se mide, cuándo, quién lo mide y qué resultado se considera un éxito. Los indicadores derivan de las métricas del enfoque objetivo, pregunta y métrica presentado en la tabla 11, y se miden en las fases del proyecto indicadas en la tabla 43. La evaluación no espera al final: el kit se evalúa en cada cambio de mapeo desde la fase F2, el agente de mapeo se mide con el arnés de evaluación en la fase F4 y el esfuerzo por país se contrasta con la línea base manual en la fase F5.

Tabla 43. *Plan de evaluación de la solución*

Indicador	Cuándo se mide	Quién	Criterio de éxito
Tasa de detección en ejecución	Cada cambio de mapeo, en el gate de integración continua	Equipo de integración y QA	Al menos 80 % de los defectos inyectados, informado con su intervalo de confianza del 95 %
Falsos positivos del kit	Cada ejecución del banco y del conjunto dorado	QA	Cero sobre salidas correctas, con la cota superior de la regla de tres
Comprobantes verificados antes del envío	Continuo, en producción	Equipo de integración	100 % de los comprobantes pasan el kit antes de salir al proveedor
Esfuerzo por país o proveedor nuevo	Al cerrar cada incorporación, desde la fase F5	Gerente del proyecto	Ahorro frente a la línea base manual de 27,7 personas-semana, medido con el protocolo del apartado 3.9.3
Exactitud del agente de mapeo	Con el arnés de evaluación, en la fase F4	Ingeniero de IA	Medida y comparada con la línea base heurística
Plazo y costo del proyecto	En cada hito de control, con el seguimiento sobre la ruta crítica	Gerente del proyecto	Plazo dentro del P80 y costo dentro del presupuesto con reserva
Riesgos materializados	Revisión quincenal del registro de riesgos	Gerente del proyecto y líder técnico	Ningún riesgo de impacto alto sin plan de contingencia activo

Fuente: elaboración propia.

Los factores de éxito son tres. El primero es que el kit supere la meta de detección sin falsos positivos, porque de ello depende que la revisión humana pueda confiar en lo que recibe. El segundo es que el esfuerzo asistido por agentes sea efectivamente menor que el manual, lo que la planificación formula como hipótesis y la fase F5 comprueba con la primera incorporación real. El tercero es que el proyecto se mantenga dentro del plazo y del presupuesto del percentil 80, con la reserva de contingencia como margen.

5. Resultados obtenidos

Este capítulo presenta los resultados del trabajo organizados por la evidencia que los respalda. Primero, la prueba de concepto y su banco de defectos, que validan el diseño del kit de verificación (apartados 5.1 a 5.5). Después, los resultados de la planificación, con la simulación de Monte Carlo y el punto de equilibrio (5.6), y las amenazas a la validez de lo medido (5.7).

Como el trabajo combina un prototipo con un plan, no todos sus resultados tienen la misma naturaleza. La tabla 44 distingue los resultados medidos en el prototipo, los estimados por el modelo de planificación o por juicio experto y las hipótesis que el proyecto debe comprobar, e indica el apartado donde se presenta cada uno.

Tabla 44. *Naturaleza de los resultados*

Resultado	Tipo	Apartado
Frecuencia relativa de los tipos de defecto (análisis de Pareto)	Estimado por juicio experto	1.1.1
Pruebas automatizadas y ejecución del pipeline	Medido en el prototipo	5.1 y 5.5
Detección por mecanismo, falsos positivos e intervalos de confianza en el banco	Medido en el prototipo	5.2
Exactitud del agente heurístico y bloqueo por el kit	Medido en el prototipo	5.3
Madurez tecnológica (TRL)	Estimado por valoración del autor	5.5.1
Plazo y costo en P50 y P80	Estimado por el modelo	5.6.1
Escenario con dos personas de QA e índice de criticidad	Estimado por el modelo	5.6.1
Esfuerzo asistido de 20,8 personas-semana y ahorro de 6,9 por país	Hipótesis	5.6.2
Punto de equilibrio n*	Estimado por el modelo bajo la hipótesis de esfuerzo	5.6.2

Fuente: elaboración propia.

5.1. Prueba de concepto: alcance, implementación y pruebas

Para no usar información de ninguna organización, se construyó una especificación sintética, el **Perfil Alfa v2.1**, que imita la estructura de los anexos técnicos latinoamericanos basados en UBL 2.1: secciones numeradas, catálogos versionados, reglas de validación y elementos retirados por versión. Se añadió un segundo formato, el del **proveedor Beta**, de texto por segmentos con separador reservado, para cubrir el defecto D6. El generador produce comprobantes sintéticos reproducibles con semilla fija, y 12 ejemplos validados cubren cada tipo de documento, cada tipo de identificación, cada tributo y datos adversarios.

5.1.1. Implementación

Tabla 45. *Componentes implementados en la prueba de concepto*

Componente	Tecnología	Contenido	Líneas
verification-kit	Java 21, solo JDK	XSD, reglas D1 a D6 con procedencia, ejemplos validados, CLI y servidor HTTP	725
agentic-plane	Python 3.12, Pydantic, lxml, SDK de MCP 2.x, FastAPI	Canónico, transformadores, agente de mapeo, orquestador, registro, servidor MCP, API, banco	997
review-console-web	HTML, CSS puro y JavaScript sin dependencias	Consola de revisión sin frameworks	321
review-console-javafx	JavaFX con hoja CSS	Consola de escritorio contra la misma API	349
Pruebas	JUnit 5, pytest, Hypothesis	32 pruebas automatizadas	223
ci y deploy	Bash, Jenkins (JCasC), Docker, Helm	Pipeline, imágenes, chart y políticas de red	408
planning	Python, NumPy	PERT, nivelación de recursos, Monte Carlo y escenarios	171

Fuente: elaboración propia. Líneas de código no vacías.

El kit no tiene dependencias externas, por decisión deliberada: el mismo JAR corre en un adaptador, en Jenkins y detrás del servidor MCP. Rechaza DOCTYPE para prevenir XXE, limita el tamaño de los documentos y devuelve código de salida 2 cuando hay hallazgos, lo que lo convierte en un gate directo para cualquier pipeline.

5.1.2. Pruebas automatizadas

Tabla 46. *Suites de pruebas y resultados*

Suite	Herramienta	Pruebas	Qué verifican	Resultado
Kit de verificación	JUnit 5	17	Cada regla D1 a D6, procedencia, cero falsos positivos, XXE, segmentos y el caso que solo detecta un ejemplo validado	17/17
Modelo canónico	pytest + Hypothesis	3	Invariantes en comprobantes generados al azar y cobertura de los ejemplos validados	3/3
Integración Python–Java	pytest	3	Mapeo dorado sin falsos positivos y banco completo con CI al 100 %	3/3
Agente de mapeo	pytest	3	Exactitud mínima, bloqueo por el kit y filtrado de alucinaciones del LLM	3/3
API de revisión	pytest + TestClient	3	409 al aprobar lo bloqueado, decisión única, validación del revisor	3/3
Servidor MCP	pytest	3	Herramientas expuestas, procedencia en la búsqueda y validador	3/3

Fuente: elaboración propia.

5.2. Banco de defectos por inyección

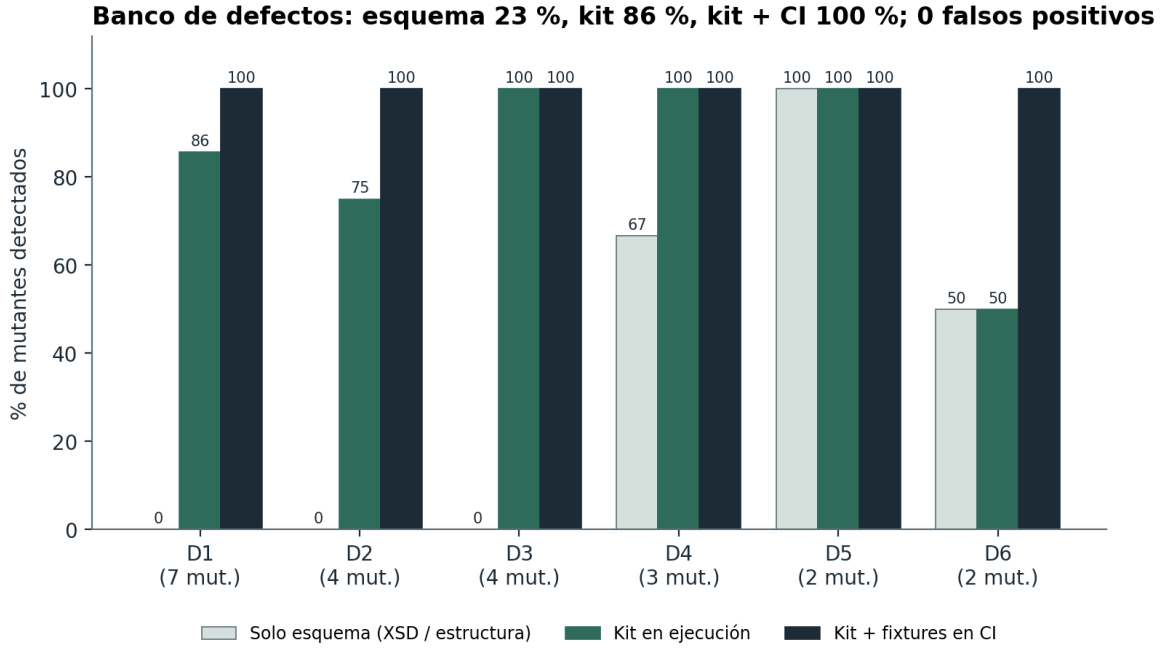
El banco aplica 22 mutantes, de los seis tipos, sobre 300 comprobantes sintéticos y 12 ejemplos validados. En total son 14.688 verificaciones, que el kit resolvió en 14,4 segundos en un solo proceso, unas 1.022 por segundo incluido el arranque de la JVM. Un mutante se considera detectado si el mecanismo rechaza al menos un documento afectado.

Tabla 47. *Mutantes detectados por tipo y mecanismo*

Tipo	Mutantes	Solo esquema	Kit en ejecución	Kit + CI
D1	7	0 %	86 %	100 %
D2	4	0 %	75 %	100 %
D3	4	0 %	100 %	100 %
D4	3	67 %	100 %	100 %
D5	2	100 %	100 %	100 %
D6	2	50 %	50 %	100 %
Total	**22**	**23 %**	**86 %**	**100 %**

Fuente: elaboración propia. Falsos positivos sobre 1.224 verificaciones de salidas correctas: 0 en los tres mecanismos.

Figura 14. *Detección de defectos por tipo y mecanismo*



Fuente: elaboración propia. Resultados reales de la prueba de concepto (anexo C).

Los resultados confirman la hipótesis de diseño con datos. La validación de esquema, la práctica habitual, detecta solo el 23 % de los mutantes: falla con todos los D1, D2 y D3, porque en UBL los códigos y literales son cadenas libres. Las reglas con procedencia elevan la detección al 86 %, sin un solo falso positivo. Los tres mutantes que escapan en ejecución explican por qué el gate debe ser doble:

- **D1-e.** Pasaporte codificado como documento de extranjero (A3); formatos idénticos. Los formatos de ambos documentos son idénticos, así que ninguna regla puede distinguirlos. Solo lo detecta la comparación con ejemplos validados por personas, que incluyen datos adversarios.
- **D2-a.** Cantidad y precio unitario intercambiados (producto idéntico). El producto cantidad × precio no cambia, así que todas las invariantes aritméticas se cumplen. Solo

lo detecta la comparación con ejemplos validados por personas, que incluyen datos adversarios.

- **D6-b.** Nombre del adquirente sin sanear el separador reservado. Ningún comprobante de la muestra contenía el separador en el nombre: el defecto existe pero los datos normales no lo activan. Solo lo detecta la comparación con ejemplos validados por personas, que incluyen datos adversarios.

La conclusión práctica es que ni el esquema ni las reglas bastan por separado: el kit en ejecución protege la producción, y los ejemplos validados en integración continua cierran los huecos que ninguna regla puede ver. Es exactamente la ADR-05.

Para no sobreinterpretar estas tasas, la tabla 48 las acompaña de su intervalo de confianza del 95 % por el método de Wilson, con el mutante como unidad (n = 22), y acota los falsos positivos con la regla de tres, con la verificación de una salida correcta como unidad. La detección en ejecución, del 86,4 %, tiene un intervalo del 66,7 % al 95,3 %: su límite inferior queda por debajo del 80 %, de modo que, con este banco de 22 mutantes, la meta se cumple como estimación puntual, pero no como garantía estadística; el banco ampliado del apartado 5.2.1 estrecha el intervalo. La detección en integración continua, del 100 %, tiene un límite inferior del 85,1 %. De los 17 mutantes que el esquema no detecta, el kit en ejecución detecta 14 y los ejemplos validados, los tres restantes. La ausencia de falsos positivos en 600 salidas correctas permite afirmar, con una confianza del 95 %, que la tasa de falsos positivos del kit en ejecución no supera el 0,5 %; en integración continua, con 24 ejemplos correctos, la cota es del 12,5 %.

Tabla 48. *Intervalos de confianza de la detección y de los falsos positivos*

Mecanismo	Detección (TP/n)	IC 95 % de la detección (Wilson)	Falsos positivos (FP/n)	Cota superior del 95 % (regla de tres)
Solo esquema	5/22 (22,7 %)	10,1 % a 43,4 %	0/600	0,5 %
Kit en ejecución	19/22 (86,4 %)	66,7 % a 95,3 %	0/600	0,5 %
Kit + CI	22/22 (100 %)	85,1 % a 100 %	0/24	12,5 %

Fuente: elaboración propia.

5.2.1. Banco ampliado ejecutado por el sistema real

Para estimar la detección con más precisión, el banco se amplió y se ejecutó por el sistema real, es decir, por la plataforma local, el registro de mapeos y la versión 0.2.0 del kit, en un perfil local sin Docker (commit 0a52161). Los mutantes se generan con los operadores existentes, con la semilla 2026 y un máximo de 30 por tipo, y no se ajustó ningún operador ni ninguna regla. Cada mutante se declara defectuoso, y los equivalentes se excluyen con un criterio escrito: el mapeo mutado es idéntico a la línea base; el cambio solo afecta a claves o

fuentes que el dominio del país de destino nunca produce; o el literal mutado coincide con el valor que la línea base ya produce en todo el dominio. De un espacio de 195 mutantes se excluyeron 21 equivalentes y, con el máximo de 30 por tipo, quedaron 129: solo D1 y D4 admiten 30 mutantes no equivalentes con los operadores existentes, y D2, D3, D5 y D6 se quedan en 27, 29, 10 y 3. La tabla 49 presenta el resultado por tipo.

Tabla 49. *Banco ampliado: detección por tipo con su intervalo de confianza del 95 % (Wilson)*

Tipo	Espacio	Equivalentes excluidos	Mutantes	Detectados en ejecución (IC 95 %)	Detectados con ejemplos validados (IC 95 %)
D1	85	15	30	24 (80,0 %; 62,7 % a 90,5 %)	30 (100 %; 88,7 % a 100 %)
D2	27	0	27	26 (96,3 %; 81,7 % a 99,3 %)	27 (100 %; 87,5 % a 100 %)
D3	35	6	29	29 (100 %; 88,3 % a 100 %)	29 (100 %; 88,3 % a 100 %)
D4	35	0	30	30 (100 %; 88,7 % a 100 %)	30 (100 %; 88,7 % a 100 %)
D5	10	0	10	10 (100 %; 72,3 % a 100 %)	10 (100 %; 72,3 % a 100 %)
D6	3	0	3	2 (66,7 %; 20,8 % a 93,9 %)	3 (100 %; 43,9 % a 100 %)
Total	195	21	129	121 (93,8 %; 88,2 % a 96,8 %)	129 (100 %; 97,1 % a 100 %)

Fuente: elaboración propia a partir de la ejecución del 20 de octubre de 2026 (commit 0a52161), con datos sintéticos del generador.

En ejecución, el kit detectó 121 de los 129 mutantes, un 93,8 %, con un intervalo de confianza del 95 % del 88,2 % al 96,8 %; con los ejemplos validados, los 129, un 100 % (97,1 % a 100 %). Escapan en ejecución 8 mutantes: seis de D1 (D1-x03, D1-x07, D1-x14, D1-x17, D1-x19 y D1-x26), cinco de ellos con códigos válidos pero intercambiados en el formato del proveedor Beta, uno de D2 (D2-x01) y uno de D6 (D6-x02). Los ejemplos validados los bloquean todos, y ningún mutante bloqueado se puede aprobar (HTTP 409). En la misma corrida, el banco original de 22 repitió su resultado: 19 de 22 en ejecución y 22 de 22 con ejemplos validados. Los resultados son idénticos, mutante por mutante, a los de una ejecución anterior con la misma huella del conjunto.

Sobre 11.402 comprobantes correctos procesados por la plataforma en los escenarios del sistema real, entre ellos un lote de 10.000, el kit no produjo ningún falso positivo, lo que acota la tasa de falsos positivos en el 0,026 % con una confianza del 95 % (regla de tres). Los únicos bloqueos del kit registrados fueron los 4 defectos intencionales de un escenario de prueba, y 400 verificaciones directas del kit no tuvieron ningún rechazo. En esa corrida, la emisión síncrona tuvo un tiempo p50 de 40,7 ms y un p95 de 57,4 ms, frente al umbral de 3 s, y el lote de 10.000 se procesó en 31,2 s sin pérdidas; estos tiempos varían entre corridas.

Con el banco ampliado, el límite inferior del intervalo de la detección en ejecución, del 88,2 %, queda por encima de la meta del 80 %, de modo que la meta se cumple también con una confianza del 95 %. La estimación sigue limitada a datos sintéticos, a mutantes generados con los operadores del mismo autor y a tipos con pocos mutantes posibles: D6 solo admite 3 no equivalentes, de los que el kit detecta 2 en ejecución, con un intervalo del 20,8 % al 93,9 %. Al cierre de esta ejecución, las suites del prototipo suman 141 pruebas Java sin fallos ni errores, con 1 omitida por falta de Docker, y 57 de 57 pruebas Python.

5.3. Agente de mapeo y ciclo gobernado

El agente de mapeo tiene dos motores detrás del mismo contrato. El motor heurístico, por similitud léxica con un glosario mínimo, sirve de línea base determinista. El motor LLM usa el SDK de Anthropic con salida JSON y filtra los campos y fuentes inexistentes, lo que mitiga las alucinaciones estructurales; se probó con un cliente simulado, porque el entorno de desarrollo del prototipo no tenía credenciales de API. La medición con un LLM real es una tarea del arnés de evaluación en la fase F4.

El ciclo gobernado se ejecutó de punta a punta con el motor heurístico:

1. El agente propuso los 24 campos del perfil y acertó 23, una exactitud del 95,8 % frente al mapeo dorado.
2. Su único error fue mapear el nombre del esquema tributario al número de línea. El kit bloqueó la propuesta en los 62 documentos verificados con la regla R-D3-02, que cita la §5.1 de la especificación y su huella digital.
3. La propuesta quedó en estado BLOCKED_BY_GATE y la API impide aprobarla (HTTP 409).
4. La propuesta corregida pasó el kit en los 62 documentos y quedó pendiente de decisión humana.

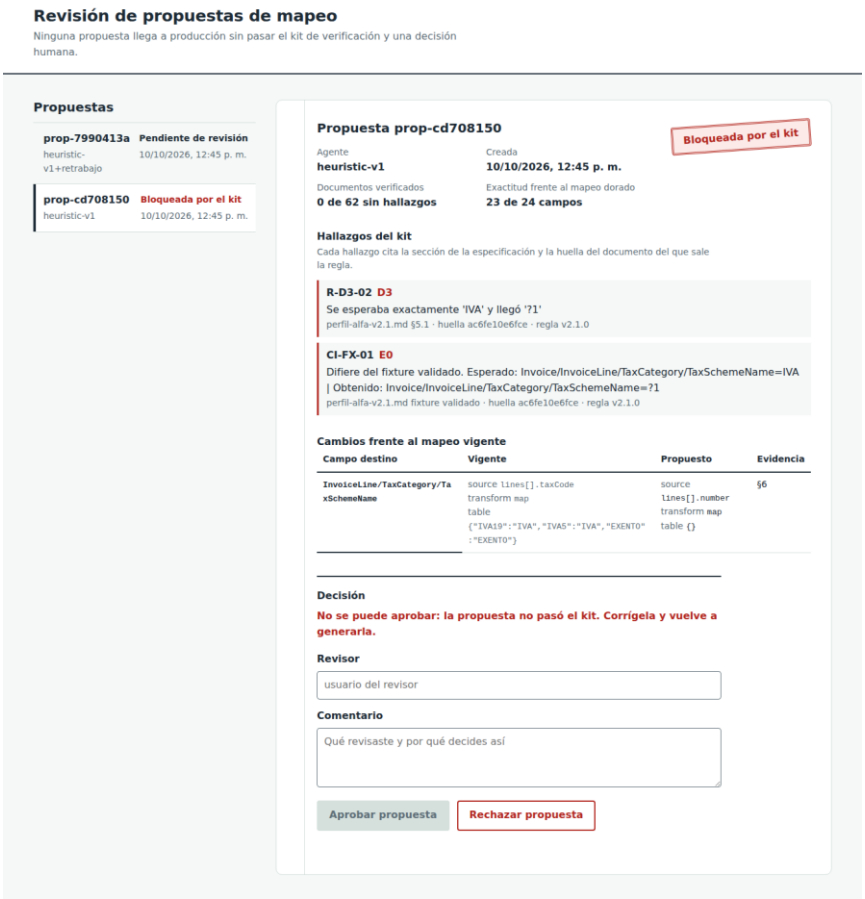
El caso ilustra el valor del diseño. Una exactitud del 95,8 % parece excelente, pero el campo equivocado habría producido un rechazo en cada comprobante. El kit convierte un error silencioso en un hallazgo explicado y trazable antes de que llegue a una persona.

5.4. Consolas de revisión

La figura 15 muestra la consola web, escrita con HTML y CSS puro, sin frameworks; el JavaScript se limita a pedir datos a la API. El sello indica el estado del gate, y cada hallazgo muestra la regla, la sección de la norma y la huella del documento. El botón de aprobar está deshabilitado mientras la propuesta no pase el kit. La figura 16 muestra la consola de

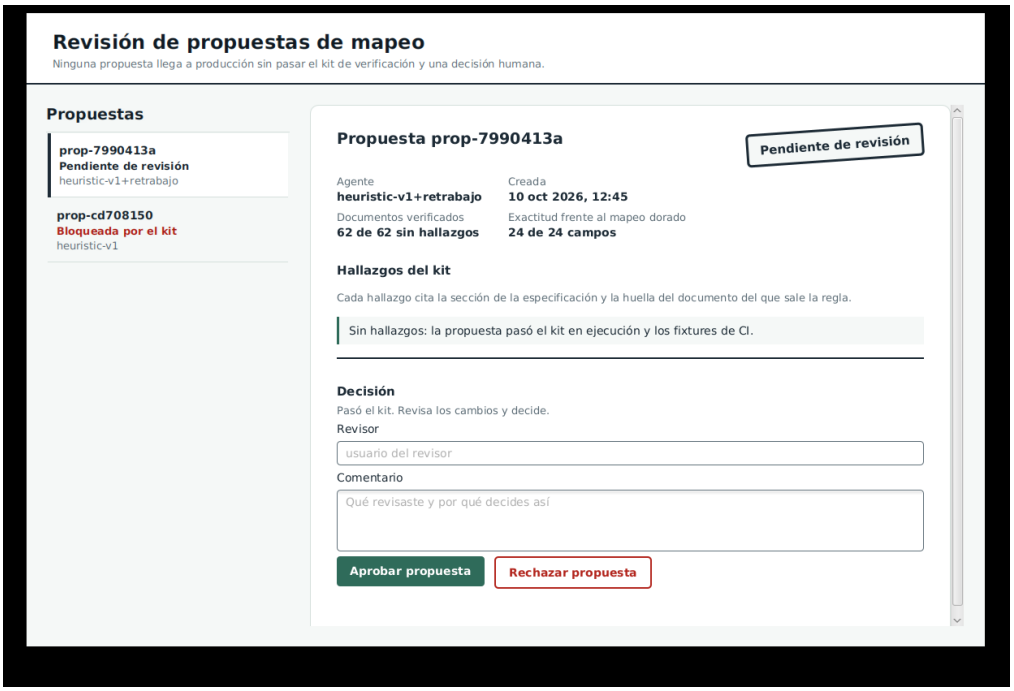
escritorio en JavaFX, con la misma identidad visual definida en una hoja CSS de JavaFX, ejecutándose contra la misma API.

Figura 15. *Consola web de revisión con una propuesta bloqueada por el kit*



Fuente: elaboración propia. Captura de la prueba de concepto en ejecución.

Figura 16. *Consola de escritorio JavaFX con una propuesta pendiente de revisión*



Fuente: elaboración propia. Captura de la aplicación JavaFX ejecutada contra la API de revisión.

5.5. Pipeline, despliegue y madurez alcanzada

El script de etapas del pipeline se ejecutó completo: compilación con advertencias tratadas como errores, 17 pruebas JUnit, 15 pruebas pytest, gate del banco de defectos, gate del mapeo vigente (112 documentos sin hallazgos), empaquetado del JAR con su huella digital y prueba de humo del servicio HTTP, que respondió 200 a un comprobante válido y 422 a uno inválido. El log completo de la ejecución se conserva como evidencia del trabajo. Jenkins se levanta con docker compose a partir de la configuración como código incluida, y ejecuta esas mismas etapas más la construcción de imágenes y el despliegue con Helm.

5.5.1. Madurez alcanzada

Según la escala de niveles de madurez tecnológica (TRL, por technology readiness level), que va de 1 (principios básicos observados) a 9 (sistema probado en entorno operativo) (Comisión Europea, 2014), el kit de verificación queda en TRL 4: validado en laboratorio con datos sintéticos y un banco de defectos. El plano agéntico queda entre TRL 3 y 4: el ciclo gobernado funciona de punta a punta, pero su motor LLM aún no se ha medido con el arnés. El proyecto planificado tiene como meta llevar el conjunto a TRL 6, demostrado en un entorno relevante con un proveedor y una especificación reales.

5.6. Resultados de la planificación

5.6.1. Análisis de incertidumbre

Cada paquete se simula con una distribución beta-PERT sobre la red nivelada. La figura 17 muestra la distribución del plazo total en 10.000 iteraciones y la tabla 50 resume los percentiles.

Tabla 50. *Resultados de la simulación de Monte Carlo*

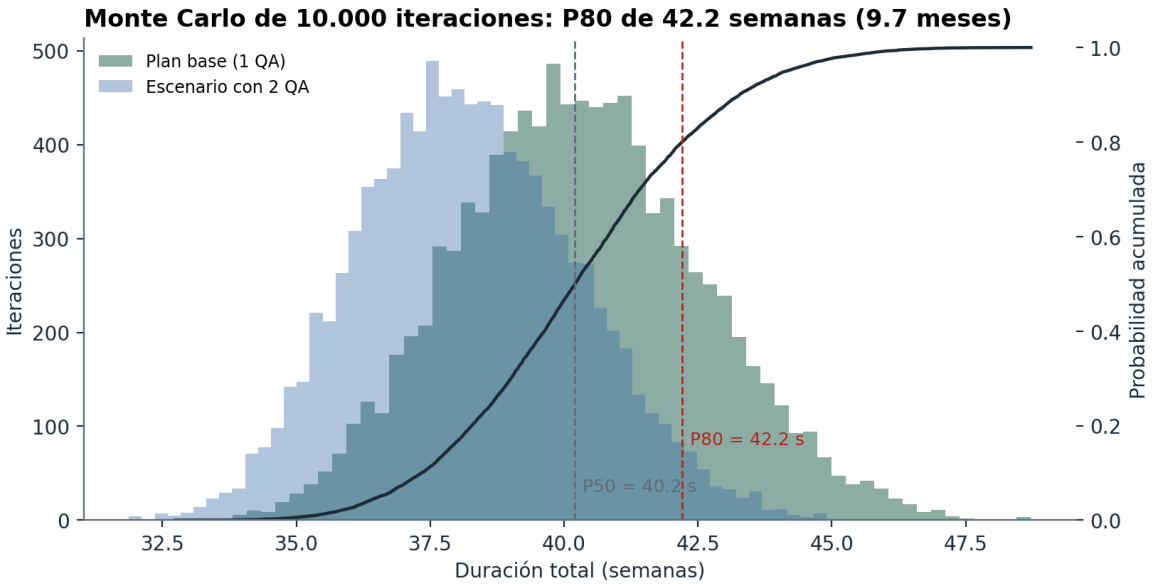
Indicador	P10	P50	P80	P90
Duración (semanas)	37,3	40,2	42,2	43,3
Duración (meses)	8,6	9,3	9,7	10,0
Costo (millones de COP)	947	1.013	1.059	1.085

Fuente: elaboración propia.

El plan comunica la fecha con su probabilidad: hay un 80 % de probabilidad de terminar en 42,2 semanas o menos (9,7 meses), y la probabilidad de cumplir el horizonte de 12 meses es prácticamente total. La diferencia entre P80 y P50 (2,0 semanas y COP 46 millones) es la reserva de contingencia.

Escenario con dos personas de QA. La nivelación revela que QA es el rol cuello de botella. Con una segunda persona de QA, el P50 baja de 40,2 a 38,1 semanas y el costo P50 baja de COP 1.013 millones a COP 982 millones, porque el ahorro en costos fijos y de dedicación continua supera el costo de la persona adicional. La recomendación del plan es incorporar una segunda persona de QA durante las fases F2 a F4.

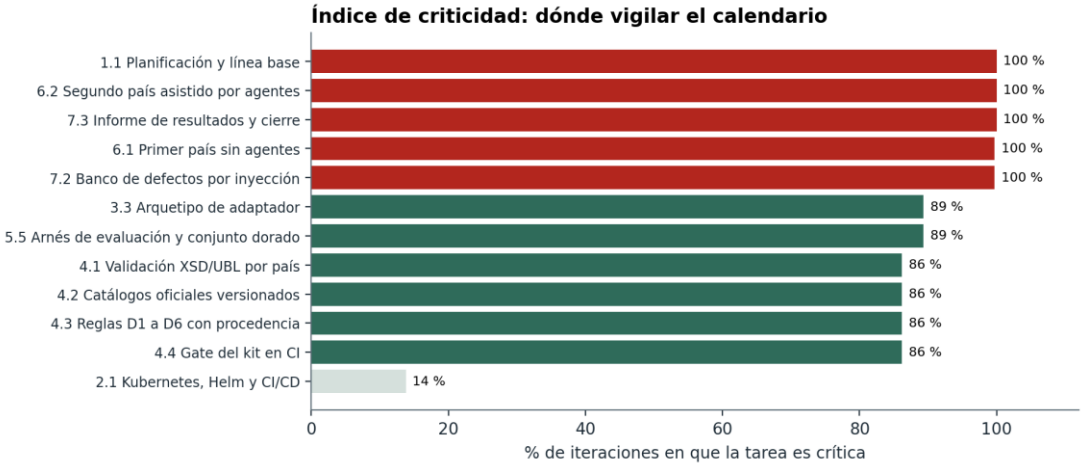
Figura 17. *Distribución del plazo total: plan base y escenario con dos personas de QA*



Fuente: elaboración propia. Salida del modelo de planificación (10.000 iteraciones, semilla fija).

El índice de criticidad, es decir, la proporción de iteraciones en que una tarea queda en la ruta crítica, indica dónde vigilar el calendario (figura 18).

Figura 18. *Índice de criticidad de los paquetes de trabajo*



Fuente: elaboración propia. Salida del modelo de planificación.

Un resultado relevante para la primera subpregunta: la gestión de accesos y credenciales (1.3) tiene un índice de criticidad de 0 % en el plan nivelado, gracias a que se inicia en la primera fase: el paquete 1.3 arranca cuando el ingeniero de operaciones termina el 2.1 (dependencia de recurso) y corre en paralelo con el resto del trabajo técnico. Si se iniciara tarde, sería el camino crítico, y ningún agente podría acortarlo.

5.6.2. Punto de equilibrio

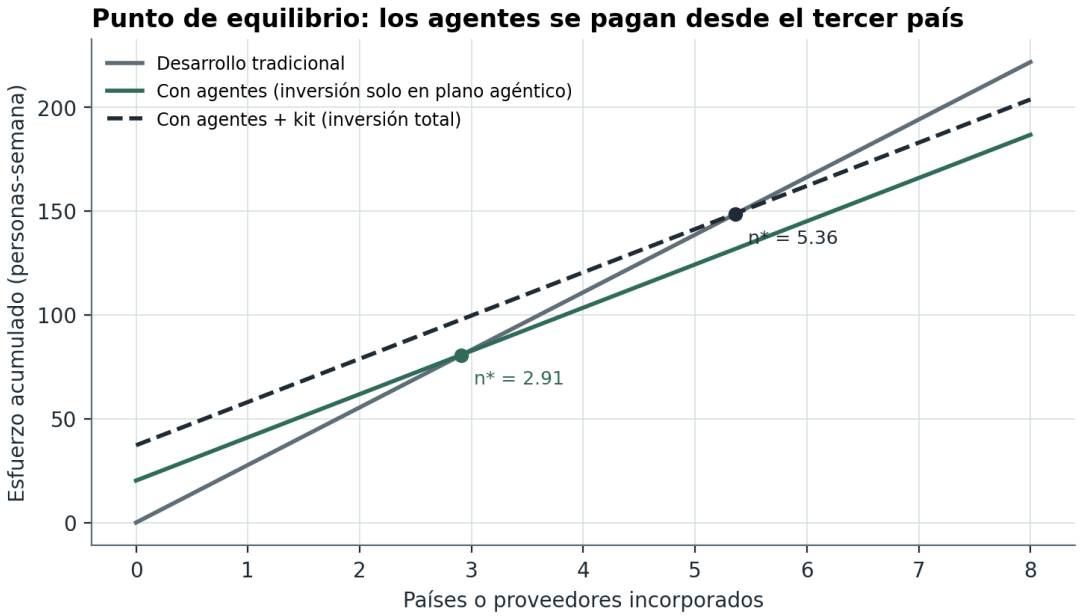
Si e_m es el esfuerzo de incorporar un país de forma manual, e_a el esfuerzo asistido y Eo la inversión inicial, la propuesta se paga a partir de:

$$n^* = Eo / (e_m - e_a)$$

Con el modelo, e_m = 27,7 p-s (paquete 6.1), e_a = 20,8 p-s (paquete 6.2) y el ahorro es de 6,9 p-s por país. Si Eo es solo la inversión en el plano agéntico (20,2 p-s), n = *2,91: los agentes se pagan desde el tercer país. Si se cuenta también el kit y el arquetipo (37,2 p-s), n* = 5,36. Esta segunda cifra es conservadora, porque el kit reduce rechazos aunque no haya agentes.

Para una organización que opera en varios países y cambia de proveedor con cierta frecuencia, la inversión se justifica por esfuerzo: los agentes se pagan a partir del tercer país (n* = 2,9) y la inversión completa, kit incluido, a partir del sexto (n* = 5,4). Si no alcanza esas cifras, la inversión debe justificarse por calidad y trazabilidad, y el plan lo declara así.

Figura 19. *Punto de equilibrio en esfuerzo acumulado*



Fuente: elaboración propia. Salida del modelo de planificación. e_a es una hipótesis que el proyecto mide en la fase F5.

La tabla 51 declara los supuestos del cálculo. Todos son salidas del modelo de planificación con las estimaciones de la EDT, y el más débil es el esfuerzo asistido, que es una hipótesis. Para ver cuánto depende de él el resultado, el modelo se recalculó con tres valores del esfuerzo asistido, correspondientes a las duraciones optimista, esperada y pesimista del paquete 6.2 multiplicadas por su dedicación de 3,2 personas a tiempo completo (tabla 52). Con el valor optimista, los agentes se pagarían desde el segundo país y la inversión completa desde el tercero; con el pesimista, el esfuerzo asistido superaría al manual y la inversión no se recuperaría por esfuerzo, sino solo por calidad y trazabilidad. Por eso el plan mide el esfuerzo real en la fase F5 antes de comprometer las incorporaciones siguientes.

Tabla 51. *Supuestos del punto de equilibrio*

Variable	Valor	Origen
Esfuerzo manual por país, e_m	27,7 p-s	Paquete 6.1: t_e de 8,67 semanas × 3,2 FTE
Esfuerzo asistido por país, e_a	20,8 p-s	Paquete 6.2: t_e de 6,5 semanas × 3,2 FTE; hipótesis por comprobar en F5
Inversión en el plano agéntico, E_0	20,2 p-s	Paquetes 5.1 a 5.5 y 7.2
Inversión completa, E_0	37,2 p-s	Anterior más los paquetes 4.1 a 4.4 y 3.3
Ahorro por país	6,9 p-s, constante	Supuesto: sin efecto de aprendizaje ni costo de mantenimiento por país
Unidad de medida	Personas-semana	Independiente de las tarifas por rol

Fuente: elaboración propia.

Tabla 52. *Sensibilidad del punto de equilibrio al esfuerzo asistido*

Escenario	Duración del paquete 6.2 (semanas)	e_a (p-s)	Ahorro por país (p-s)	n* agentes	n* inversión completa
Optimista	4,0	12,8	14,9	1,35	2,49
Esperado (base)	6,5	20,8	6,9	2,91	5,36
Pesimista	11,0	35,2	−7,5	No se recupera	No se recupera

Fuente: elaboración propia.

5.7. Amenazas a la validez

- **Diseño por el mismo autor:** la especificación, las reglas y los mutantes los diseñó la misma persona, lo que puede favorecer la detección. En el proyecto, los mutantes los propone un tercero.
- **Ejemplos validados por construcción:** la detección del 100 % en integración continua depende de que los ejemplos validados cubran los casos; su cobertura es en sí un requisito de calidad.
- **Tamaño del banco:** el banco original tiene 22 mutantes y el ampliado 129 (5.2.1); ambos son sintéticos y se generaron con los operadores que informaron el diseño de las reglas. Bastan para mostrar el patrón, no para estimar tasas con precisión estadística.
- **Motor heurístico:** la exactitud medida es la de la línea base, no la de un LLM.
- **Sin proveedor real:** la prueba de concepto no mide rechazos externos ni tiempos de proveedor.

6. Conclusiones

Apartado en desarrollo.

Referencias Bibliográficas

Abdalla, A., Thie, V., Schaub, J., Eisenbarth, M., Lee, S.-Y., y Andert, J. (2026). Multi-agent software development for automotive model-based graphical programming. *IEEE Access, 14*, 115016-115028. https://doi.org/10.1109/ACCESS.2026.3711515

Abtahi, F., Astaraki, M., y Seoane, F. (2026). Leveraging imperfection with MEDLEY: A multi-model approach harnessing bias in medical AI. *Frontiers in Artificial Intelligence, 9*, 1701665. https://doi.org/10.3389/frai.2026.1701665

Adnyana, K., y Schwung, A. (2026). From Chain-of-Note to explainable cognitive prompting: Reproducible multi-layer validation for auditable LLM-assisted PLC programming: Plainability and deterministic reasoning for industrial PLC applications. En *12th 2026 International Conference on Control, Decision and Information Technologies, CoDIT 2026* (pp. 130-135). https://doi.org/10.1109/CoDIT70676.2026.11630927

Ajdinović, S., Maisch, N., Kimmel, R., Rasouli, T., Lechler, A., Wortmann, A., y Riedel, O. (2025). AI-powered semantic matching and data harmonization for industrial applications with OPC UA & Asset Administration Shells. En *Proceedings of the International Conference on Mechanical and Intelligent Manufacturing Technologies, ICMIMT* (pp. 248-252). https://doi.org/10.1109/ICMIMT65123.2025.11091954

Al Sunny, S. M. N., Nadaf, L., Srivastav, G. K., Singh, S., Manukonda, Y. R., y Panahi, M. (2026). Engineering guided determinism in neuro-symbolic agents: A comparative study of orchestration strategies for supply chain replenishment. En *Proceedings - 2026 IEEE 50th Annual Computers, Software, and Applications Conference, COMPSAC 2026* (pp. 3292-3297). https://doi.org/10.1109/COMPSAC69091.2026.00491

Alquran, S., Alauthman, M., Al-Jokhadar, A., y Al-Maqusi, A. (2026). AI-enhanced BIM for smart city SDGs: Governance and environment pillars. En *2026 2nd International Conference on Computational Intelligence Approaches and Applications, ICCIAA 2026 - Proceedings*. https://doi.org/10.1109/ICCIAA68481.2026.11543908

Alves, A., Montandon, J. E., y Hora, A. (2026). Testing framework migration with large language models. En *Proceedings - 2026 7th ACM/IEEE International Conference on Automation of Software Test, AST 2026* (pp. 12-22). https://doi.org/10.1145/3793654.3793749

Arjarapu, J. P., Kalakota, S. C., y Pisipati, R. K. (2026). SAGE: Explainable semantic-aware graph-based entity integration for heterogeneous data sources. En *SIGMOD*

Companion 2026 - Companion of the International Conference on Management of Data (pp. 10-13). https://doi.org/10.1145/3788853.3801599

Arsal, M., Ashfaq, H., Mahmood, M. A., y Ashfaq, A. (2025). Towards intelligent tax systems: Evaluating NLP approaches for Pakistan's fiscal needs. En *2025 International Conference on Frontiers of Information Technology, FIT 2025*. https://doi.org/10.1109/FIT67061.2025.11333725

Banco Interamericano de Desarrollo [BID]. (s. f.). *Usos prácticos de la factura electrónica para el control tributario: experiencias de las administraciones tributarias*. Blog Gestión Fiscal. https://blogs.iadb.org/gestion-fiscal/es/usos-practicos-de-la-factura-electronica-para-el-control-tributario-experiencias-de-las-administraciones-tributarias

Barreix, A., y Zambrano, R. (Eds.). (2018). *Factura electrónica en América Latina* (Monografía del BID N.º 595). Banco Interamericano de Desarrollo; Centro Interamericano de Administraciones Tributarias. https://publications.iadb.org/es/factura-electronica-en-america-latina

Barron, F. H., y Barrett, B. E. (1996). Decision quality using ranked attribute weights. *Management Science, 42*(11), 1515-1523. https://doi.org/10.1287/mnsc.42.11.1515

Basili, V. R., Caldiera, G., y Rombach, H. D. (1994). The goal question metric approach. En J. J. Marciniak (Ed.), *Encyclopedia of software engineering* (pp. 528-532). Wiley.

Bass, L., Clements, P., y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Brown, S. (s. f.). *The C4 model for visualising software architecture*. https://c4model.com

Buk. (2026). *Costo real de un empleado en Colombia 2026*. https://www.buk.co/blog/costo-real-de-un-empleado-en-colombia-2026

Cabrera, L., Romero, S., y Aguirre, J. (2026). De la factura física al documento digital: transformación tributaria y deducción del gasto en Perú. *Revista InveCom, 6*(1), 1-10. https://zenodo.org/records/15529473

Calijuri, M. (2024). Revolucionando la tributación: la administración tributaria 3.0 ya está aquí (Nota técnica). Banco Interamericano de Desarrollo. https://publications.iadb.org/publications/spanish/document/revolucionando-la-tributacion-la-administracion-tributaria-ya-esta-aqui.pdf

Cheng, L.-C., Hung, C.-Y., y Chien, T.-N. (2026). The evolution of clinical intelligence through GenAI co-pilots: A systematic review and thematic synthesis. *International*

Journal of Advanced Computer Science and Applications, 17(7), 360-375. https://doi.org/10.14569/IJACSA.2026.0170733

CMMI Product Team. (2010). *CMMI for Development, version 1.3* (CMU/SEI-2010-TR-033). Software Engineering Institute, Carnegie Mellon University.

Colombia Move. (2026). *Salarios en Colombia por industria: rangos reales 2026* [Entrada de blog]. https://colombiamove.com/blog/es/average-salaries-colombia-by-industry/

Comisión Europea. (2014). *Horizon 2020 – Work Programme 2014-2015: General annexes. G. Technology readiness levels (TRL)* (Decisión de la Comisión C(2014)4995). https://ec.europa.eu/research/participants/data/ref/h2020/wp/2014_2015/annexes/h2020-wp1415-annex-g-trl_en.pdf

Comité Europeo de Normalización [CEN]. (2017). *EN 16931-1:2017. Electronic invoicing – Part 1: Semantic data model of the core elements of an electronic invoice*.

Darrazão, E., Oliveira, K., y Gomes, L. C., Jr. (2025). Sequence labeling in product descriptions on invoices: Comparing LLM-based settings with a CRF baseline. *Journal of the Brazilian Computer Society, 31*(1), 1203-1212. https://doi.org/10.5753/jbcs.2025.5743

De Velazco, J. L. (2016). La facturación electrónica en el Perú. *Lidera, 11*, 4-10. https://revistas.pucp.edu.pe/index.php/revistalidera/article/view/16873

Departamento Nacional de Planeación [DNP]. (2025). *Documento CONPES 4144: Política Nacional de Inteligencia Artificial*. Consejo Nacional de Política Económica y Social.

Dirección de Impuestos y Aduanas Nacionales [DIAN]. (2023). *Resolución 000165 de 2023, por la cual se desarrolla el sistema de facturación, los proveedores tecnológicos, se adopta la versión 1.9 del anexo técnico de factura electrónica de venta y se dictan otras disposiciones*.

Dirección de Impuestos y Aduanas Nacionales [DIAN]. (2024). *Resolución 000008 de 2024, por la cual se modifica el artículo 23 y parcialmente el parágrafo del artículo 62 de la Resolución 000165 de 2023*. https://normograma.mintic.gov.co/docs/pdf/resolucion_dian_0008_2024.pdf

Escobar, M. (2019). La factura electrónica. Un caso de análisis del principio de equivalencia funcional para el derecho chileno. *Revista de Derecho, 242*, 159-188. https://revistas.udec.cl/index.php/revista_de_derecho/article/view/752

Fraser, T., Li, P., Niveyro, S., Lawrence, P., Dobransky, M., Kuang, L., Guo, Y., Murphy, E., y Gao, H. O. (2026). Accelerating data insights with generative AI for emissions

reporting in urban systems. *Procedia Computer Science, 288*, 160-169. https://doi.org/10.1016/j.procs.2026.08.195

Gao, Y., Cao, J., Xie, Z., Wang, L., y Yang, W. (2026). TaxAgent-MA: A multi-agent framework for auditable and explainable tax recommendation generation. En *2026 29th International Conference on Computer Supported Cooperative Work in Design (CSCWD)* (pp. 3557-3562). IEEE. https://doi.org/10.1109/CSCWD68734.2026.11582797

Gao, Y., Yiu, T. W., Shen, X., y Tam, V. W. Y. (2026). Graph retrieval augmented generation for multimodal construction and demolition waste data integration toward traceable disposal recommendations. En *Proceedings of the International Symposium on Automation and Robotics in Construction* (pp. 888-895). https://doi.org/10.22260/ISARC2026/0114

Giraldo-Hurtado, J. S. (2023). *Catálogo de herramientas de gestión de la configuración que implementan la metodología GitOps* [Trabajo fin de máster, Universidad Internacional de La Rioja]. Re-UNIR. https://reunir.unir.net/handle/123456789/15912

Gogani Khiabani, S., Trivedi, A., Saha, D., y Tizpaz-Niari, S. (2026). An LLM agentic approach for legal-critical software: A case study for tax prep software. En *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering* (pp. 272-283). ACM. https://doi.org/10.1145/3744916.3764575

Gray, C. F., y Larson, E. W. (2009). *Administración de proyectos* (4.ª ed.). McGraw-Hill.

Hanley, J. A., y Lippman-Hand, A. (1983). If nothing goes wrong, is everything all right? Interpreting zero numerators. *JAMA, 249*(13), 1743-1745. https://doi.org/10.1001/jama.1983.03330370053031

He, R., Zhang, Y., Zhang, M., y Rausch, A. (2025). LLM-assisted tool for joint generation of formulas and functions in rule-based verification of map transformations. En *Proceedings - 2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops, ASEW 2025* (pp. 169-172). https://doi.org/10.1109/ASEW67777.2025.00040

He, Z., Li, M., y Zhang, Y. (2026). A brief survey of LLMs and intelligent agents in financial investment decision making. *Procedia Computer Science, 287*, 1333-1339. https://doi.org/10.1016/j.procs.2026.08.177

Hernández Sampieri, R., Fernández Collado, C., y Baptista Lucio, P. (2014). *Metodología de la investigación* (6.ª ed.). McGraw-Hill.

Hevner, A. R., March, S. T., Park, J., y Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75-105. https://doi.org/10.2307/25148625

Hohpe, G., y Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., y Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology, 33*(8).

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., y Liu, T. (2025). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems, 43*(2). https://doi.org/10.1145/3703155

Iijima, T., Muranaka, N., Masui, T., Hasegawa, T., y Mori, H. (2026). LLM-driven network configuration generation with LangGraph. *IEICE Transactions on Communications, E109-B*(10), 1224-1234. https://doi.org/10.23919/transcom.2026EBP3005

Ishikawa, K. (1986). *¿Qué es el control total de calidad? La modalidad japonesa*. Editorial Norma.

ISO/IEC. (2023). *ISO/IEC 25010:2023. Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model.*

Jia, Y., y Harman, M. (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering, 37*(5), 649-678.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., y Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? En *International Conference on Learning Representations (ICLR 2024).* https://arxiv.org/abs/2310.06770

Joshi, V., Kothari, N., y Kumar, L. (2026). KISG: A knowledge-infused agentic framework for logically consistent synthetic data generation. En *Proceedings - IEEE Madhya Pradesh Section Conference, MPCON 2026* (pp. 1592-1596). https://doi.org/10.1109/MPCON69668.2026.11508492

Juran, J. M. (1990). *Juran y la planificación para la calidad*. Díaz de Santos.

Kendall, K. E., y Kendall, J. E. (2011). *Análisis y diseño de sistemas* (8.ª ed.). Pearson Educación.

Kolisch, R. (1996). Serial and parallel resource-constrained project scheduling methods revisited: Theory and computation. *European Journal of Operational Research, 90*(2), 320-333.

La FM. (2024). *Las profesiones mejor pagadas: se gana hasta $26 millones al mes*. https://www.lafm.com.co/empleo/las-profesiones-mejor-pagadas-se-gana-hasta-26-millones-al-mes

Lee, E. Y., Leeroy, G. C., y Leeroy, W. (2024). Impact of blockchain on improving taxpayers' compliance: Empirical evidence from panel data model and agent-based simulation. *Journal of Emerging Technologies in Accounting, 21*(1), 89-109. https://doi.org/10.2308/JETA-2022-046

Leguizamón, J. D., y Lugo, S. (2024). Implementación de la facturación electrónica en América Latina: revisión de literatura. *Arandu UTIC, 11*(2), 2564-2587. https://doi.org/10.69639/arandu.v11i2.446

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., y Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459-9474.

Lianghua, H., Xiaoyan, T., y Le, L. (2026). Measuring corporate political identity and its impact on tax compliance behaviour: Evidence from China. *Journal of Business Research, 217*, 116426. https://doi.org/10.1016/j.jbusres.2026.116426

Liu, Y., Peña, E., Santos, A., Wu, E., y Freire, J. (2025). Magneto: Combining small and large language models for schema matching. *Proceedings of the VLDB Endowment, 18*(8), 2681-2694. https://doi.org/10.14778/3742728.3742757

Luiz Firmino Alves, A., de Souza Baptista, C., Itallo Martins Silva Diniz, J., Igor de Lima Mendes, F., y Queiroz Cunha, M. (2025). An approach for product record linkage using cross-lingual learning and large language models. En *International Conference on Enterprise Information Systems, ICEIS - Proceedings* (Vol. 1, pp. 63-74). https://doi.org/10.5220/0013285000003929

Lushnei, S., Shumskyi, D., Shykula, S., Jiménez-Ruiz, E., y d'Avila Garcez, A. (2026). Large language models as oracles for ontology alignment. En *EACL 2026 - 19th Conference of the European Chapter of the Association for Computational Linguistics, Proceedings of the Conference, Vol. 1 - (Long Papers)* (pp. 2435-2449). https://doi.org/10.18653/v1/2026.eacl-long.110

Malcolm, D. G., Roseboom, J. H., Clark, C. E., y Fazar, W. (1959). Application of a technique for research and development program evaluation. *Operations Research, 7*(5), 646-669.

MITRE. (s. f.). *MITRE ATLAS: Adversarial threat landscape for AI systems*. The MITRE Corporation. https://atlas.mitre.org/

Model Context Protocol. (2026). *Model Context Protocol specification* (revisión 2026-07-28). https://github.com/modelcontextprotocol/specification/releases/tag/2026-07-28

Mohan, B., Surasani, V. R., y Kumar, R. (2026). Autonomous data stewardship: Multi-agent AI for real-time master data management in financial services. En *2026 IEEE 16th Annual Computing and Communication Workshop and Conference, CCWC 2026* (pp. 689-698). https://doi.org/10.1109/CCWC67433.2026.11393832

Narayan, A., Chami, I., Orr, L., y Ré, C. (2022). Can foundation models wrangle your data? *Proceedings of the VLDB Endowment, 16*(4), 738-746. https://www.vldb.org/pvldb/vol16/p738-narayan.pdf

National Institute of Standards and Technology [NIST]. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). https://doi.org/10.6028/NIST.AI.100-1

Nguyen, Q. T., Pham, T.-B., Bui, H. P., y Chen, P.-H. (2026). Applying action research to developing a GPT-based assistant for construction cost code verification in state-funded projects in Vietnam. *Buildings, 16*(3), 499. https://doi.org/10.3390/buildings16030499

Nie, R., Wu, H., Ma, L., Liu, Z., Wang, Z., y Zhang, P. (2024). Towards a conversational invoice issuance LLM-based agent. En *ACAI 2024 - 2024 7th International Conference on Algorithms, Computing and Artificial Intelligence*. https://doi.org/10.1109/ACAI63924.2024.10899737

Nygard, M. (2011). *Documenting architecture decisions*. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

OASIS. (2013). *Universal Business Language version 2.1* [OASIS Standard]. http://docs.oasis-open.org/ubl/os-UBL-2.1/UBL-2.1.html

OCDE. (2022). *Tax administration 3.0 and electronic invoicing: Initial findings*. https://www.oecd.org/en/publications/tax-administration-3-0-and-electronic-invoicing_2ffc88ed-en.html

OCDE y Eurostat. (2006). *Manual de Oslo: guía para la recogida e interpretación de datos sobre innovación* (3.ª ed.). Grupo Tragsa.

OWASP. (2024). *OWASP Top 10 for LLM applications 2025*. OWASP GenAI Security Project. https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/

Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hróbjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., McDonald, S., … Alonso-Fernández, S. (2021). Declaración PRISMA 2020: una guía actualizada para la publicación de revisiones sistemáticas. *Revista Española de Cardiología, 74*(9), 790-799. https://doi.org/10.1016/j.recesp.2021.06.016

Parasuraman, R., y Manzey, D. H. (2010). Complacency and bias in human use of automation: An attentional integration. *Human Factors, 52*(3), 381-410. https://doi.org/10.1177/0018720810376055

Parciak, M., Vandevoort, B., Neven, F., Peeters, L. M., y Vansummeren, S. (2024). Schema matching with large language models: An experimental study. En *Proceedings of Workshops at the 50th International Conference on Very Large Data Bases (VLDB 2024)*. https://vldb.org/workshops/2024/proceedings/TaDA/TaDA.8.pdf

Peffers, K., Tuunanen, T., Rothenberger, M. A., y Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45-77. https://doi.org/10.2753/MIS0742-1222240302

Pinto, D., Feremans, L., Neven, F., Parciak, M., y Vansummeren, S. (2026). LLM-generated schema matching with formally verifiable explanations. En *2026 IEEE 42nd International Conference on Data Engineering Workshops (ICDEW)* (pp. 176-186). IEEE. https://doi.org/10.1109/ICDEW71238.2026.00023

Portafolio. (2026). *Salarios en tecnología en Colombia 2026: cuánto ganan perfiles en inteligencia artificial, datos y software*. https://www.portafolio.co/tecnologia/salarios-en-tecnologia-en-colombia-2026-cuanto-ganan-perfiles-en-inteligencia-artificial-datos-y-software-491946

Pressman, R. S. (2010). *Ingeniería del software: un enfoque práctico* (7.ª ed.). McGraw-Hill.

Project Management Institute [PMI]. (2021). *Guía de los fundamentos para la dirección de proyectos (Guía del PMBOK)* (7.ª ed.).

Quevedo-Tumailli, V., Arias, S. E., Ortega, V. A., y Ortega-Tenezaca, B. (2025). Impacto de modelos de lenguaje de gran tamaño en calidad y eficiencia de generación de código:

revisión sistemática de literatura. *Novasinergia, 8*(1), 52-66. https://doi.org/10.37135/ns.01.15.10

Rahm, E., y Bernstein, P. A. (2001). A survey of approaches to automatic schema matching. *The VLDB Journal, 10*(4), 334-350.

Ramírez-Álvarez, J., Oliva, N., y Andino, M. (2022). Cumplimiento tributario y facturación electrónica en Ecuador: evaluación de impacto. *Problemas del Desarrollo. Revista Latinoamericana de Economía, 53*(208), 97-123. https://doi.org/10.22201/iiec.20078951e.2022.208.69712

Raza, M. R., Rabhi, F., Joukhadar, G., y Kumar, A. (2026). AI-driven business process composition for enterprise information exchange within IE-BSE framework: An empirical case analysis on e-invoicing. *Lecture Notes in Business Information Processing, 584*, 143-154. https://doi.org/10.1007/978-3-032-26363-6_11

Ren, D., y Chen, H. (2026). Theoretical innovation and empirical testing of intelligent fiscal and taxation systems in the digital economy. En *Proceedings of the 2nd International Conference on Artificial Intelligence, Digital Media Technology and Social Computing, ICAIDS 2026* (pp. 702-707). https://doi.org/10.1145/3806262.3806364

Romero, C. (1993). *Teoría de la decisión multicriterio: conceptos, técnicas y aplicaciones*. Alianza Editorial.

Santos, A., y Muniz Silva, P. S. (2026). Automated transformation of financial regulations into SBVR: An ontology-based and natural language processing approach. *Artificial Intelligence and Law*. https://doi.org/10.1007/s10506-026-09532-7

Shostack, A. (2014). *Threat modeling: Designing for security*. Wiley.

Shrikaran, P., Mamalaivasan, H., Chitradevi, D., Kumar, S. T., y Abishekwoolridge. (2025). A secure on-premises ETL pipeline for enterprise data warehousing: Integrating OCR and local LLMs. En *Proceedings of 8th International Conference on Computing Methodologies and Communication, ICCMC 2025* (pp. 1859-1864). https://doi.org/10.1109/ICCMC65190.2025.11141002

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

Srinivas, D., Das, R., Tizpaz-Niari, S., Trivedi, A., y Pacheco, M. L. (2023). On the potential and limitations of few-shot in-context learning to generate metamorphic specifications for tax preparation software. En *NLLP 2023 - Natural Legal Language Processing Workshop 2023, Proceedings of the Workshop* (pp. 230-243).

Superintendencia Nacional de Aduanas y de Administración Tributaria [SUNAT]. (2010). *Resolución de Superintendencia N.º 188-2010/SUNAT*.

Tudor, C., Achirei, S. D., y Simionescu, C. (2025). ContaGPT: A domain-adapted LLM for Romanian financial and accounting applications. En *Proceedings - 27th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing, SYNASC 2025* (pp. 302-308). https://doi.org/10.1109/SYNASC69064.2025.00047

Valora Analitik. (2026). *Estos trabajos en Colombia están pagando hasta $23 millones mensuales*. https://www.valoraanalitik.com/?p=572216

Vergés, R., Gaspar, K., Forcada, N., y Hosseini, M. R. (2026). Human-in-the-loop agent for product regulatory screening: Case study on concrete. *Automation in Construction, 185*, 106876. https://doi.org/10.1016/j.autcon.2026.106876

Vieira-Vieira, C. H., Kulkarni, S. S., Zalewski, A., Löffler, J., Münch, J., y Kreuchwig, A. (2025). From data silos to insights: The PRINCE multi-agent knowledge engine for preclinical drug development. *Frontiers in Artificial Intelligence, 8*, 1636809. https://doi.org/10.3389/frai.2025.1636809

Villanueva-Miranda, I., Gu, Z., Xiao, G., y Xie, Y. (2026). HarmoMed: An agentic framework for auditable multimodal biomedical data harmonization. En *Proceedings of the 17th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics* (pp. 1-6). ACM. https://doi.org/10.1145/3807503.3819371

Wahed, M. A., Aljawarneh, N. M., Almarashdi, O., y Almarashdi, F. (2026). Agentic automation in healthcare finance: A next-generation model for invoice processing. En *2026 International Conference on Integrated Intelligence and Cognitive Engineering, ICIICE 2026*. https://doi.org/10.1109/ICIICE69672.2026.11565502

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., y Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*(6), 186345. https://doi.org/10.1007/s11704-024-40231-1

Wang, X., Chen, H., Zeng, X., Liu, J., y Fang, X. (2025). Design and application of electronic invoice information systems using rule-based model view controller framework with YOLOV8 model. En *International Conference on Communication, Computer and Information Technology, IC3IT 2025*. https://doi.org/10.1109/IC3IT66137.2025.11341366

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association, 22*(158), 209-212. https://doi.org/10.1080/01621459.1927.10502953

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., y Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. En *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629

Yu, M., Guo, Z., Sun, D., y Zhi, Q. (2026). Practical exploration and research on AI-based privacy compliance testing for automotive apps. En *2026 5th International Conference on Artificial Intelligence, Internet of Things and Cloud Computing Technology, AIoTC 2026* (pp. 914-919). https://doi.org/10.1109/AIoTC71250.2026.11688636

Zhang, Z., Li, J., y Yang, Z. (2026). From implicit knowledge to audit-ready evidence: Optimizing LLM reasoning for advanced manufacturing tax compliance. En *Proceedings of 2026 5th International Conference on Cyber Security, Artificial Intelligence and Digital Economy, CSAIDE 2026* (pp. 909-913). https://doi.org/10.1145/3819836.3819971

Anexos

Anexo A. Decisiones arquitectónicas completas

ADR-01. Topics de Kafka por país y proveedor

Estado: aceptada. **Contexto:** Cada país tiene volúmenes, ventanas de cierre y proveedores distintos. Un fallo del proveedor de un país no puede detener la emisión de los demás.

- **Cola única con filtrado por cabecera:** Simple de operar; un consumidor lento o un mensaje envenenado afecta a todos los países.
- **Topic por país y proveedor (elegida):** Aislamiento de fallos, escalado y retención por país; más topics que gobernar.
- **Colas por tipo de documento:** Aísla por tipo, no por país; no resuelve el aislamiento de proveedores.

Decisión: Se usan topics con la convención {pais}.invoice.{request|response|dlq}.v{n} y {pais}.{proveedor}.{request|response}.v{n}, con particionado por número de documento para conservar el orden por comprobante.

Consecuencias: Positivas: un incidente queda contenido en su país; la retención y las réplicas se ajustan por volumen. Negativas: más topics y ACL que administrar, mitigado con plantillas de infraestructura como código.

Escenario de calidad: Fiabilidad: un proveedor caído durante 2 horas no degrada la emisión de los otros países (0 mensajes perdidos, latencia del resto sin cambio).
Evidencia en la prueba de concepto: Fuera del alcance de la prueba de concepto (no incluye Kafka); se valida en la fase F1 del proyecto planificado.

ADR-02. Adaptador por proveedor generado con arquetipo

Estado: aceptada. **Contexto:** Los formatos de los proveedores son heterogéneos (JSON, texto por segmentos, XML/UBL). Un transformador genérico configurable tiende a convertirse en un motor de reglas opaco.

- **Transformador genérico configurable:** Un solo componente; la configuración crece hasta ser un lenguaje propio difícil de probar.
- **Adaptador por proveedor desde arquetipo (elegida):** Código explícito por proveedor y comprobable mediante pruebas; el arquetipo fija estructura, pruebas y verificación.

- **iPaaS con mapeo visual:** Rápido al inicio; dependencia del fabricante y mapeo manual en cada país.

Decisión: Cada proveedor tiene su adaptador generado con un arquetipo Maven que incluye el kit de verificación embebido y las pruebas de contrato.

Consecuencias: Positivas: cada adaptador es pequeño, comprobable mediante pruebas y desplegable por separado. Negativas: duplicación controlada entre adaptadores, que se reduce moviendo lo común al kit y a librerías compartidas.

Escenario de calidad: Mantenibilidad: incorporar un proveedor nuevo parte de un esqueleto que ya compila, pasa el kit y tiene su pipeline. **Evidencia en la prueba de concepto:** El transformador de la prueba de concepto separa el mapeo (datos) del motor de renderizado: el arquetipo empaqueta ambos.

ADR-03. Python para el plano agéntico

Estado: aceptada. **Contexto:** El plano de integración es Java. Los agentes necesitan el ecosistema de LLM, MCP y orquestación de agentes, donde Python es la referencia.

- **Spring AI en Java:** Un solo lenguaje; ecosistema de agentes y MCP menos maduro.
- **Python con LangGraph y SDK de MCP (elegida):** Ecosistema más maduro y documentado; dos lenguajes en el equipo.
- **TypeScript:** Buen SDK de MCP; menor afinidad con el equipo de datos e IA.

Decisión: El plano agéntico se implementa en Python 3.12. La frontera con Java es el kit de verificación, expuesto por HTTP y por CLI.

Consecuencias: Positivas: acceso directo al ecosistema de IA. Negativas: dos cadenas de herramientas; el contrato del kit debe ser estable y versionado.

Escenario de calidad: Mantenibilidad: el plano agéntico puede cambiar de proveedor de LLM sin tocar el plano de integración. **Evidencia en la prueba de concepto:** La prueba de concepto implementa el agente, el servidor MCP y la API en Python, e invoca el kit Java por CLI y HTTP.

ADR-04. Los agentes solo proponen mediante solicitud de cambio

Estado: aceptada. **Contexto:** Un LLM puede producir salidas plausibles e incorrectas. En facturación, una salida incorrecta es un documento rechazado o una contingencia fiscal.

- **Agentes con permiso de despliegue:** Máxima velocidad; responsabilidad difusa y riesgo inaceptable en un dominio regulado.

- **Agentes que proponen por solicitud de cambio (elegida):** Toda salida pasa por revisión y por el pipeline; añade latencia humana.
- **Sin agentes:** Elimina el riesgo de IA; no reduce el costo de incorporar países.

Decisión: Los agentes solo crean ramas y solicitudes de cambio. No tienen permiso de merge ni acceso a producción. Cada propuesta queda registrada en agent_proposal con su verificación, revisor y decisión.

Consecuencias: Positivas: responsabilidad clara y trazabilidad completa. Negativas: la revisión humana es un cuello de botella, mitigado porque el kit filtra antes lo que no puede aprobarse.

Escenario de calidad: Seguridad (responsabilidad): cualquier cambio en producción se puede rastrear hasta su propuesta, su verificación y la persona que la aprobó. **Evidencia en la prueba de concepto:** La API de revisión impide aprobar una propuesta bloqueada por el kit (HTTP 409), comprobado en las pruebas automatizadas.

ADR-05. Verificación determinista como gate en ejecución y en integración continua

Estado: aceptada. **Contexto:** La revisión humana por sí sola no detecta de forma fiable códigos válidos pero equivocados ni intercambios de campos. Se necesita un verificador que no dependa del juicio del modelo.

- **Revisión humana como único control:** Sin inversión inicial; detección irregular y no medible.
- **Solo validación XSD:** Barata; en la prueba de concepto detecta el 23 % de los mutantes.
- **Kit en ejecución + ejemplos validados en integración continua (elegida):** En la prueba de concepto detecta el 86 % en ejecución y el 100 % con integración continua, sin falsos positivos.

Decisión: El kit se embebe en cada adaptador (verificación en ejecución) y se ejecuta como gate obligatorio en el pipeline con ejemplos validados por personas.

Consecuencias: Positivas: control objetivo, medible y trazable a la norma. Negativas: hay que mantener reglas y ejemplos validados por versión normativa; se asume como parte del costo de cada país.

Escenario de calidad: Adecuación funcional: el 100 % de los comprobantes pasa XSD y reglas antes de salir; ninguna propuesta que falle el kit puede aprobarse. **Evidencia**

en la prueba de concepto: Banco de 22 mutantes: esquema 23 %, kit en ejecución 86 %, kit + integración continua 100 %; 0 falsos positivos en 1.224 verificaciones correctas.

ADR-06. Respuesta con fuente primaria y respaldo

Estado: aceptada. **Contexto:** La respuesta del proveedor puede perderse o llegar incompleta. La constancia oficial de la autoridad también es recuperable.

- **Solo la respuesta del proveedor:** Simple; un fallo del proveedor deja comprobantes sin estado.
- **Fuente primaria y respaldo (elegida):** Más resiliente; requiere correlación y reconciliación.
- **Consulta directa a la autoridad:** No siempre disponible y depende de cada país.

Decisión: La respuesta del proveedor es la fuente primaria; la constancia oficial en UBL 2.1 es el respaldo. Ambas se correlacionan por correlationId y número de documento.

Consecuencias: Positivas: menos comprobantes en estado incierto. Negativas: lógica de reconciliación y pruebas adicionales.

Escenario de calidad: Fiabilidad: ningún comprobante autorizado queda sin estado final tras la reconciliación diaria. **Evidencia en la prueba de concepto:** Fuera del alcance de la prueba de concepto; se valida en la fase F3.

ADR-07. Archivo legal asíncrono de mejor esfuerzo con reintento

Estado: aceptada. **Contexto:** El archivo legal (PDF, XML y constancia) es obligatorio, pero no debe bloquear la actualización del estado del comprobante.

- **Archivo síncrono bloqueante:** Consistencia inmediata; un fallo del almacenamiento detiene la emisión.
- **Archivo asíncrono con reintento y alerta (elegida):** Disponibilidad del flujo principal; consistencia eventual controlada.

Decisión: El archivado ocurre de forma asíncrona, con reintentos, cuarentena en DLQ y alerta si se supera el umbral. El almacenamiento es inmutable (WORM) con retención por país.

Consecuencias: Positivas: el flujo principal no depende del almacenamiento. Negativas: ventana corta de consistencia eventual que debe monitorearse.

Escenario de calidad: Disponibilidad: una caída del almacenamiento de 30 minutos no detiene la emisión y se recupera sin pérdida. **Evidencia en la prueba de concepto:** Fuera del alcance de la prueba de concepto; se valida en la fase F3.

ADR-08. Schema Registry con compatibilidad BACKWARD

Estado: aceptada. **Contexto:** El modelo canónico evolucionará (por ejemplo, el correo del adquirente pasó a ser obligatorio). Los consumidores no se despliegan todos a la vez.

- **Contratos sin registro:** Libertad total; roturas silenciosas entre servicios.
- **Registro con compatibilidad BACKWARD (elegida):** Evolución segura; disciplina en los cambios de esquema.

Decisión: El modelo canónico InvoiceRequest se registra en Schema Registry con compatibilidad BACKWARD; los campos nuevos nacen opcionales con valor por defecto.

Consecuencias: Positivas: los consumidores antiguos siguen funcionando. Negativas: un campo obligatorio requiere un proceso de dos versiones.

Escenario de calidad: Mantenibilidad: un cambio del canónico no rompe a ningún consumidor desplegado. **Evidencia en la prueba de concepto:** El defecto D4 de la prueba de concepto reproduce justamente un campo obligatorio que el canónico antiguo no traía.

ADR-09. Migraciones de base de datos con Flyway

Estado: aceptada. **Contexto:** Cada adaptador persiste estado y auditoría. Los entornos SIT, UAT y PROD deben tener el mismo esquema.

- **Scripts manuales:** Sin herramienta; deriva entre entornos y errores de orden.
- **Flyway versionado en el repositorio (elegida):** Reproducible y auditable; disciplina de versiones.

Decisión: Cada servicio versiona sus migraciones con Flyway; el pipeline las aplica antes del despliegue.

Consecuencias: Positivas: entornos reproducibles. Negativas: las migraciones destructivas exigen un procedimiento de dos pasos.

Escenario de calidad: Mantenibilidad: cualquier entorno se reconstruye desde cero con el mismo esquema. **Evidencia en la prueba de concepto:** La prueba de concepto usa SQLite con esquema declarado en código para agent_proposal; en el proyecto se migra con Flyway.

ADR-10. RAG con procedencia para leer especificaciones

Estado: aceptada. **Contexto:** Las especificaciones son largas y cambian por versión. El agente necesita citar exactamente de dónde sale cada decisión.

- **Especificación completa en el prompt:** Simple; costoso, sin trazas y sin control de versión.
- **RAG con procedencia (elegida):** Cada fragmento lleva sección y huella digital del documento; costo menor por consulta.
- **Ajuste fino del modelo:** Costoso y rígido ante cambios normativos.

Decisión: El lector de especificaciones devuelve fragmentos con su sección y el SHA-256 del documento. Toda regla y toda propuesta guardan esa procedencia.

Consecuencias: Positivas: trazabilidad regla → norma y auditoría de cambios normativos. Negativas: hay que reindexar cada versión de la norma.

Escenario de calidad: Seguridad (responsabilidad): cada regla y propuesta se rastrea hasta la sección y la versión exacta del documento que la origina. **Evidencia en la prueba de concepto:** El servidor MCP devuelve sección y huella; cada hallazgo del kit incluye fuente, sección, huella digital y versión de la regla.

ADR-11. Identidad del token y mínimo privilegio por rol

Estado: aceptada. **Contexto:** La API de revisión y el registro de mapeos registran quién decide. Si la identidad se toma del cuerpo de la petición, cualquiera que alcance la API puede decidir en nombre de otra persona y la auditoría deja de ser fiable; una revisión automática de seguridad encontró ese patrón en la prueba de concepto inicial.

- **Identidad declarada en el cuerpo de la petición:** Simple; permite la suplantación y anula la auditoría.
- **Identidad del token y roles con mínimo privilegio (elegida):** Auditoría fiable y separación de funciones; exige gestionar tokens y roles.
- **Una sola identidad compartida por el equipo:** Sin gestión de usuarios; elimina la responsabilidad individual.

Decisión: La identidad sale siempre del token autenticado, nunca del cuerpo ni de cabeceras editables. Los roles son REVIEWER, AGENT, OPERATOR y SYSTEM, cada uno con el mínimo privilegio: solo REVIEWER decide, AGENT solo propone y quien propone una versión de mapeo no puede aprobarla.

Consecuencias: Positivas: cada decisión es atribuible a una persona y la separación de funciones es verificable. Negativas: el proyecto necesita un proveedor de identidad real; el prototipo usa tokens de demostración.

Escenario de calidad: Seguridad (autenticidad): ninguna decisión se registra con una identidad distinta de la del token autenticado; un agente nunca aprueba. **Evidencia en**

el prototipo ampliado (commit 0b6406c): pruebas test_without_token_everything_is_401, test_agent_and_operator_cannot_decide, test_impersonation_in_body_has_no_effect, test_headers_do_not_change_identity y proposerCannotApprove.

ADR-12. Integridad de la normativa y de las reglas

Estado: aceptada. **Contexto:** El kit decide qué puede llegar a revisión a partir de la normativa, los catálogos y las reglas. Un documento o una regla alterados, por error o de forma maliciosa, harían que el kit aceptara lo que debía rechazar sin que nadie lo notara.

- **Confianza en el almacenamiento compartido:** Sin costo; un cambio no autorizado pasa inadvertido.
- **Huella por documento, reglas versionadas y cambios revisados (elegida):** Toda alteración es detectable y atribuible; exige disciplina de versiones.
- **Firma digital de cada documento por su emisor:** Máxima garantía; depende de que la autoridad o el proveedor publiquen documentos firmados, algo que el proyecto no controla.

Decisión: Cada documento normativo tiene una huella SHA-256 registrada en la tabla de procedencia; el kit la calcula en cada ejecución y el gate de integración continua la compara con la registrada. Las reglas se versionan y todo cambio en reglas, catálogos o normativa entra por solicitud de cambio aprobada por una persona distinta de quien la propone. En ejecución, la normativa es de solo lectura.

Consecuencias: Positivas: una alteración detiene el pipeline y queda atribuida. Negativas: cada versión normativa exige registrar su huella y revisar las reglas afectadas.

Escenario de calidad: Seguridad (integridad): ningún documento normativo ni regla cambia sin solicitud de cambio revisada por otra persona y sin huella verificada. **Evidencia en el prototipo ampliado (commit 0b6406c):** el kit calcula la huella en cada ejecución y la incluye en cada hallazgo, y el proponente no puede aprobar su propio mapeo; la comparación de huellas en el gate está implementada y probada (ci/gate-normative.sh, 8 de 8 casos); no se ha ejecutado en un servidor Jenkins.

Anexo B. Supuestos de planificación

Los supuestos son del autor. Los costos por rol se calibraron contra los referentes públicos del apartado 4.8; los de IA/ML, QA y especialista tributario no tienen una fuente pública verificada y se declaran como supuestos.

Tabla 53. *Costo empresa mensual por rol (supuesto)*

Rol	COP por mes (tiempo completo)
Gerente del proyecto	18.000.000
Arquitecto de soluciones	24.000.000
Líder técnico de integración	19.000.000
Desarrolladores backend Java (3)	13.000.000
Ingeniero de IA/ML	18.000.000
Ingeniero DevOps/SRE	18.000.000
QA de automatización	10.000.000
Especialista tributario	14.000.000

Fuente: elaboración propia.

Tabla 54. *Otros supuestos del modelo*

Supuesto	Valor
Semanas por mes	4,33
TRM	4.000 COP/USD
Nube (tres entornos)	18.000.000 COP/mes
Licencias (única vez)	25.000.000 COP
Herramientas	3.000.000 COP/mes
Precio LLM	USD 3,00 por millón de tokens de entrada y USD 15,00 de salida
Iteraciones de Monte Carlo y semilla	10.000; semilla 2026
Distribución de duraciones	Beta-PERT ($\lambda = 4$) sobre (o, m, p)
Nivelación	Esquema serial con prioridad por inicio tardío; capacidad por rol según la tabla de equipo

Fuente: elaboración propia.

Anexo C. Resultados detallados del banco de defectos

Tabla 55. *Resultado por mutante*

Mutante	Descripción	Docs. afectados	Esquema	Ejecución	CI	Detección por doc.
D1-a	Factura codificada como nota débito (92), código válido pero equivocado	214	no	sí	sí	100 %
D1-b	Notas crédito codificadas como factura (01)	56	no	sí	sí	100 %
D1-c	Código de tipo de documento inexistente (07) tomado de una nota verbal	214	no	sí	sí	100 %
D1-d	Documento nacional (CC) codificado como empresa (A1)	107	no	sí	sí	100 %
D1-e	Pasaporte codificado como documento de extranjero (A3); formatos idénticos	35	no	no	sí	0 %
D1-f	IVA general (IVA19) codificado como IVA reducido (T02)	272	no	sí	sí	100 %
D1-g	Beta: nota débito con código inexistente (93)	30	no	sí	sí	100 %
D2-a	Cantidad y precio unitario intercambiados (producto idéntico)	300	no	no	sí	0 %
D2-b	Precio unitario y valor de línea intercambiados	295	no	sí	sí	100 %
D2-c	Total a pagar tomado del subtotal	290	no	sí	sí	100 %
D2-d	Impuesto total tomado del subtotal	300	no	sí	sí	100 %
D3-a	Identificador de perfil en minúsculas	300	no	sí	sí	100 %

Mutante	Descripción	Docs. afectados	Esquema	Ejecución	CI	Detección por doc.
D3-b	Nombre del esquema tributario con espacio final	272	no	sí	sí	100 %
D3-c	Nombre del esquema tributario en mayúscula inicial	290	no	sí	sí	100 %
D3-d	Moneda en minúsculas	300	no	sí	sí	100 %
D4-a	Correo del adquirente sin mapear (el canónico antiguo no lo traía)	300	sí	sí	sí	100 %
D4-b	Correo del adquirente mapeado a un literal vacío	300	no	sí	sí	100 %
D4-c	Fecha de emisión sin mapear	300	sí	sí	sí	100 %
D5-a	Se envía LegacyOrderReference, retirado en v2	300	sí	sí	sí	100 %
D5-b	Se envía TaxExchangeRateLegacy, retirado en v2	300	sí	sí	sí	100 %
D6-a	Descripción sin sanear el separador reservado	42	sí	sí	sí	100 %
D6-b	Nombre del adquirente sin sanear el separador reservado	0	no	no	sí	—

Fuente: elaboración propia. Detección por documento: proporción de documentos afectados que el kit en ejecución rechazó. Un mutante sin documentos afectados en la muestra solo puede detectarse con ejemplos validados.

Anexo D. Matriz del estado del arte

Este anexo presenta una selección dirigida de 40 estudios del cruce de brecha y del núcleo analizado, basado en título y resumen. Entraron, en este orden: (1) los 15 registros que, al leer su resumen, combinan realmente dos o más ejes del cruce, 10 de las intersecciones del diccionario base y 5 identificados en la lectura de los trabajos fiscales y de la validación; (2) los 6 que los combinan parcialmente; (3) los registros de facturación o fiscalidad que mencionan modelos de lenguaje o agentes en el título o en las palabras clave; y (4) cuotas por tema sobre el núcleo, por mayor número de ejes y luego año más reciente. La columna «Criterio» indica por cuál entró cada estudio. La información procede del título, el resumen y las palabras clave, sin lectura de textos completos, y «no declarada» significa que el resumen no lo dice. Las fichas bibliográficas completas de los 40 estudios figuran en las Referencias. La matriz completa del núcleo, con 164 registros, se conserva como material de apoyo.

Tabla 56. *Matriz del estado del arte: selección dirigida de 40 estudios*

Referencia	Tema	Técnica	Verificación determinista	Dominio fiscal o regulado	Limitación declarada	Criterio
Pinto et al. (2026), doi:10.1109/ICDEW71238.2026.00023	a	Correspondencias de esquemas generadas por LLM con explicaciones formales verificables por máquina	Sí (explicaciones lógicas verificables)	No	Investigación inicial	1

Referencia	Tema	Técnica	Verificación determinista	Dominio fiscal o regulado	Limitación declarada	Criterio
Lushnei et al. (2026), doi:10.18653/v1/2026.eacl-long.110	a	LLM como oráculo que valida correspondencias inciertas de alineación de ontologías	No (la validación la hace el LLM)	No	No declarada en el resumen	1
He et al. (2025), doi:10.1109/ASEW67777.2025.00040	a	LLM genera fórmulas y predicados para la verificación basada en reglas de transformaciones de mapas	Sí (verificador por reglas en lógica de primer orden)	No	Prototipo evaluado con escenarios sintéticos	1
Vieira-Vieira et al. (2025), doi:10.3389/frai.2025.1636809	a	Plataforma multiagente LLM de integración de datos con RAG y Text-to-SQL	No declarada	Regulado (farmacia)	No declarada en el resumen	1
Shrikaran et al. (2025), doi:10.1109/ICCMC65190.2025.11141002	a	Pipeline ETL con OCR y LLM local que mapea y estructura datos antes de cargarlos	No declarada	Regulado (GDPR y HIPAA)	No declarada en el resumen	1
Ajdinović et al. (2025), doi:10.1109/ICMIMT65123.2025.11091954	a	Emparejamiento semántico con LLM hacia modelos estandarizados (Asset Administration Shell)	No declarada	Regulado (pasaporte de baterías)	No declarada en el resumen	1
Mohan et al. (2026), doi:10.1109/CCWC67433.2026.11393832	a	Agentes LangGraph para resolución de entidades en datos maestros, con revisión humana	Trazas de auditoría deterministas	Regulado (servicios financieros)	No declarada en el resumen	1
Nguyen et al. (2026), doi:10.3390/buildings16030499	c	Asistente GPT con comprobaciones deterministas de códigos y precios contra catálogos oficiales	Sí (igualdad exacta gobernada por reglas)	Regulado (obra pública)	Alcance diagnóstico estrecho; una sola provincia	1

Referencia	Tema	Técnica	Verificación determinista	Dominio fiscal o regulado	Limitación declarada	Criterio
Yu et al. (2026), doi:10.1109/AIoTC71250.2026.11688636	c	Pruebas de cumplimiento de privacidad con LLM, análisis estático y base de conocimiento normativa	Análisis estático y pruebas dinámicas	Regulado (protección de datos)	La IA aún no sustituye el juicio legal final	1
Santos y Muniz Silva (2026), doi:10.1007/s10506-026-09532-7	c	LLM transforma regulaciones financieras en reglas SBVR con trazabilidad al texto normativo	Reglas formales; no declara verificación determinista	Regulado (finanzas)	No declarada en el resumen	1
Gogani Khiabani et al. (2026), doi:10.1145/3744916.3764575	c	Agentes LLM traducen código tributario a software con pruebas metamórficas	Sí (oráculo metamórfico)	Fiscal (EE. UU.)	Tasa de aprobación en el peor caso del 45 %	1
Srinivas et al. (2023), sin DOI	c	LLM genera especificaciones metamórficas en lógica de primer orden para software tributario	Especificaciones formales para pruebas	Fiscal (EE. UU.)	Se declara como análisis de potencial y límites	1
Gao et al. (2026), doi:10.1109/CSCWD68734.2026.11582797	c	Multiagente LLM con razonamiento por reglas y verificación de consistencia para recomendaciones tributarias	Razonamiento por reglas y verificación de consistencia	Fiscal	No declarada en el resumen	1
Vergés et al. (2026), doi:10.1016/j.autcon.2026.106876	c	Agente LLM con revisión humana: lleva documentos a un esquema unificado y ejecuta comprobaciones normativas	Sí (comprobaciones deterministas contra códigos parametrizados)	Regulado (construcción)	No declarada en el resumen	1

Referencia	Tema	Técnica	Verificación determinista	Dominio fiscal o regulado	Limitación declarada	Criterio
Villanueva-Miranda et al. (2026), doi:10.1145/3807503.3819371	a	Agentes LLM generan especificaciones de mapeo de campos; módulos deterministas construyen la representación canónica	Sí (ejecución determinista de especificaciones validadas)	Regulado (datos biomédicos)	No declarada en el resumen	1
Joshi et al. (2026), doi:10.1109/MPCON69668.2026.11508492	b	Agente generativo corregido por un validador simbólico para datos sintéticos de ETL	Sí (validador simbólico)	No	No declarada en el resumen	2
Nie et al. (2024), doi:10.1109/ACAI63924.2024.10899737	d	Agente LLM conversacional para emitir facturas electrónicas con módulo de cumplimiento	No declarada	Fiscal (factura electrónica)	No declarada en el resumen	2
Raza et al. (2026), doi:10.1007/978-3-032-26363-6_11	d	Agentes de IA para descubrir y componer API de facturación electrónica	No declarada	Fiscal (factura electrónica)	No declarada en el resumen	2
Luiz Firmino Alves et al. (2025), doi:10.5220/0013285000003929	a	Vinculación de productos en descripciones de facturas electrónicas con LLM	No declarada	Fiscal (factura electrónica)	No declarada en el resumen	2
Wahed et al. (2026), doi:10.1109/ICIICE69672.2026.11565502	d	Agentes de IA para extraer, validar, conciliar y registrar facturas en el ERP, con revisión humana	Validación de negocio; no declara determinista	Regulado (finanzas sanitarias)	No declarada en el resumen	2
Wang et al. (2025), doi:10.1109/IC3IT66137.2025.11341366	d	Sistema de facturas con OCR y YOLO y validación por reglas según la normativa tributaria	Sí (validación basada en reglas)	Fiscal (factura electrónica)	No declarada en el resumen	2

Referencia	Tema	Técnica	Verificación determinista	Dominio fiscal o regulado	Limitación declarada	Criterio
Zhang et al. (2026), doi:10.1145/3819836.3819971	d	Prompting iterativo para extraer evidencia verificable de créditos fiscales de I+D	Entidades verificables por reglas	Fiscal	No declarada en el resumen	3
Lianghua et al. (2026), doi:10.1016/j.jbusres.2026.116426	d	Medida de identidad política con embeddings de LLM y su relación con el cumplimiento tributario	No	Fiscal	No declarada en el resumen	3
Ren y Chen (2026), doi:10.1145/3806262.3806364	d	Modelo teórico de sistema fiscal inteligente con IA generativa y aprendizaje federado	No declarada	Fiscal	No declarada en el resumen	3
Tudor et al. (2025), doi:10.1109/SYNASC69064.2025.00047	d	LLM adaptado a contabilidad y fiscalidad rumana con RAG	No	Fiscal y contable	No declarada en el resumen	3
Darrazão et al. (2025), doi:10.5753/jbcs.2025.5743	d	Etiquetado de descripciones de productos en facturas electrónicas con LLM frente a CRF	No	Fiscal (factura electrónica)	No declarada en el resumen	3
Arsal et al. (2025), doi:10.1109/FIT67061.2025.11333725	d	Asistente tributario: sistema por reglas, LLM ajustado y RAG con citas a documentos oficiales	No	Fiscal	No declarada en el resumen	3
Lee et al. (2024), doi:10.2308/JETA-2022-046	d	Datos de panel y simulación basada en agentes del efecto de blockchain en el cumplimiento tributario	No	Fiscal	No declarada en el resumen	3

Referencia	Tema	Técnica	Verificación determinista	Dominio fiscal o regulado	Limitación declarada	Criterio
Alquran et al. (2026), doi:10.1109/ICCIAA68481.2026.11543908	a	Arquitectura de referencia y ciclo MLOps/LLMOps para gemelos digitales urbanos	No declarada	Regulado (gobierno urbano)	No declarada en el resumen	4
Fraser et al. (2026), doi:10.1016/j.procs.2026.08.195	a	Informes de emisiones generados con IA generativa a partir de inventarios integrados	No declarada	Regulado (ambiental)	No declarada en el resumen	4
Cheng et al. (2026), doi:10.14569/IJACSA.2026.0170733	c	Revisión sistemática de copilotos clínicos con IA generativa	No declarada	Regulado (salud)	No declarada en el resumen	4
Al Sunny et al. (2026), doi:10.1109/COMPSAC69091.2026.00491	c	Agentes neuro-simbólicos con «determinismo guiado» y text-to-SQL	Sí (aplicación simbólica de restricciones)	No	No declarada en el resumen	4
Gao et al. (2026), doi:10.22260/ISARC2026/0114	a	GraphRAG multimodal que integra datos de residuos con cláusulas normativas citadas	No declarada	Regulado (ambiental)	No declarada en el resumen	4
Abdalla et al. (2026), doi:10.1109/ACCESS.2026.3711515	b	Multiagente LLM que genera modelos Simulink, pruebas y comprobación de cumplimiento	Ejecución de pruebas; no declara determinista	Regulado (automotriz)	No declarada en el resumen	4
Alves et al. (2026), doi:10.1145/3793654.3793749	b	Migración de pruebas de unittest a Pytest con LLM, ejecutadas en las suites reales	Ejecución de pruebas	No	El 51,5 % de las migraciones falla	4
He et al. (2026), doi:10.1016/j.procs.2026.08.177	b	Revisión de LLM y agentes en decisiones de inversión con supervisión humana	No declarada	Regulado (finanzas)	Retos abiertos: anclaje temporal, veracidad multimodal, sesgo y evaluación	4

Referencia	Tema	Técnica	Verificación determinista	Dominio fiscal o regulado	Limitación declarada	Criterio
Abtahi et al. (2026), doi:10.3389/frai.2026.1701665	c	Ensamble de más de 30 LLM que conserva desacuerdos para la supervisión clínica	No	Regulado (salud)	Aún no es una herramienta clínica validada	4
Arjarapu et al. (2026), doi:10.1145/3788853.3801599	a	Integración de entidades con grafo sin fusión, clasificación con LLM y uniones validadas	Validación explícita de uniones	No	No declarada en el resumen	4
Iijima et al. (2026), doi:10.23919/transcom.2026EBP3005	b	Generación de configuraciones de red con LLM, ReAct, multiagente y revisión humana	No declarada	No	Documentos de diseño incompletos producen configuraciones inexactas	4
Adnyana y Schwung (2026), doi:10.1109/CoDIT70676.2026.11630927	c	Prompting estructurado y validación en tres capas para código PLC (IEC 61131-3)	No (validadores LLM y humanos)	Regulado (normas industriales)	No declarada en el resumen	4

Fuente: elaboración propia a partir de registros de Scopus (20 de octubre de 2026). Criterio: 1, combina realmente; 2, combina parcialmente; 3, facturación o fiscalidad con modelos de lenguaje o agentes; 4, cuota por tema. Temas del apartado 2.3.3.

Resumen de la verificación de referencias. La lista final tiene 105 referencias. Las 71 de la lista anterior se contrastaron con fuentes primarias el 20 de octubre de 2026, mediante la resolución del DOI en Crossref o la consulta de la URL citada, y las 26 pendientes se volvieron a consultar de forma automática el 24 de octubre de 2026, sin nuevas confirmaciones. Las 34 añadidas corresponden a estudios del anexo D; sus metadatos se tomaron de Scopus y su DOI se comprobó en doi.org y en Crossref el 24 de octubre de 2026. La tabla 57 resume el resultado. Las cinco referencias dudosas de la tabla 58 quedan pendientes de verificación del autor y no se han corregido.

Tabla 57. *Resumen de la verificación de las referencias*

Indicador	Valor
Referencias en la lista final	105
En español	30
Verificadas por DOI en Crossref	18 (12 de la lista anterior y 6 añadidas)

Indicador	Valor
Estudios del anexo D (tabla 56)	40: 34 añadidas con metadatos tomados de Scopus y 6 que ya figuraban en la lista; DOI resuelto: 39 de 40 (un estudio no tiene DOI en Scopus)
Verificadas por la URL citada (respuesta 200)	17
Pendientes de comprobación manual	26 (libros y normas sin DOI ni URL, enlaces caídos o bloqueados al acceso automático; sin cambios en el nuevo intento automático del 24 de octubre de 2026)
Mejoras de formato propuestas (añadir un DOI existente)	5, no aplicadas
Dudosas, pendientes de verificación del autor	5
Publicadas entre 2022 y 2026	67 (63,8 % del total; 65,7 % de las 102 con fecha)

Fuente: elaboración propia a partir de los registros de Scopus y de consultas a doi.org, a Crossref y a las URL citadas (20 y 24 de octubre de 2026).

Tabla 58. *Referencias dudosas, pendientes de verificación del autor*

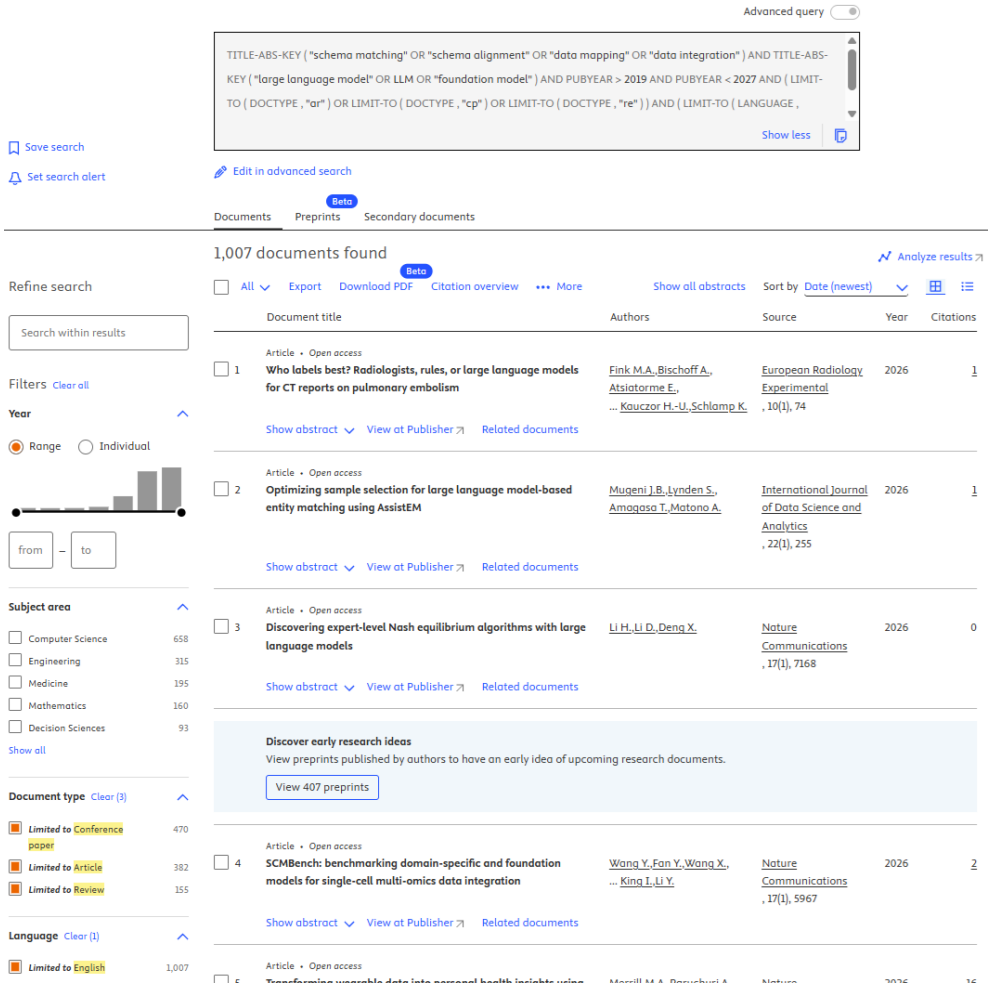
Referencia	Motivo
Colombia Move (2026)	La página responde, pero su título actual es distinto del citado
Hevner et al. (2004)	Crossref indica las páginas 75-106 y la referencia, 75-105
Leguizamón y Lugo (2024)	Crossref registra los apellidos Leguizamón Gaette y Lugo Bareiro
OCDE y Eurostat (2006)	Crossref asocia el Manual de Oslo en español a la OCDE en 2007
Ramírez-Álvarez et al. (2022)	Crossref registra el apellido sin guion

Fuente: elaboración propia.

Anexo E. Evidencia de las búsquedas en Scopus

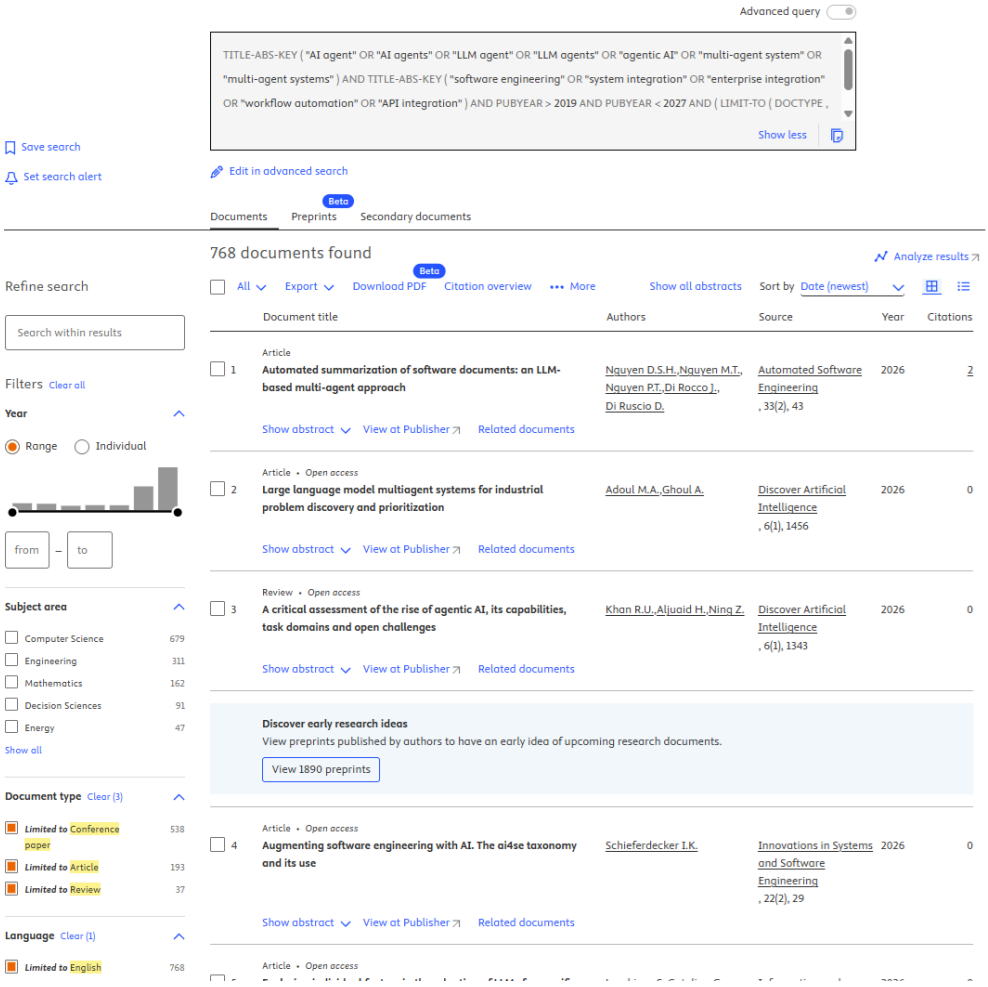
Las figuras de este anexo reproducen, recortadas, capturas de pantalla de Scopus tomadas el 20 de octubre de 2026 durante la ejecución de las búsquedas; se omitió la barra superior de la página. Las cadenas completas figuran en la tabla 10.

Figura 20. *Búsqueda S1 con los filtros de tipo de documento e idioma: 1007 documentos*



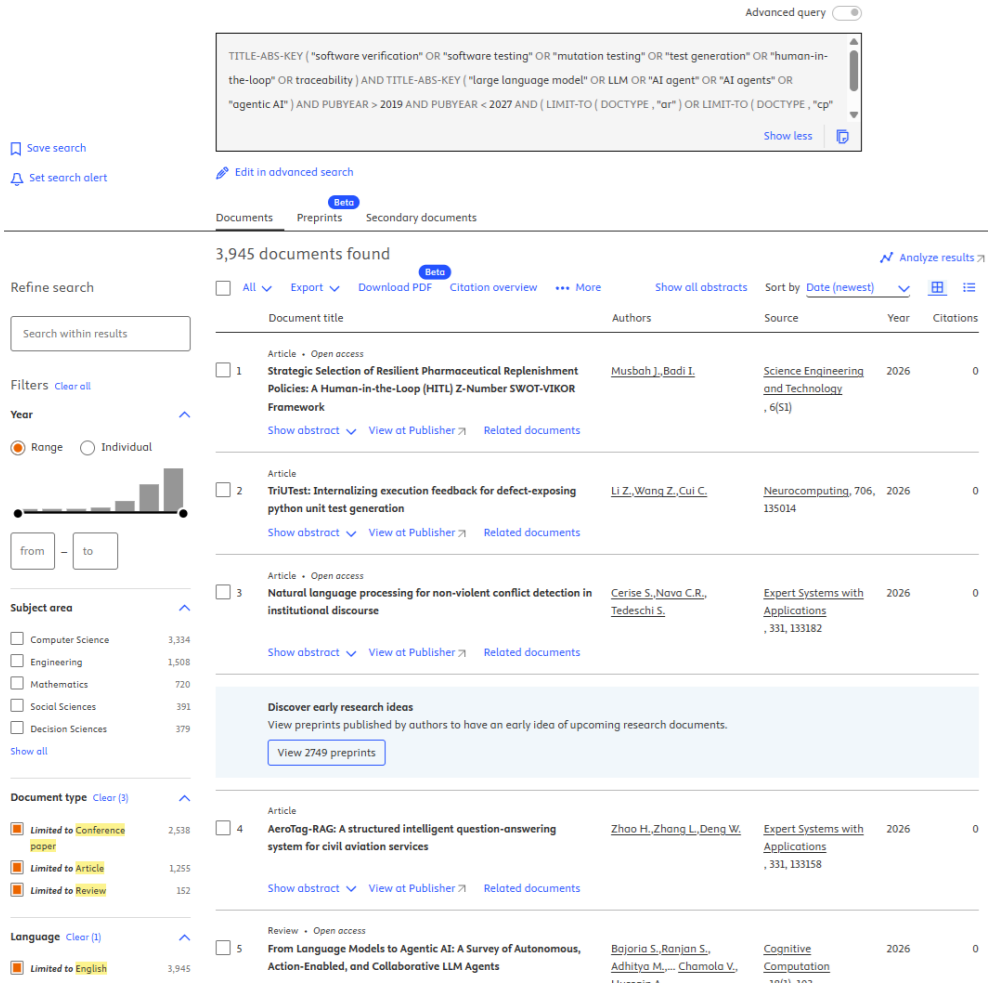
Fuente: elaboración propia a partir de capturas de Scopus del 20 de octubre de 2026.

Figura 21. *Búsqueda S2 con los filtros aplicados: 768 documentos*



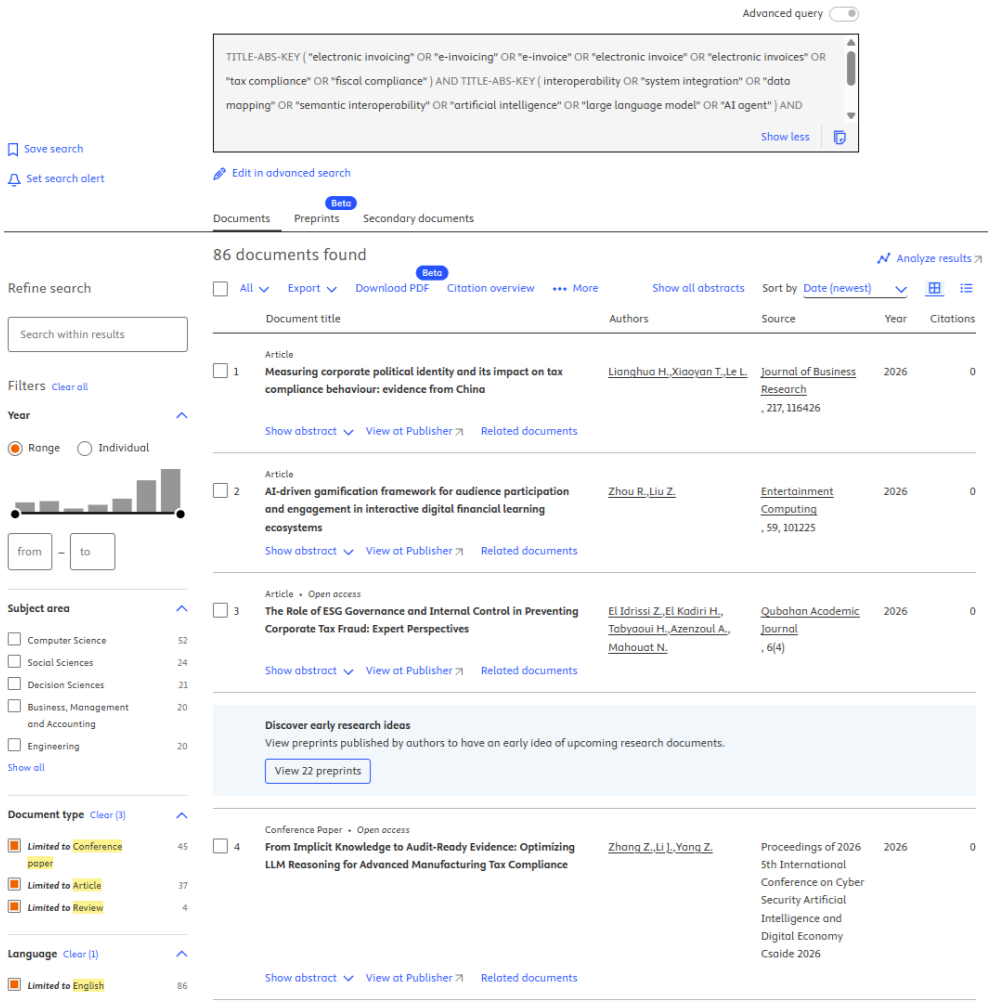
Fuente: elaboración propia a partir de capturas de Scopus del 20 de octubre de 2026.

Figura 22. *Búsqueda S3 con los filtros aplicados: 3945 documentos*



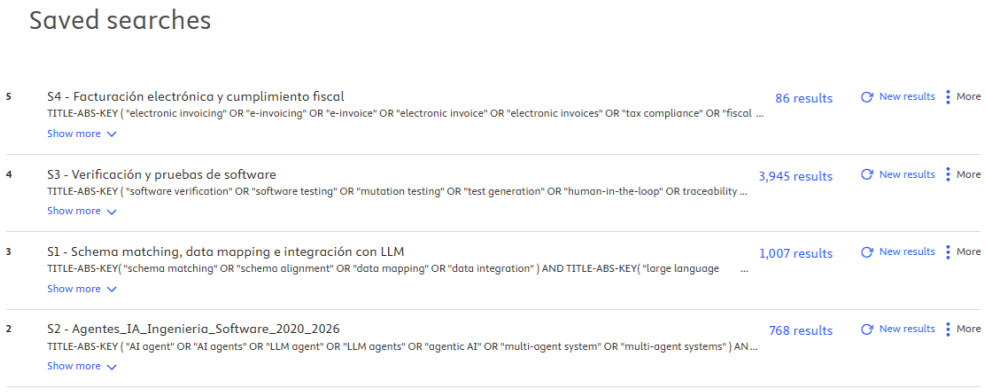
Fuente: elaboración propia a partir de capturas de Scopus del 20 de octubre de 2026.

Figura 23. *Búsqueda S4 con los filtros aplicados: 86 documentos*



Fuente: elaboración propia a partir de capturas de Scopus del 20 de octubre de 2026.

Figura 24. *Búsquedas guardadas S1 a S4 con su número de resultados*



Fuente: elaboración propia a partir de capturas de Scopus del 20 de octubre de 2026.